

System Feature Overview

ID	Node Name	Description	Children IDs	Parent IDs
1.1	Develop and test the LIN protocol stack	This feature implements and tests a LIN (Local Interconnect Network) protocol stack, covering configuration, layer implementation, data validation, hardware interaction, and software testing.	5.1#open_lin_data_layer.h, 5.2#open_lin_data_layer.c, 3.7, 3.8, 2.1, 2.2	N/A
2.1	Implement LIN protocol layers	This parent feature covers the implementation of the various layers of the LIN protocol stack, including the data, network, and transport layers for both master and slave roles.	3.2, 3.3, 3.4, 3.6	1.1
2.2	Manage communication hardware	This parent feature includes functionalities for interacting with underlying communication hardware, such as specific LIN transceivers or general serial ports used for communication or testing.	3.1, 3.5	1.1
3.1	Handle LIN hardware communication	Includes functionalities for low-level control and interaction with specific LIN communication hardware implementations.	4.1#open_lin_hw.h, 4.15#implementation/ST32/open_lin_hw_st32.c, 4.19#implementation/pic16/open_lin_hw_pic16.c	2.2
3.2	Implement LIN data layer for master	Provides the data layer functionality specifically for a LIN master device, managing scheduling and data transfer.	4.3#open_lin_master_data_layer.h, 4.9#open_lin_master_data_layer.c	2.1
3.3	Implement LIN data layer for slave	Includes functionalities for managing a LIN slave device, handling data reception, initialization, and state.	4.7#open_lin_slave_data_layer.c, 4.8#open_lin_slave_data_layer.h, 4.13#implementation/x86/main_slave_example.c	2.1
3.4	Manage LIN network layer	Covers the processing of incoming and outgoing LIN messages and the	4.2#open_lin_network_layer.c, 4.5#open_lin_network_layer.h,	2.1

ID	Node Name	Description	Children IDs	Parent IDs
		handling of LIN frames at the network level.	4.16#implementation/unit_test/test_run.cpp	
3.5	Manage serial port communication	Provides functionalities for managing serial communication ports (e.g., RS232) for specific platform implementations or testing.	4.12#implementation/x86/rs232.h, 4.14#implementation/x86/rs232.c	2.2
3.6	Process LIN transport layer	Handles messages and requests based on the LIN transport layer protocol, including diagnostic services.	4.4#open_lin_transport_layer.h, 4.10#open_lin_transport_layer.c	2.1
3.7	Define and configure LIN protocol	This parent feature includes functionalities for defining the basic data types and constants of the LIN protocol and configuring the Open LIN stack.	4.6#open_lin_types.h, 4.11#open_lin_cfg.h	1.1
3.8	Implement software testing	This parent feature includes functionalities for executing software tests, reporting results, and simulating hardware for testing the LIN protocol stack.	4.17#implementation/unit_test/catch.hpp, 4.18#implementation/unit_test/open_lin_frame_winOS.c	1.1
4.1#open_lin_hw.h	Manage LIN communication	Handles low-level control and interaction with the LIN communication hardware.	6.6#open_lin_hw.h, 5.3#open_lin_hw.h, 5.4#open_lin_hw.h	3.1
4.2#open_lin_network_layer.c	Process incoming LIN messages	Handles messages received from the LIN network, using their identifier to find relevant configuration.	6.15#open_lin_network_layer.c, 5.5#open_lin_network_layer.c	3.4
4.3#open_lin_master_data_layer.h	Manage LIN communication as master	This feature provides the data layer functionality for a LIN master device, handling tasks like scheduling, receiving data, processing events, and transmitting frames.	6.21#open_lin_master_data_layer.h, 5.6#open_lin_master_data_layer.h, 5.7#open_lin_master_data_layer.h, 5.8#open_lin_master_data_layer.h	3.2

ID	Node Name	Description	Children IDs	Parent IDs
4.4#open_lin_transport_layer.h	Process LIN Transport Layer Messages	This feature processes messages based on the LIN transport layer protocol, managing service identifiers and frame types.	6.24#open_lin_transport_layer.h, 6.32#open_lin_transport_layer.h, 5.9#open_lin_transport_layer.h	3.6
4.5#open_lin_network_layer.h	Manage LIN network frames	This feature handles and manages LIN (Local Interconnect Network) frames within the network communication layer.	6.36#open_lin_network_layer.h, 5.10#open_lin_network_layer.h, 5.11#open_lin_network_layer.h	3.4
4.6#open_lin_types.h	Define LIN protocol basics	This feature defines the fundamental data types, constants, and error codes necessary for the LIN communication protocol.	5.12#open_lin_types.h, 5.13#open_lin_types.h, 5.14#open_lin_types.h	3.7
4.7#open_lin_slave_data_layer.c	Handle incoming LIN messages	Processes messages received over a LIN bus when configured as a slave device.	6.49#open_lin_slave_data_layer.c, 5.15#open_lin_slave_data_layer.c, 5.16#open_lin_slave_data_layer.c, 5.17#open_lin_slave_data_layer.c, 5.18#open_lin_slave_data_layer.c	3.3
4.8#open_lin_slave_data_layer.h	Manage LIN slave communication	Initializes, resets, and handles data reception for a LIN slave component.	6.59#open_lin_slave_data_layer.h, 5.19#open_lin_slave_data_layer.h	3.3
4.9#open_lin_master_data_layer.c	Control LIN master communication	This feature initializes and handles the timing and data transfer for a LIN master node based on a communication schedule.	6.64#open_lin_master_data_layer.c, 5.20#open_lin_master_data_layer.c, 5.21#open_lin_master_data_layer.c, 5.22#open_lin_master_data_layer.c	3.2
4.10#open_lin_transport_layer.c	Handle LIN diagnostic requests	This feature processes incoming LIN diagnostic requests, identifies the service, and dispatches it to the correct handler.	6.71#open_lin_transport_layer.c, 5.23#open_lin_transport_layer.c	3.6
4.11#open_lin_cfg.h	Configure Open LIN stack features	Configures various settings and enables specific features for the Open LIN communication stack.	6.76#open_lin_cfg.h, 6.78#open_lin_cfg.h, 5.24#open_lin_cfg.h	3.7
4.12#implementation/x86/rs232.h	Communicate using serial ports	This feature manages communication with devices	6.80#implementation/x86/rs232.h, 5.25#implementation/x86/rs232.h,	3.5

ID	Node Name	Description	Children IDs	Parent IDs
		connected via serial ports (like RS-232).	5.26#implementation/x86/rs232.h	
4.13#implementation/x86/main_slave_example.c	Initialize LIN slave node	This feature initializes a LIN slave node, setting up its communication parameters, frame slots, and data buffers based on the provided configuration.	6.90#implementation/x86/main_slave_example.c, 5.27#implementation/x86/main_slave_example.c, 5.28#implementation/x86/main_slave_example.c, 5.29#implementation/x86/main_slave_example.c	3.3
4.14#implementation/x86/rs232.c	Control RS232 ports	This feature allows managing serial communication ports (RS232), including opening, closing, configuring, reading, and writing data.	5.30#implementation/x86/rs232.c, 5.31#implementation/x86/rs232.c, 5.32#implementation/x86/rs232.c	3.5
4.15#implementation/ST32/open_lin_hw_st32.c	Manage LIN communication	This feature controls the LIN hardware interface for sending and receiving data and managing break signals.	6.102#implementation/ST32/open_lin_hw_st32.c, 5.33#implementation/ST32/open_lin_hw_st32.c, 5.34#implementation/ST32/open_lin_hw_st32.c, 5.35#implementation/ST32/open_lin_hw_st32.c	3.1
4.16#implementation/unit_test/test_run.cpp	Process incoming LIN messages	Tests the ability of a LIN slave module to receive, validate, and process incoming LIN messages and detect errors.	6.116#implementation/unit_test/test_run.cpp, 5.36#implementation/unit_test/test_run.cpp, 5.37#implementation/unit_test/test_run.cpp, 5.38#implementation/unit_test/test_run.cpp, 5.39#implementation/unit_test/test_run.cpp	3.4
4.17#implementation/unit_test/catch.hpp	Run software tests and report outcomes	This feature executes software tests, captures the results of assertions and sections, manages test configurations, and reports the detailed outcomes using various formats and configurations.	6.131#implementation/unit_test/catch.hpp, 5.40#implementation/unit_test/catch.hpp, 5.41#implementation/unit_test/catch.hpp	3.8
4.18#implementation/unit_test/open_lin_frame_winOS.c	Simulate LIN hardware	Simulates LIN hardware interactions for testing the LIN protocol stack without physical hardware.	6.139#implementation/unit_test/open_lin_frame_winOS.c, 5.42#implementation/unit_test/open_lin_frame_winOS.c, 5.43#implementation/unit_test/open_lin_frame_winOS.c, 5.44#implementation/unit_test/open_lin_frame_winOS.c	3.8
4.19#implementation/pic16/open_lin_hw_pic16	Manage LIN communication	This feature handles sending and	6.141#implementation/pic16/open_lin_hw_pic16	3.1

ID	Node Name	Description	Children IDs	Parent IDs
c16/open_lin_hw_pic16.c		receiving data over the LIN bus using hardware peripherals.	.c, 6.143#implementation/pic16/open_lin_hw_pic16.c, 6.146#implementation/pic16/open_lin_hw_pic16.c, 5.45#implementation/pic16/open_lin_hw_pic16.c	
5.1#open_lin_data_layer.h	Manage LIN frame structure and validation	This parent feature defines the structure used for LIN frames and includes functionalities for calculating checksum and parity bits, crucial for validating LIN frames within the LIN data layer.	6.1#open_lin_data_layer.h, 6.2#open_lin_data_layer.h, 6.3#open_lin_data_layer.h	1.1
5.2#open_lin_data_layer.c	Calculate LIN protocol values	This parent feature includes functionalities related to calculating specific values required by the LIN protocol, such as checksums and PID parity bits, for data integrity and validation.	6.4#open_lin_data_layer.c, 6.5#open_lin_data_layer.c	1.1
5.3#open_lin_hw.h	Initialize LIN hardware	This parent feature includes functionalities related to initializing, resetting, and configuring the LIN hardware, including setting receiver status and initiating auto baud.	6.7#open_lin_hw.h, 6.8#open_lin_hw.h, 6.9#open_lin_hw.h, 6.10#open_lin_hw.h	4.1#open_lin_hw.h
5.4#open_lin_hw.h	Transmit LIN data	This parent feature includes functionalities for transmitting various types of data and signals over the LIN hardware, such as break signals, single bytes, and blocks of data.	6.11#open_lin_hw.h, 6.12#open_lin_hw.h, 6.13#open_lin_hw.h	4.1#open_lin_hw.h
5.5#open_lin_network_layer.c	Process incoming LIN frames	Manages the handling and processing of received LIN frames, including retrieving the corresponding frame slot based on the PID.	6.14#open_lin_network_layer.c, 6.16#open_lin_network_layer.c	4.2#open_lin_network_layer.c
5.6#open_lin_master_data_layer.h	Configure LIN Master Data Layer	Includes defining the necessary data structures and performing initial setup for the LIN master data	6.17#open_lin_master_data_layer.h, 6.19#open_lin_master_data_layer.h	4.3#open_lin_master_data_layer.h

ID	Node Name	Description	Children IDs	Parent IDs
		layer, particularly related to the frame schedule.		
5.7#open_lin_master_data_layer.h	Handle LIN Frame Transmission	Manages the process of sending LIN frames, including transmitting the header and the subsequent data and checksum.	6.22#open_lin_master_data_layer.h, 6.23#open_lin_master_data_layer.h	4.3#open_lin_master_data_layer.h
5.8#open_lin_master_data_layer.h	Process Received LIN Data	Includes processing individual bytes received from the LIN bus and providing notification once a complete frame has been successfully received.	6.18#open_lin_master_data_layer.h, 6.20#open_lin_master_data_layer.h	4.3#open_lin_master_data_layer.h
5.9#open_lin_transport_layer.h	Define LIN Transport Layer Elements	This parent feature includes the definition of various types, structures, and enumerations essential for representing and processing LIN transport layer messages, such as Network Address (NAD), Service Identifier (SID), Protocol Control Information (PCI), frame types, and specific message structures.	6.25#open_lin_transport_layer.h, 6.26#open_lin_transport_layer.h, 6.27#open_lin_transport_layer.h, 6.28#open_lin_transport_layer.h, 6.29#open_lin_transport_layer.h, 6.30#open_lin_transport_layer.h, 6.31#open_lin_transport_layer.h	4.4#open_lin_transport_layer.h
5.10#open_lin_network_layer.h	Manage LIN frame configurations	This parent feature includes functionalities related to defining and managing data structures for LIN frame slots and dynamic ID translation.	6.34#open_lin_network_layer.h, 6.37#open_lin_network_layer.h, 6.38#open_lin_network_layer.h	4.5#open_lin_network_layer.h
5.11#open_lin_network_layer.h	Process received LIN frames	This parent feature includes functionalities related to handling and processing incoming LIN frames upon reception.	6.33#open_lin_network_layer.h, 6.35#open_lin_network_layer.h	4.5#open_lin_network_layer.h
5.12#open_lin_types.h	Define LIN Constants and Macros	This parent feature includes the definition of constants for frame structure, diagnostic IDs, boolean values, and utility macros for bit manipulation and memory	6.41#open_lin_types.h, 6.42#open_lin_types.h, 6.44#open_lin_types.h, 6.47#open_lin_types.h	4.6#open_lin_types.h

ID	Node Name	Description	Children IDs	Parent IDs
		operations.		
5.13#open_lin_types.h	Define LIN Data Types and Structures	This parent feature includes the definition of basic C types, specific LIN protocol types like PID and checksum, and enumerations for frame types.	6.40#open_lin_types.h, 6.45#open_lin_types.h, 6.46#open_lin_types.h	4.6#open_lin_types.h
5.14#open_lin_types.h	Implement LIN Error Handling	This parent feature includes the definition of an enumeration for LIN error codes and the declaration of a function prototype for handling these errors.	6.39#open_lin_types.h, 6.43#open_lin_types.h	4.6#open_lin_types.h
5.15#open_lin_slave_data_layer.c	Handle Data and Checksum Transfer	This parent feature manages the transfer of data bytes and the checksum for a LIN frame, covering both receiving data and validating the checksum for RX frames, and calculating the checksum and transmitting data for TX frames.	6.55#open_lin_slave_data_layer.c, 6.57#open_lin_slave_data_layer.c	4.7#open_lin_slave_data_layer.c
5.16#open_lin_slave_data_layer.c	Initialize and Reset Slave State	This parent feature includes functionalities for initializing the LIN slave module upon startup and resetting its internal state machine and data counters to a default idle state.	6.51#open_lin_slave_data_layer.c, 6.56#open_lin_slave_data_layer.c	4.7#open_lin_slave_data_layer.c
5.17#open_lin_slave_data_layer.c	Manage Slave State Machine	This parent feature oversees the state transitions of the LIN slave's data layer state machine based on received bytes and events, ensuring correct processing of different parts of the LIN frame (Break, SYNC, PID, Data, Checksum) and handling errors for invalid states.	6.50#open_lin_slave_data_layer.c, 6.52#open_lin_slave_data_layer.c	4.7#open_lin_slave_data_layer.c
5.18#open_lin_slave_data_layer.c	Process LIN Frame Header	This parent feature handles the processing of the initial part of a LIN frame, including detecting and	6.48#open_lin_slave_data_layer.c, 6.53#open_lin_slave_data_layer.c, 6.54#open_lin_slave_data_layer.c	4.7#open_lin_slave_data_layer.c

ID	Node Name	Description	Children IDs	Parent IDs
		handling the break signal, validating the synchronization byte (SYNC), and receiving and validating the protected identifier (PID).		
5.19#open_lin_slave_data_layer.h	Manage LIN slave component state	Includes functionalities for initializing, resetting, and defining the operational states of the LIN slave component.	6.58#open_lin_slave_data_layer.h, 6.60#open_lin_slave_data_layer.h, 6.61#open_lin_slave_data_layer.h	4.8#open_lin_slave_data_layer.h
5.20#open_lin_master_data_layer.c	Handle LIN data transmission	This includes the functionalities for transmitting the LIN frame header and the data payload.	6.68#open_lin_master_data_layer.c, 6.69#open_lin_master_data_layer.c	4.9#open_lin_master_data_layer.c
5.21#open_lin_master_data_layer.c	Manage LIN master state and schedule	This includes the core state machine execution, handling timing, state transitions, and managing the frame schedule index.	6.62#open_lin_master_data_layer.c, 6.65#open_lin_master_data_layer.c, 6.66#open_lin_master_data_layer.c	4.9#open_lin_master_data_layer.c
5.22#open_lin_master_data_layer.c	Report LIN communication errors	This feature handles and reports various communication errors encountered during LIN master operation, such as checksum mismatches, transmission failures, and reception timeouts, by calling a global error handler.	6.63#open_lin_master_data_layer.c, 6.67#open_lin_master_data_layer.c	4.9#open_lin_master_data_layer.c
5.23#open_lin_transport_layer.c	Process diagnostic requests	Handles incoming LIN diagnostic requests, checks the Node Address (NAD), registers and dispatches to specific service ID (SID) handlers, including specific service implementations like Assign NAD.	6.70#open_lin_transport_layer.c, 6.72#open_lin_transport_layer.c, 6.73#open_lin_transport_layer.c	4.10#open_lin_transport_layer.c
5.24#open_lin_cfg.h	Configure LIN protocol behavior	Includes configuration options for the LIN protocol stack, such as frame synchronization, network layer addressing, and transport layer enabling.	6.74#open_lin_cfg.h, 6.75#open_lin_cfg.h, 6.77#open_lin_cfg.h	4.11#open_lin_cfg.h
5.25#implementation/x	Manage Serial Port State	This parent feature includes	6.79#implementation/x86/rs232.h,	4.12#implementation/x

ID	Node Name	Description	Children IDs	Parent IDs
86/rs232.h		functionalities for controlling and monitoring the state and configuration of the serial communication port, including connection, control lines, break condition, and data buffers.	6.81#implementation/x86/rs232.h, 6.82#implementation/x86/rs232.h, 6.83#implementation/x86/rs232.h, 6.84#implementation/x86/rs232.h	86/rs232.h
5.26#implementation/x86/rs232.h	Perform Serial Data Transfer	This parent feature includes functionalities for transmitting and receiving data over the serial communication port.	6.85#implementation/x86/rs232.h, 6.86#implementation/x86/rs232.h	4.12#implementation/x86/rs232.h
5.27#implementation/x86/main_slave_example.c	Configure LIN Frame Handling	Sets up the parameters and data buffers for the LIN slave to manage incoming and outgoing frames based on their identifiers and types.	6.87#implementation/x86/main_slave_example.c, 6.88#implementation/x86/main_slave_example.c	4.13#implementation/x86/main_slave_example.c
5.28#implementation/x86/main_slave_example.c	Implement LIN Callbacks	Provides callback functions required by the LIN stack to notify the application of specific events, such as received frames or unused master functions.	6.89#implementation/x86/main_slave_example.c, 6.92#implementation/x86/main_slave_example.c	4.13#implementation/x86/main_slave_example.c
5.29#implementation/x86/main_slave_example.c	Process Incoming LIN Data	Implements the logic for handling data received from the LIN bus, from processing individual bytes to reacting to the completion of full frames.	6.91#implementation/x86/main_slave_example.c, 6.89#implementation/x86/main_slave_example.c	4.13#implementation/x86/main_slave_example.c
5.30#implementation/x86/rs232.c	Control serial line state	Set and query the state of the serial communication line, including break conditions and hardware flow control signals.	6.93#implementation/x86/rs232.c, 6.94#implementation/x86/rs232.c	4.14#implementation/x86/rs232.c
5.31#implementation/x86/rs232.c	Manage serial port connections and resources	Handle opening, closing, and identifying serial ports and managing their internal buffers.	6.95#implementation/x86/rs232.c, 6.96#implementation/x86/rs232.c	4.14#implementation/x86/rs232.c
5.32#implementation/x86/rs232.c	Transfer data via serial port	Send and receive data over the serial connection.	6.97#implementation/x86/rs232.c, 6.98#implementation/x86/rs232.c	4.14#implementation/x86/rs232.c

ID	Node Name	Description	Children IDs	Parent IDs
5.33#implementation/S T32/open_lin_hw_st32. c	Configure and Initialize LIN Hardware	Includes functionalities for initializing, resetting, enabling/disabling the LIN hardware and configuring features like auto-baud.	6.100#implementation/ST32/open_lin_hw_st32.c, 6.101#implementation/ST32/open_lin_hw_st32.c, 6.104#implementation/ST32/open_lin_hw_st32.c	4.15#implementation/S T32/open_lin_hw_st32. c
5.34#implementation/S T32/open_lin_hw_st32. c	Receive LIN Data and Detect Signals	Handles the reception of LIN data, detection of specific LIN signals like the break character, and processing received frames.	6.99#implementation/ST32/open_lin_hw_st32.c, 6.103#implementation/ST32/open_lin_hw_st32.c	4.15#implementation/S T32/open_lin_hw_st32. c
5.35#implementation/S T32/open_lin_hw_st32. c	Transmit LIN Messages and Signals	Covers features related to sending data blocks, single bytes, or special LIN break characters over the hardware.	6.105#implementation/ST32/open_lin_hw_st32.c, 6.106#implementation/ST32/open_lin_hw_st32.c, 6.107#implementation/ST32/open_lin_hw_st32.c	4.15#implementation/S T32/open_lin_hw_st32. c
5.36#implementation/u nit_test/test_run.cpp	Configure test data and structures	This parent feature includes functionalities related to defining and configuring data buffers and frame slot structures used in the unit tests.	6.108#implementation/unit_test/test_run.cpp, 6.109#implementation/unit_test/test_run.cpp, 6.113#implementation/unit_test/test_run.cpp	4.16#implementation/u nit_test/test_run.cpp
5.37#implementation/u nit_test/test_run.cpp	Setup testing environment and dependencies	This parent feature includes the setup of the testing framework, inclusion of necessary headers, and declaration of external components and simulation variables for the unit tests.	6.110#implementation/unit_test/test_run.cpp, 6.111#implementation/unit_test/test_run.cpp, 6.112#implementation/unit_test/test_run.cpp, 6.114#implementation/unit_test/test_run.cpp, 6.115#implementation/unit_test/test_run.cpp	4.16#implementation/u nit_test/test_run.cpp
5.38#implementation/u nit_test/test_run.cpp	Test LIN protocol calculation logic	This parent feature includes unit tests for verifying the correctness of core LIN protocol calculations such as checksum and PID parity.	6.118#implementation/unit_test/test_run.cpp, 6.121#implementation/unit_test/test_run.cpp	4.16#implementation/u nit_test/test_run.cpp
5.39#implementation/u nit_test/test_run.cpp	Test LIN slave state machine and frame handling	This parent feature includes comprehensive unit tests for the LIN slave's operational behavior, including frame reception and transmission, break detection, reset functionality, error handling (checksum, parity), and dynamic ID	6.117#implementation/unit_test/test_run.cpp, 6.119#implementation/unit_test/test_run.cpp, 6.120#implementation/unit_test/test_run.cpp, 6.122#implementation/unit_test/test_run.cpp, 6.123#implementation/unit_test/test_run.cpp, 6.124#implementation/unit_test/test_run.cpp, 6.125#implementation/unit_test/test_run.cpp,	4.16#implementation/u nit_test/test_run.cpp

ID	Node Name	Description	Children IDs	Parent IDs
		translation.	6.126#implementation/unit_test/test_run.cpp	
5.40#implementation/unit_test/catch.hpp	Configure Library and Provide Support	This parent feature includes functionalities for setting up and managing the testing library's configuration and internal state, and providing general-purpose utility functions to support test writing and execution.	6.128#implementation/unit_test/catch.hpp, 6.130#implementation/unit_test/catch.hpp	4.17#implementation/unit_test/catch.hpp
5.41#implementation/unit_test/catch.hpp	Execute and Manage Test Runs	This parent feature encompasses the core process of running tests, including discovering, filtering, executing test cases, handling assertions, managing test flow, and dealing with runtime errors and exceptions.	6.127#implementation/unit_test/catch.hpp, 6.129#implementation/unit_test/catch.hpp, 7.48#implementation/unit_test/catch.hpp	4.17#implementation/unit_test/catch.hpp
5.42#implementation/unit_test/open_lin_frame_winOS.c	Simulate LIN hardware reception and events	Provides simulation for detecting LIN break signals and stores data from simulated received LIN frames via callback.	6.134#implementation/unit_test/open_lin_frame_winOS.c, 6.135#implementation/unit_test/open_lin_frame_winOS.c	4.18#implementation/unit_test/open_lin_frame_winOS.c
5.43#implementation/unit_test/open_lin_frame_winOS.c	Simulate LIN hardware transmissions	Implements simulation of sending LIN data, including break signals and individual bytes, and manages the internal buffer storing the simulated transmitted data.	6.136#implementation/unit_test/open_lin_frame_winOS.c, 6.137#implementation/unit_test/open_lin_frame_winOS.c, 6.138#implementation/unit_test/open_lin_frame_winOS.c	4.18#implementation/unit_test/open_lin_frame_winOS.c
5.44#implementation/unit_test/open_lin_frame_winOS.c	Simulate LIN errors and resets	This parent feature includes functionalities for simulating specific LIN hardware or system events like errors and resets for unit testing purposes.	6.132#implementation/unit_test/open_lin_frame_winOS.c, 6.133#implementation/unit_test/open_lin_frame_winOS.c	4.18#implementation/unit_test/open_lin_frame_winOS.c
5.45#implementation/pic16/open_lin_hw_pic16.c	Manage LIN Hardware Communication	This parent feature includes all functionalities related to interacting directly with the LIN hardware for sending, receiving, controlling, and checking status.	6.140#implementation/pic16/open_lin_hw_pic16.c, 6.142#implementation/pic16/open_lin_hw_pic16.c, 6.144#implementation/pic16/open_lin_hw_pic16.c, 6.145#implementation/pic16/open_lin_hw_pic16.c, 6.147#implementation/pic16/open_lin_hw_pic16.c, 6.148#implementation/pic16/open_lin_hw_pic16.c	4.19#implementation/pic16/open_lin_hw_pic16.c

ID	Node Name	Description	Children IDs	Parent IDs
			n_lin_hw_pic16.c	
6.1#open_lin_data_layer.h	Declare function for frame checksum calculation	Declares an external function used to compute the checksum for a LIN frame based on its PID, length, and data.	CS-3.1#open_lin_data_layer.h	5.1#open_lin_data_layer.h
6.2#open_lin_data_layer.h	Declare function for PID parity calculation	Declares an external function responsible for calculating the parity bits for a given LIN PID.	CS-3.2#open_lin_data_layer.h	5.1#open_lin_data_layer.h
6.3#open_lin_data_layer.h	Define LIN data layer frame structure	Defines the structure for a LIN frame at the data layer, including fields for PID, data length, data pointer, and checksum.	CS-3.3#open_lin_data_layer.h	5.1#open_lin_data_layer.h
6.4#open_lin_data_layer.c	Calculate LIN frame checksum	This function calculates the checksum for a LIN frame. It applies the classic checksum method (starting sum from 0) for diagnostic frames (PID 0x3C and 0x3D) and the enhanced checksum method (starting sum from the PID with parity) for other frames. It iteratively sums the data bytes, handles overflow by subtracting 0xFF, and returns the bitwise NOT of the final sum as per the LIN protocol standard.	CS-3.4#open_lin_data_layer.c	5.2#open_lin_data_layer.c
6.5#open_lin_data_layer.c	Calculate LIN PID parity	This function calculates the parity bits (P0 and P1) for a given LIN Protocol Identifier (PID) and combines them with the original ID, following the LIN protocol specification. This is used for basic PID validation.	CS-3.5#open_lin_data_layer.c	5.2#open_lin_data_layer.c
6.6#open_lin_hw.h	Check for break signal	Checks if break signal occurred on UART	CS-3.6#open_lin_hw.h	4.1#open_lin_hw.h

ID	Node Name	Description	Children IDs	Parent IDs
6.7#open_lin_hw.h	Initialize hardware	Initializes the LIN hardware.	CS-3.7#open_lin_hw.h	5.3#open_lin_hw.h
6.8#open_lin_hw.h	Reset hardware	resets hardware to default state	CS-3.8#open_lin_hw.h	5.3#open_lin_hw.h
6.9#open_lin_hw.h	Set RX enabled status	Sets the enabled status for the receiver hardware.	CS-3.9#open_lin_hw.h	5.3#open_lin_hw.h
6.10#open_lin_hw.h	Start auto baud procedure	Starts auto baud procedure	CS-3.10#open_lin_hw.h	5.3#open_lin_hw.h
6.11#open_lin_hw.h	Transmit break signal	Transmits a break signal over the LIN hardware.	CS-3.11#open_lin_hw.h	5.4#open_lin_hw.h
6.12#open_lin_hw.h	Transmit byte	Transmits a single byte over the LIN hardware.	CS-3.12#open_lin_hw.h	5.4#open_lin_hw.h
6.13#open_lin_hw.h	Transmit data	Transmits a block of data over the LIN hardware.	CS-3.13#open_lin_hw.h	5.4#open_lin_hw.h
6.14#open_lin_network_layer.c	Handle received LIN frames	This function serves as a handler for received LIN frames. It uses the received PID to find the corresponding frame slot and then calls a callback function (open_lin_on_rx_frame) to process the received data.	CS-3.14#open_lin_network_layer.c	5.5#open_lin_network_layer.c
6.15#open_lin_network_layer.c	Initialize the LIN network layer	This function initializes the LIN network layer by setting the frame slot array, its length, and enabling hardware reception.	CS-3.15#open_lin_network_layer.c	4.2#open_lin_network_layer.c
6.16#open_lin_network_layer.c	Retrieve a frame slot by PID	This function searches the initialized frame slot array for a slot matching the given Protocol Identifier (PID). It supports optional dynamic PID translation if OPEN_LIN_DYNAMIC_IDS is defined.	CS-3.16#open_lin_network_layer.c	5.5#open_lin_network_layer.c
6.17#open_lin_master_data_layer.h	Define master frame table item structure	Defines the structure used to represent an entry in the master's	CS-3.17#open_lin_master_data_layer.h	5.6#open_lin_master_data_layer.h

ID	Node Name	Description	Children IDs	Parent IDs
		schedule table, specifying frame timing (offset, response wait) and frame content.		
6.18#open_lin_master_data_layer.h	Handle received byte	Processes a single incoming byte received on the LIN bus as part of the data reception mechanism.	CS-3.18#open_lin_master_data_layer.h	5.8#open_lin_master_data_layer.h
6.19#open_lin_master_data_layer.h	Initialize master data layer	Initializes the LIN master data layer, typically by setting up the master's frame schedule table.	CS-3.19#open_lin_master_data_layer.h	5.6#open_lin_master_data_layer.h
6.20#open_lin_master_data_layer.h	Notify completion of frame reception	Provides a callback mechanism to notify the higher layer when a frame has been successfully received by the master.	CS-3.20#open_lin_master_data_layer.h	5.8#open_lin_master_data_layer.h
6.21#open_lin_master_data_layer.h	Process data layer tasks periodically	Handles periodic tasks for the master data layer, likely managing the schedule table, triggering frame transmissions, and handling timeouts.	CS-3.21#open_lin_master_data_layer.h	4.3#open_lin_master_data_layer.h
6.22#open_lin_master_data_layer.h	Transmit frame data and checksum	Handles the transmission of the data bytes and the checksum for a given LIN frame slot after the header has been sent.	CS-3.22#open_lin_master_data_layer.h	5.7#open_lin_master_data_layer.h
6.23#open_lin_master_data_layer.h	Transmit frame header	Handles the transmission of the header part of a LIN frame (sync break, sync field, identifier) for a given frame slot.	CS-3.23#open_lin_master_data_layer.h	5.7#open_lin_master_data_layer.h
6.24#open_lin_transport_layer.h	Declare Transport Layer Handling Function	Declares the main function responsible for processing LIN transport layer messages.	CS-3.24#open_lin_transport_layer.h	4.4#open_lin_transport_layer.h
6.25#open_lin_transport_layer.h	Define LIN Transport Layer Types	Defines aliases for Network Address (NAD) and Service Identifier (SID) types as unsigned 8-bit integers.	CS-3.25#open_lin_transport_layer.h	5.9#open_lin_transport_layer.h

ID	Node Name	Description	Children IDs	Parent IDs
6.26#open_lin_transport_layer.h	Define PCI Frame Types	Enumerates the different types of LIN transport layer frames based on the PCI field (Single Frame, First Frame, Consecutive Frame).	CS-3.26#open_lin_transport_layer.h	5.9#open_lin_transport_layer.h
6.27#open_lin_transport_layer.h	Define Protocol Control Information Structure	Defines the structure for Protocol Control Information (PCI) used in transport layer frames, including additional data and PCI type.	CS-3.27#open_lin_transport_layer.h	5.9#open_lin_transport_layer.h
6.28#open_lin_transport_layer.h	Define Single Frame Request Structure	Defines the structure for a LIN diagnostic Single Frame (SF) request message, including NAD, PCI, SID, and data payload.	CS-3.28#open_lin_transport_layer.h	5.9#open_lin_transport_layer.h
6.29#open_lin_transport_layer.h	Define Single Frame Response Structure	Defines the structure for a LIN diagnostic Single Frame (SF) response message, including NAD, PCI, response SID (RSID), and data payload.	CS-3.29#open_lin_transport_layer.h	5.9#open_lin_transport_layer.h
6.30#open_lin_transport_layer.h	Define Special Network Address (NAD) Values	Enumerates special Network Address values for SLEEP, FUNCTIONAL, and BROADCAST communication.	CS-3.30#open_lin_transport_layer.h	5.9#open_lin_transport_layer.h
6.31#open_lin_transport_layer.h	Define Standard LIN Service Identifiers (SIDs)	Enumerates standard diagnostic service identifiers used in the LIN transport layer.	CS-3.31#open_lin_transport_layer.h	5.9#open_lin_transport_layer.h
6.32#open_lin_transport_layer.h	Enable/Disable Transport Layer	Allows conditional compilation of the LIN transport layer definition based on the OPEN_LIN_TRANSPORT_LAYER macro.	CS-3.32#open_lin_transport_layer.h	4.4#open_lin_transport_layer.h
6.33#open_lin_network_layer.h	Declare callback for received frame	Declares a callback function signature that is intended to be executed when a LIN frame is successfully received and processed by the network layer.	CS-3.33#open_lin_network_layer.h	5.11#open_lin_network_layer.h

ID	Node Name	Description	Children IDs	Parent IDs
6.34#open_lin_network_layer.h	Declare function to get frame slot by PID	Declares a function signature for retrieving a specific LIN frame slot definition based on its Process Identifier (PID).	CS-3.34#open_lin_network_layer.h	5.10#open_lin_network_layer.h
6.35#open_lin_network_layer.h	Declare function to handle received frames	Declares the function signature for the network layer's handler responsible for processing incoming LIN frames based on their PID.	CS-3.35#open_lin_network_layer.h	5.11#open_lin_network_layer.h
6.36#open_lin_network_layer.h	Declare function to initialize network layer	Declares the function signature for initializing the LIN network layer, typically by providing an array of frame slot definitions and its length.	CS-3.36#open_lin_network_layer.h	4.5#open_lin_network_layer.h
6.37#open_lin_network_layer.h	Define dynamic ID translation structure	Defines a structure (conditionally compiled) for mapping IDs in the LIN table to input IDs, enabling dynamic ID handling.	CS-3.37#open_lin_network_layer.h	5.10#open_lin_network_layer.h
6.38#open_lin_network_layer.h	Define LIN frame slot structure	Defines the data structure used to represent a LIN frame slot, including its PID, frame type, data length, and a pointer to the data.	CS-3.38#open_lin_network_layer.h	5.10#open_lin_network_layer.h
6.39#open_lin_types.h	Declare error handler function	This feature declares a function prototype for a generic error handler that accepts an error code defined in the <code>`t_open_lin_error`</code> enum, intended for reporting or processing communication errors.	CS-3.39#open_lin_types.h	5.14#open_lin_types.h
6.40#open_lin_types.h	Define basic LIN types	This feature defines type aliases for common C types (<code>`bool`</code> , <code>`uint8_t`</code> , <code>`uint16_t`</code>) used within the LIN context, improving code readability and portability.	CS-3.40#open_lin_types.h	5.13#open_lin_types.h
6.41#open_lin_types.h	Define boolean macros	This feature defines macros for standard boolean values (<code>`true`</code> , <code>`false`</code>) using the defined <code>`I_bool`</code>	CS-3.41#open_lin_types.h	5.12#open_lin_types.h

ID	Node Name	Description	Children IDs	Parent IDs
		type.		
6.42#open_lin_types.h	Define diagnostic frame IDs	This feature defines constants for the standard LIN diagnostic request and response frame identifiers.	CS-3.42#open_lin_types.h	5.12#open_lin_types.h
6.43#open_lin_types.h	Define error code enum	This feature defines an extensive enumeration listing various possible error conditions that can occur during LIN communication, covering both slave and master specific errors.	CS-3.43#open_lin_types.h	5.14#open_lin_types.h
6.44#open_lin_types.h	Define frame constants and macros	This feature defines constants and macros related to LIN frame structure, including maximum frame length, synchronization byte value, and bit manipulation for PID.	CS-3.44#open_lin_types.h, CS-3.44#open_lin_types.h	5.12#open_lin_types.h
6.45#open_lin_types.h	Define frame type enum	This feature defines an enumeration to represent the two main types of LIN frames: transmit and receive.	CS-3.45#open_lin_types.h	5.13#open_lin_types.h
6.46#open_lin_types.h	Define LIN-specific types	This feature defines type aliases for specific LIN protocol elements like Process Identifier (PID) and checksum, using the previously defined basic LIN types.	CS-3.46#open_lin_types.h	5.13#open_lin_types.h
6.47#open_lin_types.h	Define memory copy macro	This feature defines a macro to alias the standard C `memcpy` function, potentially for abstraction or future modification.	CS-3.47#open_lin_types.h	5.12#open_lin_types.h
6.48#open_lin_slave_data_layer.c	Detect and handle break signal	Checks if a break signal is detected on the LIN bus. If detected, it handles potential errors if the slave was not in the IDLE state and transitions the state to prepare for SYNC or PID reception, potentially setting auto-baud if configured.	CS-3.48#open_lin_slave_data_layer.c, CS-3.48#open_lin_slave_data_layer.c	5.18#open_lin_slave_data_layer.c

ID	Node Name	Description	Children IDs	Parent IDs
6.49#open_lin_slave_data_layer.c	Determine frame details from PID	Looks up the frame configuration (data length and data pointer) based on the received PID from the network layer's frame slots. It also determines if the frame is meant for transmission (TX) or reception (RX) by the slave and updates the internal state accordingly.	CS-3.49#open_lin_slave_data_layer.c	4.7#open_lin_slave_data_layer.c
6.50#open_lin_slave_data_layer.c	Handle invalid state error	Serves as a default case in the state machine to catch any received byte that occurs when the slave is in an unexpected or invalid state. It triggers an error handler and resets the slave to a safe state.	CS-3.50#open_lin_slave_data_layer.c	5.17#open_lin_slave_data_layer.c
6.51#open_lin_slave_data_layer.c	Initialize LIN slave	Initializes the LIN slave module by calling the reset function to set its internal state to idle.	CS-3.51#open_lin_slave_data_layer.c	5.16#open_lin_slave_data_layer.c
6.52#open_lin_slave_data_layer.c	Process incoming LIN byte	This is the main handler for processing each received byte on the LIN bus. It manages the state machine for receiving the LIN header (Break, SYNC, PID) and subsequent data/checksum bytes, transitioning states and invoking appropriate processing based on the current state and received byte.	CS-3.52#open_lin_slave_data_layer.c, CS-3.52#open_lin_slave_data_layer.c, CS-3.52#open_lin_slave_data_layer.c, CS-3.52#open_lin_slave_data_layer.c, CS-3.52#open_lin_slave_data_layer.c, CS-3.52#open_lin_slave_data_layer.c	5.17#open_lin_slave_data_layer.c
6.53#open_lin_slave_data_layer.c	Process PID byte	Handles the slave's state when expecting the PID byte. It checks the parity of the received byte. If parity is valid, it extracts the PID and uses it to determine the frame type (TX/RX) and data parameters using the 'Determine frame details from PID' feature. If the PID is valid and known, it transitions to the appropriate data state (TX or RX); otherwise, it signals a parity or ID	CS-3.53#open_lin_slave_data_layer.c	5.18#open_lin_slave_data_layer.c

ID	Node Name	Description	Children IDs	Parent IDs
		not found error and resets the slave state.		
6.54#open_lin_slave_data_layer.c	Process SYNC byte	Handles the slave's state when expecting the SYNC byte (0x55). It validates the received byte. If valid, it transitions the state to expect the PID; otherwise, it signals an error and resets the slave state.	CS-3.54#open_lin_slave_data_layer.c, CS-3.54#open_lin_slave_data_layer.c	5.18#open_lin_slave_data_layer.c
6.55#open_lin_slave_data_layer.c	Receive data bytes and checksum	Handles the slave's state when receiving data bytes and the checksum for an RX frame. It collects the specified number of data bytes into an internal buffer. Once all data bytes are received, it expects the checksum byte, calculates the checksum of the received data, and validates it against the received checksum. If valid, it copies the data to the network layer's buffer and notifies the network layer; otherwise, it signals a checksum error. The state is reset after validation.	CS-3.55#open_lin_slave_data_layer.c, CS-3.55#open_lin_slave_data_layer.c, CS-3.55#open_lin_slave_data_layer.c, CS-3.55#open_lin_slave_data_layer.c	5.15#open_lin_slave_data_layer.c
6.56#open_lin_slave_data_layer.c	Reset LIN slave state	Resets the internal state machine and data counter of the LIN slave back to the idle state, preparing it for a new LIN frame.	CS-3.56#open_lin_slave_data_layer.c	5.16#open_lin_slave_data_layer.c
6.57#open_lin_slave_data_layer.c	Transmit data and checksum	If the slave is in the Data TX state (determined after processing a TX PID), this feature calculates the checksum for the data to be transmitted, then sends the data bytes followed by the checksum byte over the hardware interface. It handles potential hardware transmission errors and resets the state upon successful transmission	CS-3.57#open_lin_slave_data_layer.c	5.15#open_lin_slave_data_layer.c

ID	Node Name	Description	Children IDs	Parent IDs
		or error.		
6.58#open_lin_slave_data_layer.h	Define LIN slave operational states	Declares an enumeration type representing the different operational states of the LIN slave component, such as idle, receiving sync, PID, or data, and transmitting data.	CS-3.58#open_lin_slave_data_layer.h	5.19#open_lin_slave_data_layer.h
6.59#open_lin_slave_data_layer.h	Handle received header byte	Declares a function signature for processing incoming bytes, specifically those expected during the reception of the LIN header (Sync Break, Sync Field, and PID).	CS-3.59#open_lin_slave_data_layer.h	4.8#open_lin_slave_data_layer.h
6.60#open_lin_slave_data_layer.h	Initialize LIN slave component	Declares a function signature for initializing the LIN slave component, likely setting up its initial state and resources.	CS-3.60#open_lin_slave_data_layer.h	5.19#open_lin_slave_data_layer.h
6.61#open_lin_slave_data_layer.h	Reset LIN slave component	Declares a function signature for resetting the LIN slave component to a default state.	CS-3.61#open_lin_slave_data_layer.h	5.19#open_lin_slave_data_layer.h
6.62#open_lin_master_data_layer.c	Execute periodic master state machine	A periodic handler that drives the master's state machine. It increments time, checks if the offset for the current scheduled frame has passed in the idle state to initiate transmission, manages state transitions based on TX/RX start, and handles reception timeouts.	CS-3.62#open_lin_master_data_layer.c, CS-3.62#open_lin_master_data_layer.c, CS-3.62#open_lin_master_data_layer.c, CS-3.62#open_lin_master_data_layer.c, CS-3.62#open_lin_master_data_layer.c, CS-3.62#open_lin_master_data_layer.c	5.21#open_lin_master_data_layer.c
6.63#open_lin_master_data_layer.c	Handle LIN communication errors	Reports specific communication errors (checksum mismatch, header transmission failure, data transmission failure, reception timeout) by calling the global error handler function.	CS-3.63#open_lin_master_data_layer.c	5.22#open_lin_master_data_layer.c
6.64#open_lin_master	Initialize master data layer	Initializes the LIN master data layer	CS-3.64#open_lin_master_data_layer.c	4.9#open_lin_master_

ID	Node Name	Description	Children IDs	Parent IDs
_data_layer.c		by setting the frame schedule table and its size, and resetting the master's state to idle.		data_layer.c
6.65#open_lin_master_data_layer.c	Manage frame schedule index	Maintains the current index in the master frame schedule table, provides functions to get the current frame item, and advances the index to the next item, wrapping around at the end of the table.	CS-3.65#open_lin_master_data_layer.c, CS-3.65#open_lin_master_data_layer.c, CS-3.65#open_lin_master_data_layer.c	5.21#open_lin_master_data_layer.c
6.66#open_lin_master_data_layer.c	Manage LIN master state	Defines the possible states for the LIN master (idle, receiving, transmitting) and provides a function to reset the master's state to idle, clearing counters and optionally moving to the next schedule item.	CS-3.66#open_lin_master_data_layer.c, CS-3.66#open_lin_master_data_layer.c, CS-3.66#open_lin_master_data_layer.c	5.21#open_lin_master_data_layer.c
6.67#open_lin_master_data_layer.c	Process received LIN data bytes	Handles incoming bytes during the data reception phase of a LIN frame. It buffers the data, checks the checksum upon completion, copies valid data to the destination buffer, and triggers a reception callback. It also handles checksum errors.	CS-3.67#open_lin_master_data_layer.c	5.22#open_lin_master_data_layer.c
6.68#open_lin_master_data_layer.c	Transmit LIN frame data payload	Transmits the data bytes and the calculated checksum byte for a LIN frame using the hardware abstraction layer.	CS-3.68#open_lin_master_data_layer.c, CS-3.68#open_lin_master_data_layer.c	5.20#open_lin_master_data_layer.c
6.69#open_lin_master_data_layer.c	Transmit LIN frame header	Transmits the Break, Sync (0x55), and Protected Identifier (PID) bytes of a LIN frame using the hardware abstraction layer.	CS-3.69#open_lin_master_data_layer.c	5.20#open_lin_master_data_layer.c
6.70#open_lin_transport_layer.c	Define and Register Service ID Handlers	This feature defines a structure to associate a Service ID (SID) with its corresponding handler function and	CS-3.70#open_lin_transport_layer.c, CS-3.70#open_lin_transport_layer.c	5.23#open_lin_transport_layer.c

ID	Node Name	Description	Children IDs	Parent IDs
		initializes an array to register specific handlers, such as the one for the 'Assign NAD' service.		
6.71#open_lin_transport_layer.c	Handle LIN Transport Layer Messages	This function serves as the main entry point for the transport layer, processing incoming messages based on their PID (Protocol Identifier) to differentiate between diagnostic requests and responses.	CS-3.71#open_lin_transport_layer.c, CS-3.71#open_lin_transport_layer.c	4.10#open_lin_transport_layer.c
6.72#open_lin_transport_layer.c	Implement Assign NAD Service (SID 0x27)	This static function handles the 'Assign NAD' diagnostic service. It checks if the supplier ID and function ID in the request match the device's configured IDs. If they match, the device's NAD is updated with the NAD provided in the request.	CS-3.72#open_lin_transport_layer.c	5.23#open_lin_transport_layer.c
6.73#open_lin_transport_layer.c	Process Incoming Diagnostic Requests	This static function handles diagnostic requests. It checks if the request's NAD (Node Address) matches the device's NAD or the broadcast NAD. For single frame requests, it iterates through registered Service ID (SID) handlers and calls the appropriate handler based on the request's SID.	CS-3.73#open_lin_transport_layer.c	5.23#open_lin_transport_layer.c
6.74#open_lin_cfg.h	Configure hardware break handling	Specifies that the hardware break signal should be interpreted as the synchronization byte in the LIN protocol.	CS-3.74#open_lin_cfg.h	5.24#open_lin_cfg.h
6.75#open_lin_cfg.h	Configure network layer dynamic IDs	Enables the use of dynamic identifiers within the LIN network layer, allowing for flexible node addressing.	CS-3.75#open_lin_cfg.h	5.24#open_lin_cfg.h
6.76#open_lin_cfg.h	Configure platform	Defines the endianness of the target	CS-3.76#open_lin_cfg.h	4.11#open_lin_cfg.h

ID	Node Name	Description	Children IDs	Parent IDs
	endianness	platform, indicating whether it uses big-endian byte ordering.		
6.77#open_lin_cfg.h	Enable transport layer	Activates the transport layer functionality for handling segmented data transmission and reception over the LIN bus.	CS-3.77#open_lin_cfg.h	5.24#open_lin_cfg.h
6.78#open_lin_cfg.h	Prevent multiple inclusions	Implements a header guard to ensure that the contents of this header file are included only once during compilation, preventing potential redefinition errors.	CS-3.78#open_lin_cfg.h, CS-3.78#open_lin_cfg.h	4.11#open_lin_cfg.h
6.79#implementation/x86/rs232.h	Check serial line status	This parent feature includes functionalities for checking the status of various RS232 control lines like CTS, DCD, and DSR.	7.1#implementation/x86/rs232.h, 7.2#implementation/x86/rs232.h, 7.3#implementation/x86/rs232.h	5.25#implementation/x86/rs232.h
6.80#implementation/x86/rs232.h	Handle header and library includes	This parent feature includes structural elements for managing header guards, operating system specific includes, standard library includes, and C++ compatibility.	7.6#implementation/x86/rs232.h, 7.11#implementation/x86/rs232.h, 7.12#implementation/x86/rs232.h, 7.17#implementation/x86/rs232.h, 7.18#implementation/x86/rs232.h, 7.26#implementation/x86/rs232.h	4.12#implementation/x86/rs232.h
6.81#implementation/x86/rs232.h	Manage break condition	This parent feature includes functionalities for setting and clearing the break condition on the serial line.	7.4#implementation/x86/rs232.h, 7.25#implementation/x86/rs232.h	5.25#implementation/x86/rs232.h
6.82#implementation/x86/rs232.h	Manage data buffers	This parent feature includes functionalities for flushing (discarding) data in the serial port's receive and transmit buffers.	7.13#implementation/x86/rs232.h, 7.14#implementation/x86/rs232.h, 7.15#implementation/x86/rs232.h	5.25#implementation/x86/rs232.h
6.83#implementation/x86/rs232.h	Manage serial handshaking lines	This parent feature includes functionalities for controlling the state of the Data Terminal Ready (DTR) and Request To Send (RTS) lines.	7.7#implementation/x86/rs232.h, 7.8#implementation/x86/rs232.h, 7.9#implementation/x86/rs232.h, 7.10#implementation/x86/rs232.h	5.25#implementation/x86/rs232.h

ID	Node Name	Description	Children IDs	Parent IDs
6.84#implementation/x86/rs232.h	Manage serial port connection	This parent feature includes functionalities for opening and closing serial communication ports and retrieving port information.	7.5#implementation/x86/rs232.h, 7.16#implementation/x86/rs232.h, 7.19#implementation/x86/rs232.h	5.25#implementation/x86/rs232.h
6.85#implementation/x86/rs232.h	Receive data from serial port	This parent feature includes functionalities for reading or polling data from the serial port's receive buffer, including options with timeout.	7.20#implementation/x86/rs232.h, 7.21#implementation/x86/rs232.h	5.26#implementation/x86/rs232.h
6.86#implementation/x86/rs232.h	Transmit data over serial	This parent feature includes functionalities for sending data in various formats (single byte, buffer, string) to the serial port.	7.22#implementation/x86/rs232.h, 7.23#implementation/x86/rs232.h, 7.24#implementation/x86/rs232.h	5.26#implementation/x86/rs232.h
6.87#implementation/x86/main_slave_example.c	Configure LIN frame slots	Configures the array of LIN frame slots specifying the PID, type (transmit/receive), data length, and associated data buffer for each frame handled by the slave.	CS-3.87#implementation/x86/main_slave_example.c	5.27#implementation/x86/main_slave_example.c
6.88#implementation/x86/main_slave_example.c	Define data buffers	Defines global arrays to store data for LIN transmit and receive frames.	CS-3.88#implementation/x86/main_slave_example.c	5.27#implementation/x86/main_slave_example.c
6.89#implementation/x86/main_slave_example.c	Handle received LIN frames	Implements a callback function that is executed when a complete LIN frame is received, printing the frame's PID and data bytes to the console.	CS-3.89#implementation/x86/main_slave_example.c	5.28#implementation/x86/main_slave_example.c, 5.29#implementation/x86/main_slave_example.c
6.90#implementation/x86/main_slave_example.c	Initialize LIN stack and run main loop	Initializes the LIN hardware and network layers with the defined frame slots, then enters an infinite loop to keep the slave application running and processing LIN communication events.	CS-3.90#implementation/x86/main_slave_example.c	4.13#implementation/x86/main_slave_example.c
6.91#implementation/x86/main_slave_example.c	Process incoming LIN bytes	Defines a handler function that takes an individual byte received	CS-3.91#implementation/x86/main_slave_example.c, CS-3.91#implementation/x86/main_slave	5.29#implementation/x86/main_slave_example.c

ID	Node Name	Description	Children IDs	Parent IDs
e.c		from the LIN bus and passes it to the open-LIN slave's receive header processing function.	_example.c	e.c
6.92#implementation/x86/main_slave_example.c	Provide unused master RX callback	Provides a callback function intended for master reception, but is explicitly marked as not used in this slave configuration.	CS-3.92#implementation/x86/main_slave_example.c	5.28#implementation/x86/main_slave_example.c
6.93#implementation/x86/rs232.c	Control break condition	This parent feature includes functionalities to set or clear the break state on the serial communication line.	7.30#implementation/x86/rs232.c, 7.46#implementation/x86/rs232.c	5.30#implementation/x86/rs232.c
6.94#implementation/x86/rs232.c	Control flow signals	This parent feature includes functionalities to control and query the state of serial port hardware flow control signals (DTR, RTS, CTS, DCD, DSR).	7.27#implementation/x86/rs232.c, 7.28#implementation/x86/rs232.c, 7.29#implementation/x86/rs232.c, 7.32#implementation/x86/rs232.c, 7.33#implementation/x86/rs232.c, 7.34#implementation/x86/rs232.c, 7.35#implementation/x86/rs232.c	5.30#implementation/x86/rs232.c
6.95#implementation/x86/rs232.c	Manage communication buffers	This parent feature includes functionalities to clear the transmit and receive buffers of the serial port.	7.36#implementation/x86/rs232.c, 7.37#implementation/x86/rs232.c, 7.38#implementation/x86/rs232.c	5.31#implementation/x86/rs232.c
6.96#implementation/x86/rs232.c	Manage serial ports	This parent feature includes functionalities for opening, closing, and identifying serial communication ports.	7.31#implementation/x86/rs232.c, 7.39#implementation/x86/rs232.c, 7.40#implementation/x86/rs232.c	5.31#implementation/x86/rs232.c
6.97#implementation/x86/rs232.c	Receive data	This parent feature includes functionalities for reading data from the serial port, including polling with or without break detection.	7.41#implementation/x86/rs232.c, 7.42#implementation/x86/rs232.c	5.32#implementation/x86/rs232.c
6.98#implementation/x86/rs232.c	Transmit data	This parent feature includes functionalities for sending data, including single bytes, buffers, and null-terminated strings, over the	7.43#implementation/x86/rs232.c, 7.44#implementation/x86/rs232.c, 7.45#implementation/x86/rs232.c	5.32#implementation/x86/rs232.c

ID	Node Name	Description	Children IDs	Parent IDs
		serial port.		
6.99#implementation/ST32/open_lin_hw_st32.c	Check for LIN break detection	Checks if a LIN break character has been detected by the UART hardware and clears the corresponding flag.	CS-3.99#implementation/ST32/open_lin_hw_st32.c	5.34#implementation/ST32/open_lin_hw_st32.c
6.100#implementation/ST32/open_lin_hw_st32.c	Configure auto-baud detection	Provides a placeholder function intended for configuring or requesting auto-baud detection functionality, but it is currently empty.	CS-3.100#implementation/ST32/open_lin_hw_st32.c, CS-3.100#implementation/ST32/open_lin_hw_st32.c	5.33#implementation/ST32/open_lin_hw_st32.c
6.101#implementation/ST32/open_lin_hw_st32.c	Enable or disable LIN reception	Controls the UART receiver state for LIN communication. If enabled, it calls the hardware reset function; if disabled, it clears the UART receive and parity error interrupt enable bits.	CS-3.101#implementation/ST32/open_lin_hw_st32.c	5.33#implementation/ST32/open_lin_hw_st32.c
6.102#implementation/ST32/open_lin_hw_st32.c	Handle LIN errors	Provides a placeholder function for handling errors detected within the LIN communication, although it currently contains no implementation.	CS-3.102#implementation/ST32/open_lin_hw_st32.c	4.15#implementation/ST32/open_lin_hw_st32.c
6.103#implementation/ST32/open_lin_hw_st32.c	Handle received LIN frame callback	Provides a callback function placeholder intended to process a received LIN frame at the data link layer.	CS-3.103#implementation/ST32/open_lin_hw_st32.c	5.34#implementation/ST32/open_lin_hw_st32.c
6.104#implementation/ST32/open_lin_hw_st32.c	Reset and initialize LIN hardware	Resets the LIN hardware, specifically the associated UART instance, by clearing break flags, initializing the UART for LIN mode with 11-bit break detection, starting reception in interrupt mode, and enabling the corresponding UART interrupt.	CS-3.104#implementation/ST32/open_lin_hw_st32.c	5.33#implementation/ST32/open_lin_hw_st32.c
6.105#implementation/	Transmit a block of data	Transmits a block of data of a	CS-3.105#implementation/ST32/open_lin_hw_st	5.35#implementation/S

ID	Node Name	Description	Children IDs	Parent IDs
ST32/open_lin_hw_st32.c		specified length over the configured UART interface with a timeout.	32.c, CS-3.105#implementation/ST32/open_lin_hw_st32.c	T32/open_lin_hw_st32.c
6.106#implementation/ST32/open_lin_hw_st32.c	Transmit a LIN break character	Sends a LIN break character using the HAL LIN function.	CS-3.106#implementation/ST32/open_lin_hw_st32.c	5.35#implementation/ST32/open_lin_hw_st32.c
6.107#implementation/ST32/open_lin_hw_st32.c	Transmit a single byte	Transmits a single byte of data over the configured UART interface with a timeout.	CS-3.107#implementation/ST32/open_lin_hw_st32.c	5.35#implementation/ST32/open_lin_hw_st32.c
6.108#implementation/unit_test/test_run.cpp	Calculate Slot Array Length	Calculates the number of entries in the 'slot_array' using standard C array size calculation.	CS-3.108#implementation/unit_test/test_run.cpp	5.36#implementation/unit_test/test_run.cpp
6.109#implementation/unit_test/test_run.cpp	Configure Frame Slots	Defines an array of 'frame slots' specifying the PID, type (transmit/receive), length, and data buffer for each supported LIN frame.	CS-3.109#implementation/unit_test/test_run.cpp	5.36#implementation/unit_test/test_run.cpp
6.110#implementation/unit_test/test_run.cpp	Declare External LIN Components	Declares functions and variables from the Open LIN C code to be accessible within this C++ test file.	CS-3.110#implementation/unit_test/test_run.cpp	5.37#implementation/unit_test/test_run.cpp
6.111#implementation/unit_test/test_run.cpp	Declare External Simulation Variables	Declares variables used for simulating hardware events or states during testing, such as break detection.	CS-3.111#implementation/unit_test/test_run.cpp	5.37#implementation/unit_test/test_run.cpp
6.112#implementation/unit_test/test_run.cpp	Define Simulation Variables for Transmission	Defines boolean flags used by the simulated hardware transmission function to indicate invalid transmission or completion.	CS-3.112#implementation/unit_test/test_run.cpp	5.37#implementation/unit_test/test_run.cpp
6.113#implementation/unit_test/test_run.cpp	Define Test Data Buffers	Initializes static data arrays used as buffers for simulating LIN frame data during tests.	CS-3.113#implementation/unit_test/test_run.cpp	5.36#implementation/unit_test/test_run.cpp
6.114#implementation/unit_test/test_run.cpp	Include LIN Stack Headers	Includes necessary headers for the Open LIN protocol stack	CS-3.114#implementation/unit_test/test_run.cpp	5.37#implementation/unit_test/test_run.cpp

ID	Node Name	Description	Children IDs	Parent IDs
		components (configuration, data layer, hardware abstraction, network layer).		
6.115#implementation/unit_test/test_run.cpp	Include Testing Framework	Includes the Catch2 testing framework headers and defines the main entry point for tests.	CS-3.115#implementation/unit_test/test_run.cpp	5.37#implementation/unit_test/test_run.cpp
6.116#implementation/unit_test/test_run.cpp	Simulate Hardware Data Transmission	Provides a mock implementation of `open_lin_hw_tx_data` that checks if the transmitted data and length match expected values based on test scenarios, setting flags accordingly.	CS-3.116#implementation/unit_test/test_run.cpp	4.16#implementation/unit_test/test_run.cpp
6.117#implementation/unit_test/test_run.cpp	Support Dynamic ID Translation (Conditional)	If `OPEN_LIN_DYNAMIC_IDS` is defined, this block defines a translation table for dynamic LIN IDs and includes a test case to verify the slave can receive frames based on translated IDs.	CS-3.117#implementation/unit_test/test_run.cpp	5.39#implementation/unit_test/test_run.cpp
6.118#implementation/unit_test/test_run.cpp	Test Frame Checksum Calculation	Tests the correctness of the `open_lin_data_layer_checksum` function using different data payloads and PIDs.	CS-3.118#implementation/unit_test/test_run.cpp	5.38#implementation/unit_test/test_run.cpp
6.119#implementation/unit_test/test_run.cpp	Test Invalid Checksum Handling	Simulates receiving a complete receive frame with an incorrect checksum and verifies that the slave reports the `OPEN_LIN_SLAVE_ERROR_INVALID_CHECKSUM` error.	CS-3.119#implementation/unit_test/test_run.cpp	5.39#implementation/unit_test/test_run.cpp
6.120#implementation/unit_test/test_run.cpp	Test Invalid PID Parity Handling	Simulates receiving a header with an incorrect PID parity and verifies that the slave reports the `OPEN_LIN_SLAVE_ERROR_PID_PARITY` error.	CS-3.120#implementation/unit_test/test_run.cpp	5.39#implementation/unit_test/test_run.cpp
6.121#implementation/unit_test/test_run.cpp	Test PID Parity Calculation	Tests the correctness of the	CS-3.121#implementation/unit_test/test_run.cpp	5.38#implementation/unit_test/test_run.cpp

ID	Node Name	Description	Children IDs	Parent IDs
unit_test/test_run.cpp		`open_lin_data_layer_parity` function for various PID values.		nit_test/test_run.cpp
6.122#implementation/unit_test/test_run.cpp	Test Slave Break Detection	Simulates receiving break signals via `open_lin_slave_rx_header` and verifies the slave's error handling for subsequent invalid break conditions.	CS-3.122#implementation/unit_test/test_run.cpp , CS-3.122#implementation/unit_test/test_run.cpp, CS-3.122#implementation/unit_test/test_run.cpp, CS-3.122#implementation/unit_test/test_run.cpp, CS-3.122#implementation/unit_test/test_run.cpp	5.39#implementation/unit_test/test_run.cpp
6.123#implementation/unit_test/test_run.cpp	Test Slave Reset Functionality	Tests the `open_lin_slave_reset` function by simulating a break after a reset and verifying no error is reported.	CS-3.123#implementation/unit_test/test_run.cpp	5.39#implementation/unit_test/test_run.cpp
6.124#implementation/unit_test/test_run.cpp	Test Slave Transmit Frame Handling	Simulates receiving the sequence for a transmit frame (Break, Sync, Transmit PID) and verifies, using the simulated hardware function, that the slave attempts to transmit the correct data.	CS-3.124#implementation/unit_test/test_run.cpp	5.39#implementation/unit_test/test_run.cpp
6.125#implementation/unit_test/test_run.cpp	Test Valid Diagnostic Frame Reception	Simulates receiving a complete and valid diagnostic frame and verifies that no error is reported.	CS-3.125#implementation/unit_test/test_run.cpp , CS-3.125#implementation/unit_test/test_run.cpp	5.39#implementation/unit_test/test_run.cpp
6.126#implementation/unit_test/test_run.cpp	Test Valid Standard Frame Reception	Simulates receiving a complete and valid standard receive frame, verifies no error is reported, and checks that the received data is correctly written to the corresponding data buffer.	CS-3.126#implementation/unit_test/test_run.cpp	5.39#implementation/unit_test/test_run.cpp
6.127#implementation/unit_test/catch.hpp	Handle Test Assertions and Comparisons	This parent feature covers the definition, evaluation, and processing of assertion results, including support for various assertion types, custom matchers, and specialized comparison logic for types like floating-point numbers and strings.	7.47#implementation/unit_test/catch.hpp, 7.50#implementation/unit_test/catch.hpp, 8.25#implementation/unit_test/catch.hpp, 8.44#implementation/unit_test/catch.hpp	5.41#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
6.128#implementation/unit_test/catch.hpp	Manage Library Configuration and State	This parent feature includes functionalities for parsing command-line arguments, using external configuration data, managing the global state of the library, internal registries, and utility components.	7.51#implementation/unit_test/catch.hpp, 7.53#implementation/unit_test/catch.hpp, 8.190#implementation/unit_test/catch.hpp	5.40#implementation/unit_test/catch.hpp
6.129#implementation/unit_test/catch.hpp	Manage Test Execution Lifecycle	This parent feature includes functionalities related to discovering, filtering, sorting, running, and managing the state and flow of test cases, test groups, sections, and specific test invokers.	7.52#implementation/unit_test/catch.hpp, 7.56#implementation/unit_test/catch.hpp, 8.225#implementation/unit_test/catch.hpp	5.41#implementation/unit_test/catch.hpp
6.130#implementation/unit_test/catch.hpp	Provide Utility Functions	This parent feature includes general-purpose helper functionalities not directly part of the core test execution or assertion process, such as string manipulation, conversion utilities, and benchmarking tools.	7.54#implementation/unit_test/catch.hpp, 7.49#implementation/unit_test/catch.hpp	5.40#implementation/unit_test/catch.hpp
6.131#implementation/unit_test/catch.hpp	Report Test Outcomes	This parent feature encompasses the outputting of test results, assertion details, and related messages, supporting multiple formats and configuration options for reporting.	7.55#implementation/unit_test/catch.hpp, 7.57#implementation/unit_test/catch.hpp	4.17#implementation/unit_test/catch.hpp
6.132#implementation/unit_test/open_lin_frame_winOS.c	Simulate and Retrieve LIN Errors	Implements a mechanism to simulate LIN error occurrences via a global variable and provides a function to retrieve the last simulated error code, clearing it afterwards.	CS-3.132#implementation/unit_test/open_lin_frame_winOS.c, CS-3.132#implementation/unit_test/open_lin_frame_winOS.c	5.44#implementation/unit_test/open_lin_frame_winOS.c
6.133#implementation/unit_test/open_lin_frame_winOS.c	Simulate Hardware Reset	Simulates a hardware reset operation by clearing the index of the simulated TX data buffer.	CS-3.133#implementation/unit_test/open_lin_frame_winOS.c	5.44#implementation/unit_test/open_lin_frame_winOS.c

ID	Node Name	Description	Children IDs	Parent IDs
6.134#implementation/unit_test/open_lin_frame_winOS.c	Simulate LIN Break Detection	Uses a global boolean variable to simulate the detection of a LIN break signal by the hardware and provides a function to check and clear this simulated state.	CS-3.134#implementation/unit_test/open_lin_frame_winOS.c	5.42#implementation/unit_test/open_lin_frame_winOS.c
6.135#implementation/unit_test/open_lin_frame_winOS.c	Simulate LIN RX Frame Callback Storage	Stores a pointer to the last received LIN frame slot provided to the master data layer receive callback, allowing its contents to be inspected for testing.	CS-3.135#implementation/unit_test/open_lin_frame_winOS.c	5.42#implementation/unit_test/open_lin_frame_winOS.c
6.136#implementation/unit_test/open_lin_frame_winOS.c	Simulate LIN TX Data Buffer	Maintains an internal buffer (`lasttx`) to store bytes transmitted by the simulated hardware functions and provides functions to retrieve the content of this buffer and the number of bytes transmitted.	CS-3.136#implementation/unit_test/open_lin_frame_winOS.c	5.43#implementation/unit_test/open_lin_frame_winOS.c
6.137#implementation/unit_test/open_lin_frame_winOS.c	Simulate Transmitting LIN Break	Simulates transmitting a LIN break signal by writing a specific value (0) to the simulated TX data buffer.	CS-3.137#implementation/unit_test/open_lin_frame_winOS.c	5.43#implementation/unit_test/open_lin_frame_winOS.c
6.138#implementation/unit_test/open_lin_frame_winOS.c	Simulate Transmitting LIN Byte	Simulates transmitting a single byte over the LIN bus by storing the byte in the simulated TX data buffer.	CS-3.138#implementation/unit_test/open_lin_frame_winOS.c	5.43#implementation/unit_test/open_lin_frame_winOS.c
6.139#implementation/unit_test/open_lin_frame_winOS.c	Simulate Unimplemented Hardware/Callback Functions	Provides empty placeholder implementations for hardware functions related to enabling reception, setting auto-baud, and a network layer receive callback, indicating these features are not simulated in this file.	CS-3.139#implementation/unit_test/open_lin_frame_winOS.c, CS-3.139#implementation/unit_test/open_lin_frame_winOS.c	4.18#implementation/unit_test/open_lin_frame_winOS.c
6.140#implementation/pic16/open_lin_hw_pic16.c	Check for LIN Break Character	Checks the hardware flag (FERR) to detect a LIN break character and clears the flag by reading the EUSART buffer.	CS-3.140#implementation/pic16/open_lin_hw_pic16.c	5.45#implementation/pic16/open_lin_hw_pic16.c
6.141#implementation/	Define Hardware Access	Defines macros to abstract access	CS-3.141#implementation/pic16/open_lin_hw_pi	4.19#implementation/pi

ID	Node Name	Description	Children IDs	Parent IDs
pic16/open_lin_hw_pic16.c	Macros	to specific hardware registers and functions (EUSART, Timer), making the code more portable or easier to read.	c16.c	c16/open_lin_hw_pic16.c
6.142#implementation/pic16/open_lin_hw_pic16.c	Enable or Disable LIN Receiver	Controls the state of the LIN receiver hardware (RCIE) and clears overrun errors (CREN), enabling it or performing a reset depending on the input flag.	CS-3.142#implementation/pic16/open_lin_hw_pic16.c, CS-3.142#implementation/pic16/open_lin_hw_pic16.c	5.45#implementation/pic16/open_lin_hw_pic16.c
6.143#implementation/pic16/open_lin_hw_pic16.c	Handle LIN Errors (Placeholder)	Provides a placeholder function for handling LIN communication errors, although the current implementation does nothing.	CS-3.143#implementation/pic16/open_lin_hw_pic16.c	4.19#implementation/pic16/open_lin_hw_pic16.c
6.144#implementation/pic16/open_lin_hw_pic16.c	Reset LIN Hardware	Resets the LIN hardware state, which currently involves checking for a break character.	CS-3.144#implementation/pic16/open_lin_hw_pic16.c	5.45#implementation/pic16/open_lin_hw_pic16.c
6.145#implementation/pic16/open_lin_hw_pic16.c	Set Auto-Baud Rate (Placeholder)	Provides a placeholder function for triggering the UART auto-baud rate detection mechanism.	CS-3.145#implementation/pic16/open_lin_hw_pic16.c	5.45#implementation/pic16/open_lin_hw_pic16.c
6.146#implementation/pic16/open_lin_hw_pic16.c	Store LIN Node Configuration	Declares global variables to store the LIN Node Address (NAD), Supplier ID, and Function ID for the device.	CS-3.146#implementation/pic16/open_lin_hw_pic16.c	4.19#implementation/pic16/open_lin_hw_pic16.c
6.147#implementation/pic16/open_lin_hw_pic16.c	Transmit Data Block	Transmits an array of bytes by iterating through the data and calling the single-byte transmit function for each byte.	CS-3.147#implementation/pic16/open_lin_hw_pic16.c	5.45#implementation/pic16/open_lin_hw_pic16.c
6.148#implementation/pic16/open_lin_hw_pic16.c	Transmit Single Byte	Transmits a single byte over the LIN EUSART interface using a hardware write function.	CS-3.148#implementation/pic16/open_lin_hw_pic16.c	5.45#implementation/pic16/open_lin_hw_pic16.c
7.1#implementation/x86/rs232.h	Check CTS Status	Declares a function to check the status of the Clear To Send (CTS) line.	CS-2.1#implementation/x86/rs232.h	6.79#implementation/x86/rs232.h

ID	Node Name	Description	Children IDs	Parent IDs
7.2#implementation/x86/rs232.h	Check DCD Status	Declares a function to check the status of the Data Carrier Detect (DCD) line.	CS-2.2#implementation/x86/rs232.h	6.79#implementation/x86/rs232.h
7.3#implementation/x86/rs232.h	Check DSR Status	Declares a function to check the status of the Data Set Ready (DSR) line.	CS-2.3#implementation/x86/rs232.h	6.79#implementation/x86/rs232.h
7.4#implementation/x86/rs232.h	Clear Break Condition	Declares a function to clear the break condition on the serial line.	CS-2.4#implementation/x86/rs232.h	6.81#implementation/x86/rs232.h
7.5#implementation/x86/rs232.h	Close Serial Port	Declares a function to close an open serial communication port.	CS-2.5#implementation/x86/rs232.h	6.84#implementation/x86/rs232.h
7.6#implementation/x86/rs232.h	Define API Header Guard	Includes a standard header guard to prevent multiple inclusions of the RS-232 API definitions.	CS-2.6#implementation/x86/rs232.h	6.80#implementation/x86/rs232.h
7.7#implementation/x86/rs232.h	Disable DTR Line	Declares a function to disable the Data Terminal Ready (DTR) line.	CS-2.7#implementation/x86/rs232.h	6.83#implementation/x86/rs232.h
7.8#implementation/x86/rs232.h	Disable RTS Line	Declares a function to disable the Request To Send (RTS) line.	CS-2.8#implementation/x86/rs232.h	6.83#implementation/x86/rs232.h
7.9#implementation/x86/rs232.h	Enable DTR Line	Declares a function to enable the Data Terminal Ready (DTR) line.	CS-2.9#implementation/x86/rs232.h	6.83#implementation/x86/rs232.h
7.10#implementation/x86/rs232.h	Enable RTS Line	Declares a function to enable the Request To Send (RTS) line.	CS-2.10#implementation/x86/rs232.h	6.83#implementation/x86/rs232.h
7.11#implementation/x86/rs232.h	End API Header Guard	Ends the header guard definition.	CS-2.11#implementation/x86/rs232.h	6.80#implementation/x86/rs232.h
7.12#implementation/x86/rs232.h	End C++ Compatibility Block	Ends the `extern "C"` block for C++ compatibility.	CS-2.12#implementation/x86/rs232.h	6.80#implementation/x86/rs232.h
7.13#implementation/x86/rs232.h	Flush Receive and Transmit Buffers	Declares a function to discard data in both the receive and transmit buffers.	CS-2.13#implementation/x86/rs232.h	6.82#implementation/x86/rs232.h
7.14#implementation/x86/rs232.h	Flush Receive Buffer	Declares a function to discard data currently in the receive buffer.	CS-2.14#implementation/x86/rs232.h	6.82#implementation/x86/rs232.h

ID	Node Name	Description	Children IDs	Parent IDs
7.15#implementation/x86/rs232.h	Flush Transmit Buffer	Declares a function to discard data currently in the transmit buffer.	CS-2.15#implementation/x86/rs232.h	6.82#implementation/x86/rs232.h
7.16#implementation/x86/rs232.h	Get Port Number by Name	Declares a function to get the internal port number representation from a port name string.	CS-2.16#implementation/x86/rs232.h	6.84#implementation/x86/rs232.h
7.17#implementation/x86/rs232.h	Handle Operating System Specific Includes	Includes necessary headers depending on the operating system (Linux/FreeBSD or Windows) for serial communication functions.	CS-2.17#implementation/x86/rs232.h	6.80#implementation/x86/rs232.h
7.18#implementation/x86/rs232.h	Include Standard Libraries	Includes standard C libraries for input/output operations and string manipulation.	CS-2.18#implementation/x86/rs232.h	6.80#implementation/x86/rs232.h
7.19#implementation/x86/rs232.h	Open Serial Port	Declares a function to open a serial communication port with specified port number, baud rate, and settings.	CS-2.19#implementation/x86/rs232.h	6.84#implementation/x86/rs232.h
7.20#implementation/x86/rs232.h	Poll Data from Port	Declares a function to read available data from the serial port's receive buffer.	CS-2.20#implementation/x86/rs232.h	6.85#implementation/x86/rs232.h
7.21#implementation/x86/rs232.h	Poll Data with Timeout	Declares a function to read data from the serial port with a specified timeout.	CS-2.21#implementation/x86/rs232.h	6.85#implementation/x86/rs232.h
7.22#implementation/x86/rs232.h	Send C String	Declares a function to send a null-terminated C string to the serial port.	CS-2.22#implementation/x86/rs232.h	6.86#implementation/x86/rs232.h
7.23#implementation/x86/rs232.h	Send Data Buffer	Declares a function to send a buffer of data bytes to the serial port.	CS-2.23#implementation/x86/rs232.h	6.86#implementation/x86/rs232.h
7.24#implementation/x86/rs232.h	Send Single Byte	Declares a function to send a single byte of data to the serial port.	CS-2.24#implementation/x86/rs232.h	6.86#implementation/x86/rs232.h
7.25#implementation/x86/rs232.h	Set Break Condition	Declares a function to set the break condition on the serial line.	CS-2.25#implementation/x86/rs232.h	6.81#implementation/x86/rs232.h

ID	Node Name	Description	Children IDs	Parent IDs
7.26#implementation/x86/rs232.h	Support C++ Compatibility	Provides compatibility for C++ compilers by wrapping the function declarations in an `extern "C"` block.	CS-2.26#implementation/x86/rs232.h	6.80#implementation/x86/rs232.h
7.27#implementation/x86/rs232.c	Check CTS status	Checks if the Clear To Send (CTS) signal is enabled on the specified serial port. Returns 1 if enabled, 0 otherwise. Handles platform specifics for Linux/FreeBSD and Windows.	CS-2.27#implementation/x86/rs232.c	6.94#implementation/x86/rs232.c
7.28#implementation/x86/rs232.c	Check DCD status	Checks if the Data Carrier Detect (DCD) signal is enabled on the specified serial port. Returns 1 if enabled, 0 otherwise. Handles platform specifics for Linux/FreeBSD and Windows.	CS-2.28#implementation/x86/rs232.c	6.94#implementation/x86/rs232.c
7.29#implementation/x86/rs232.c	Check DSR status	Checks if the Data Set Ready (DSR) signal is enabled on the specified serial port. Returns 1 if enabled, 0 otherwise. Handles platform specifics for Linux/FreeBSD and Windows.	CS-2.29#implementation/x86/rs232.c	6.94#implementation/x86/rs232.c
7.30#implementation/x86/rs232.c	Clear break signal	Clears the break state on the serial port, ending the break condition. This feature is specifically implemented for Windows.	CS-2.30#implementation/x86/rs232.c	6.93#implementation/x86/rs232.c
7.31#implementation/x86/rs232.c	Close a serial port	Closes the specified serial port, restoring the original port settings and releasing resources, including port locks (Linux/FreeBSD). Handles platform specifics for Linux/FreeBSD and Windows.	CS-2.31#implementation/x86/rs232.c	6.96#implementation/x86/rs232.c
7.32#implementation/x86/rs232.c	Disable DTR signal	Sets the Data Terminal Ready (DTR) signal low on the specified serial port. Handles platform	CS-2.32#implementation/x86/rs232.c	6.94#implementation/x86/rs232.c

ID	Node Name	Description	Children IDs	Parent IDs
		specifics for Linux/FreeBSD and Windows.		
7.33#implementation/x86/rs232.c	Disable RTS signal	Sets the Request To Send (RTS) signal low on the specified serial port. Handles platform specifics for Linux/FreeBSD and Windows.	CS-2.33#implementation/x86/rs232.c	6.94#implementation/x86/rs232.c
7.34#implementation/x86/rs232.c	Enable DTR signal	Sets the Data Terminal Ready (DTR) signal high on the specified serial port. Handles platform specifics for Linux/FreeBSD and Windows.	CS-2.34#implementation/x86/rs232.c	6.94#implementation/x86/rs232.c
7.35#implementation/x86/rs232.c	Enable RTS signal	Sets the Request To Send (RTS) signal high on the specified serial port. Handles platform specifics for Linux/FreeBSD and Windows.	CS-2.35#implementation/x86/rs232.c	6.94#implementation/x86/rs232.c
7.36#implementation/x86/rs232.c	Flush receive and transmit buffers	Clears both the receive and transmit buffers of the specified serial port, discarding all pending input and output data. Handles platform specifics for Linux/FreeBSD and Windows.	CS-2.36#implementation/x86/rs232.c	6.95#implementation/x86/rs232.c
7.37#implementation/x86/rs232.c	Flush receive buffer	Clears the receive buffer of the specified serial port, discarding any pending input data. Handles platform specifics for Linux/FreeBSD and Windows.	CS-2.37#implementation/x86/rs232.c	6.95#implementation/x86/rs232.c
7.38#implementation/x86/rs232.c	Flush transmit buffer	Clears the transmit buffer of the specified serial port, discarding any pending output data. Handles platform specifics for Linux/FreeBSD and Windows.	CS-2.38#implementation/x86/rs232.c	6.95#implementation/x86/rs232.c
7.39#implementation/x86/rs232.c	Get port number by device name	Takes a device name string (e.g., "ttyS0" or "COM1") and returns the corresponding internal comport	CS-2.39#implementation/x86/rs232.c, CS-2.39#implementation/x86/rs232.c, CS-2.39#implementation/x86/rs232.c	6.96#implementation/x86/rs232.c

ID	Node Name	Description	Children IDs	Parent IDs
		index number used by the library, or -1 if the device name is not found in the predefined list. Handles platform-specific device name prefixes.		
7.40#implementation/x86/rs232.c	Open a serial port	Opens a specified serial comport, configures it with the given baudrate, data bits, parity, and stop bits, and sets flow control signals (DTR, RTS). It handles platform-specific details for Linux/FreeBSD and Windows. Includes error handling for opening the port, setting attributes, and managing port locks (Linux/FreeBSD).	CS-2.40#implementation/x86/rs232.c	6.96#implementation/x86/rs232.c
7.41#implementation/x86/rs232.c	Poll for received data	Reads up to a specified size of bytes from the receive buffer of the specified serial port. Returns the number of bytes read or 0 if no data is immediately available (non-blocking). Handles platform specifics for Linux/FreeBSD and Windows.	CS-2.41#implementation/x86/rs232.c	6.97#implementation/x86/rs232.c
7.42#implementation/x86/rs232.c	Poll for received data with break detection	Reads up to a specified size of bytes from the receive buffer and checks for break conditions, reporting the presence of a break via an output parameter. This feature provides overlapped I/O for non-blocking reads and is specifically implemented for Windows.	CS-2.42#implementation/x86/rs232.c	6.97#implementation/x86/rs232.c
7.43#implementation/x86/rs232.c	Send a buffer of data	Writes a buffer of bytes of a specified size to the transmit buffer of the serial port. Handles platform specifics for Linux/FreeBSD and	CS-2.43#implementation/x86/rs232.c	6.98#implementation/x86/rs232.c

ID	Node Name	Description	Children IDs	Parent IDs
		Windows, including non-blocking write handling.		
7.44#implementation/x86/rs232.c	Send a null-terminated string	Iterates through a null-terminated string and sends each byte sequentially using the SendByte function. This provides a convenience method for transmitting string data.	CS-2.44#implementation/x86/rs232.c	6.98#implementation/x86/rs232.c
7.45#implementation/x86/rs232.c	Send a single byte	Writes a single byte to the transmit buffer of the specified serial port. Handles platform specifics for Linux/FreeBSD and Windows, including non-blocking write handling.	CS-2.45#implementation/x86/rs232.c	6.98#implementation/x86/rs232.c
7.46#implementation/x86/rs232.c	Set break signal	Sets the break state on the serial port, causing a break condition on the transmit line. This feature is specifically implemented for Windows.	CS-2.46#implementation/x86/rs232.c	6.93#implementation/x86/rs232.c
7.47#implementation/unit_test/catch.hpp	Handle Assertions	This parent feature covers the definition, capture, evaluation, and processing of assertion results. It includes handling different assertion types (value, message, exceptions), managing assertion state and reaction, and reporting details about assertion outcomes.	8.2#implementation/unit_test/catch.hpp, 8.8#implementation/unit_test/catch.hpp, 8.10#implementation/unit_test/catch.hpp, 8.12#implementation/unit_test/catch.hpp, 8.14#implementation/unit_test/catch.hpp, 8.15#implementation/unit_test/catch.hpp, 8.16#implementation/unit_test/catch.hpp, 8.17#implementation/unit_test/catch.hpp, 8.18#implementation/unit_test/catch.hpp, 8.21#implementation/unit_test/catch.hpp, 8.22#implementation/unit_test/catch.hpp, 8.23#implementation/unit_test/catch.hpp, 8.42#implementation/unit_test/catch.hpp, 8.43#implementation/unit_test/catch.hpp, 8.48#implementation/unit_test/catch.hpp, 8.51#implementation/unit_test/catch.hpp, 8.59#implementation/unit_test/catch.hpp, 8.70#implementation/unit_test/catch.hpp,	6.127#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
			8.75#implementation/unit_test/catch.hpp, 8.76#implementation/unit_test/catch.hpp, 8.80#implementation/unit_test/catch.hpp, 8.81#implementation/unit_test/catch.hpp, 8.83#implementation/unit_test/catch.hpp, 8.84#implementation/unit_test/catch.hpp, 8.112#implementation/unit_test/catch.hpp, 8.135#implementation/unit_test/catch.hpp, 8.136#implementation/unit_test/catch.hpp, 8.140#implementation/unit_test/catch.hpp, 8.151#implementation/unit_test/catch.hpp, 8.153#implementation/unit_test/catch.hpp, 8.154#implementation/unit_test/catch.hpp, 8.155#implementation/unit_test/catch.hpp, 8.156#implementation/unit_test/catch.hpp, 8.157#implementation/unit_test/catch.hpp, 8.158#implementation/unit_test/catch.hpp, 8.159#implementation/unit_test/catch.hpp, 8.164#implementation/unit_test/catch.hpp, 8.170#implementation/unit_test/catch.hpp, 8.171#implementation/unit_test/catch.hpp, 8.175#implementation/unit_test/catch.hpp, 8.176#implementation/unit_test/catch.hpp, 8.177#implementation/unit_test/catch.hpp, 8.184#implementation/unit_test/catch.hpp, 8.227#implementation/unit_test/catch.hpp, 8.234#implementation/unit_test/catch.hpp, 8.238#implementation/unit_test/catch.hpp, 8.242#implementation/unit_test/catch.hpp, 8.283#implementation/unit_test/catch.hpp	
7.48#implementation/unit_test/catch.hpp	Handle Exceptions and Fatal Errors	This parent feature is responsible for catching, translating, registering, and reporting exceptions and fatal error conditions that occur during test execution, including platform-specific signal and SEH handling.	8.1#implementation/unit_test/catch.hpp, 8.24#implementation/unit_test/catch.hpp, 8.27#implementation/unit_test/catch.hpp, 8.70#implementation/unit_test/catch.hpp, 8.76#implementation/unit_test/catch.hpp, 8.106#implementation/unit_test/catch.hpp, 8.108#implementation/unit_test/catch.hpp, 8.152#implementation/unit_test/catch.hpp, 8.154#implementation/unit_test/catch.hpp, 8.159#implementation/unit_test/catch.hpp,	5.41#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
			8.166#implementation/unit_test/catch.hpp, 8.167#implementation/unit_test/catch.hpp, 8.174#implementation/unit_test/catch.hpp, 8.192#implementation/unit_test/catch.hpp, 8.193#implementation/unit_test/catch.hpp, 8.305#implementation/unit_test/catch.hpp, 8.307#implementation/unit_test/catch.hpp	
7.49#implementation/unit_test/catch.hpp	Implement Benchmarking	This parent feature provides tools and logic for measuring the execution time of code blocks, including high-resolution timers, clock resolution estimation, and a looping utility for running benchmarks.	8.26#implementation/unit_test/catch.hpp, 8.28#implementation/unit_test/catch.hpp, 8.77#implementation/unit_test/catch.hpp, 8.182#implementation/unit_test/catch.hpp, 8.183#implementation/unit_test/catch.hpp, 8.233#implementation/unit_test/catch.hpp, 8.237#implementation/unit_test/catch.hpp, 8.240#implementation/unit_test/catch.hpp, 8.267#implementation/unit_test/catch.hpp, 8.268#implementation/unit_test/catch.hpp, 8.314#implementation/unit_test/catch.hpp, 8.315#implementation/unit_test/catch.hpp	6.130#implementation/unit_test/catch.hpp
7.50#implementation/unit_test/catch.hpp	Implement Matcher Framework	This parent feature provides the infrastructure for defining and using custom matchers in assertions. It includes base interfaces, concrete matchers for various types (strings, floating-point numbers, vectors, Objective-C objects), and logic for combining and negating matchers.	8.20#implementation/unit_test/catch.hpp, 8.34#implementation/unit_test/catch.hpp, 8.35#implementation/unit_test/catch.hpp, 8.36#implementation/unit_test/catch.hpp, 8.37#implementation/unit_test/catch.hpp, 8.38#implementation/unit_test/catch.hpp, 8.40#implementation/unit_test/catch.hpp, 8.41#implementation/unit_test/catch.hpp, 8.45#implementation/unit_test/catch.hpp, 8.48#implementation/unit_test/catch.hpp, 8.115#implementation/unit_test/catch.hpp, 8.116#implementation/unit_test/catch.hpp, 8.120#implementation/unit_test/catch.hpp	6.127#implementation/unit_test/catch.hpp
7.51#implementation/unit_test/catch.hpp	Manage Core Library State	This parent feature covers foundational utilities and the management of global library state, including the test execution context, various internal registries, configuration management, optional	8.3#implementation/unit_test/catch.hpp, 8.6#implementation/unit_test/catch.hpp, 8.11#implementation/unit_test/catch.hpp, 8.19#implementation/unit_test/catch.hpp, 8.53#implementation/unit_test/catch.hpp, 8.60#implementation/unit_test/catch.hpp,	6.128#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		values, random number generation, memory leak detection, and library versioning.	8.63#implementation/unit_test/catch.hpp, 8.68#implementation/unit_test/catch.hpp, 8.69#implementation/unit_test/catch.hpp, 8.82#implementation/unit_test/catch.hpp, 8.104#implementation/unit_test/catch.hpp, 8.107#implementation/unit_test/catch.hpp, 8.109#implementation/unit_test/catch.hpp, 8.117#implementation/unit_test/catch.hpp, 8.118#implementation/unit_test/catch.hpp, 8.124#implementation/unit_test/catch.hpp, 8.148#implementation/unit_test/catch.hpp, 8.149#implementation/unit_test/catch.hpp, 8.165#implementation/unit_test/catch.hpp, 8.166#implementation/unit_test/catch.hpp, 8.167#implementation/unit_test/catch.hpp, 8.244#implementation/unit_test/catch.hpp, 8.286#implementation/unit_test/catch.hpp	
7.52#implementation/unit_test/catch.hpp	Manage Test Execution	This parent feature encompasses the core test execution lifecycle, including registering, discovering, filtering, sorting, and running test cases, test groups, and sections. It also handles Objective-C test method integration and manages the overall test run state and statistics.	8.4#implementation/unit_test/catch.hpp, 8.5#implementation/unit_test/catch.hpp, 8.9#implementation/unit_test/catch.hpp, 8.24#implementation/unit_test/catch.hpp, 8.30#implementation/unit_test/catch.hpp, 8.32#implementation/unit_test/catch.hpp, 8.33#implementation/unit_test/catch.hpp, 8.39#implementation/unit_test/catch.hpp, 8.46#implementation/unit_test/catch.hpp, 8.47#implementation/unit_test/catch.hpp, 8.69#implementation/unit_test/catch.hpp, 8.70#implementation/unit_test/catch.hpp, 8.72#implementation/unit_test/catch.hpp, 8.113#implementation/unit_test/catch.hpp, 8.114#implementation/unit_test/catch.hpp, 8.122#implementation/unit_test/catch.hpp, 8.123#implementation/unit_test/catch.hpp, 8.127#implementation/unit_test/catch.hpp, 8.133#implementation/unit_test/catch.hpp, 8.134#implementation/unit_test/catch.hpp, 8.135#implementation/unit_test/catch.hpp, 8.140#implementation/unit_test/catch.hpp, 8.143#implementation/unit_test/catch.hpp,	6.129#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
			8.144#implementation/unit_test/catch.hpp, 8.145#implementation/unit_test/catch.hpp, 8.146#implementation/unit_test/catch.hpp, 8.147#implementation/unit_test/catch.hpp, 8.150#implementation/unit_test/catch.hpp, 8.165#implementation/unit_test/catch.hpp, 8.166#implementation/unit_test/catch.hpp, 8.167#implementation/unit_test/catch.hpp, 8.168#implementation/unit_test/catch.hpp, 8.169#implementation/unit_test/catch.hpp, 8.178#implementation/unit_test/catch.hpp, 8.179#implementation/unit_test/catch.hpp, 8.180#implementation/unit_test/catch.hpp, 8.185#implementation/unit_test/catch.hpp, 8.186#implementation/unit_test/catch.hpp, 8.187#implementation/unit_test/catch.hpp, 8.188#implementation/unit_test/catch.hpp, 8.189#implementation/unit_test/catch.hpp, 8.192#implementation/unit_test/catch.hpp, 8.193#implementation/unit_test/catch.hpp, 8.199#implementation/unit_test/catch.hpp, 8.200#implementation/unit_test/catch.hpp, 8.201#implementation/unit_test/catch.hpp, 8.202#implementation/unit_test/catch.hpp, 8.203#implementation/unit_test/catch.hpp, 8.206#implementation/unit_test/catch.hpp, 8.207#implementation/unit_test/catch.hpp, 8.208#implementation/unit_test/catch.hpp, 8.209#implementation/unit_test/catch.hpp, 8.210#implementation/unit_test/catch.hpp, 8.211#implementation/unit_test/catch.hpp, 8.215#implementation/unit_test/catch.hpp, 8.216#implementation/unit_test/catch.hpp, 8.218#implementation/unit_test/catch.hpp, 8.219#implementation/unit_test/catch.hpp, 8.220#implementation/unit_test/catch.hpp, 8.221#implementation/unit_test/catch.hpp, 8.222#implementation/unit_test/catch.hpp, 8.223#implementation/unit_test/catch.hpp, 8.241#implementation/unit_test/catch.hpp, 8.263#implementation/unit_test/catch.hpp	

ID	Node Name	Description	Children IDs	Parent IDs
7.53#implementation/unit_test/catch.hpp	Parse Command Line Input	This parent feature handles the interpretation of command-line arguments, including defining options and arguments, binding values to variables or lambdas, tokenizing input, and generating help/identification output.	8.49#implementation/unit_test/catch.hpp, 8.52#implementation/unit_test/catch.hpp, 8.73#implementation/unit_test/catch.hpp, 8.74#implementation/unit_test/catch.hpp, 8.78#implementation/unit_test/catch.hpp, 8.79#implementation/unit_test/catch.hpp, 8.82#implementation/unit_test/catch.hpp, 8.85#implementation/unit_test/catch.hpp, 8.86#implementation/unit_test/catch.hpp, 8.87#implementation/unit_test/catch.hpp, 8.88#implementation/unit_test/catch.hpp, 8.89#implementation/unit_test/catch.hpp, 8.90#implementation/unit_test/catch.hpp, 8.91#implementation/unit_test/catch.hpp, 8.92#implementation/unit_test/catch.hpp, 8.93#implementation/unit_test/catch.hpp, 8.94#implementation/unit_test/catch.hpp, 8.95#implementation/unit_test/catch.hpp, 8.96#implementation/unit_test/catch.hpp, 8.97#implementation/unit_test/catch.hpp, 8.98#implementation/unit_test/catch.hpp, 8.99#implementation/unit_test/catch.hpp, 8.100#implementation/unit_test/catch.hpp, 8.101#implementation/unit_test/catch.hpp, 8.128#implementation/unit_test/catch.hpp, 8.129#implementation/unit_test/catch.hpp, 8.130#implementation/unit_test/catch.hpp, 8.132#implementation/unit_test/catch.hpp, 8.141#implementation/unit_test/catch.hpp, 8.142#implementation/unit_test/catch.hpp, 8.180#implementation/unit_test/catch.hpp, 8.181#implementation/unit_test/catch.hpp	6.128#implementation/unit_test/catch.hpp
7.54#implementation/unit_test/catch.hpp	Provide String Manipulation and Conversion	This parent feature includes various utilities for manipulating strings, converting different data types (standard types, enumerations, containers, chrono durations, raw memory) to string representations, and formatting output.	8.1#implementation/unit_test/catch.hpp, 8.13#implementation/unit_test/catch.hpp, 8.31#implementation/unit_test/catch.hpp, 8.91#implementation/unit_test/catch.hpp, 8.120#implementation/unit_test/catch.hpp, 8.194#implementation/unit_test/catch.hpp, 8.195#implementation/unit_test/catch.hpp, 8.196#implementation/unit_test/catch.hpp,	6.130#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
			8.197#implementation/unit_test/catch.hpp, 8.202#implementation/unit_test/catch.hpp, 8.204#implementation/unit_test/catch.hpp, 8.205#implementation/unit_test/catch.hpp, 8.214#implementation/unit_test/catch.hpp, 8.217#implementation/unit_test/catch.hpp, 8.224#implementation/unit_test/catch.hpp, 8.226#implementation/unit_test/catch.hpp, 8.229#implementation/unit_test/catch.hpp, 8.230#implementation/unit_test/catch.hpp, 8.231#implementation/unit_test/catch.hpp, 8.232#implementation/unit_test/catch.hpp, 8.235#implementation/unit_test/catch.hpp, 8.236#implementation/unit_test/catch.hpp, 8.239#implementation/unit_test/catch.hpp, 8.249#implementation/unit_test/catch.hpp	
7.55#implementation/unit_test/catch.hpp	Report Test Results	This parent feature includes all aspects of outputting test results, supporting various formats (Console, XML, JUnit), managing different reporter types and listeners, controlling console color, redirecting standard streams, and formatting output details.	8.7#implementation/unit_test/catch.hpp, 8.24#implementation/unit_test/catch.hpp, 8.29#implementation/unit_test/catch.hpp, 8.50#implementation/unit_test/catch.hpp, 8.54#implementation/unit_test/catch.hpp, 8.55#implementation/unit_test/catch.hpp, 8.56#implementation/unit_test/catch.hpp, 8.57#implementation/unit_test/catch.hpp, 8.58#implementation/unit_test/catch.hpp, 8.61#implementation/unit_test/catch.hpp, 8.62#implementation/unit_test/catch.hpp, 8.64#implementation/unit_test/catch.hpp, 8.65#implementation/unit_test/catch.hpp, 8.66#implementation/unit_test/catch.hpp, 8.67#implementation/unit_test/catch.hpp, 8.71#implementation/unit_test/catch.hpp, 8.74#implementation/unit_test/catch.hpp, 8.102#implementation/unit_test/catch.hpp, 8.103#implementation/unit_test/catch.hpp, 8.110#implementation/unit_test/catch.hpp, 8.111#implementation/unit_test/catch.hpp, 8.119#implementation/unit_test/catch.hpp, 8.121#implementation/unit_test/catch.hpp, 8.126#implementation/unit_test/catch.hpp,	6.131#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
			8.127#implementation/unit_test/catch.hpp, 8.131#implementation/unit_test/catch.hpp, 8.137#implementation/unit_test/catch.hpp, 8.138#implementation/unit_test/catch.hpp, 8.139#implementation/unit_test/catch.hpp, 8.145#implementation/unit_test/catch.hpp, 8.146#implementation/unit_test/catch.hpp, 8.147#implementation/unit_test/catch.hpp, 8.154#implementation/unit_test/catch.hpp, 8.160#implementation/unit_test/catch.hpp, 8.161#implementation/unit_test/catch.hpp, 8.162#implementation/unit_test/catch.hpp, 8.163#implementation/unit_test/catch.hpp, 8.165#implementation/unit_test/catch.hpp, 8.172#implementation/unit_test/catch.hpp, 8.173#implementation/unit_test/catch.hpp, 8.175#implementation/unit_test/catch.hpp, 8.185#implementation/unit_test/catch.hpp, 8.186#implementation/unit_test/catch.hpp, 8.191#implementation/unit_test/catch.hpp, 8.198#implementation/unit_test/catch.hpp, 8.204#implementation/unit_test/catch.hpp, 8.205#implementation/unit_test/catch.hpp, 8.226#implementation/unit_test/catch.hpp, 8.227#implementation/unit_test/catch.hpp, 8.228#implementation/unit_test/catch.hpp, 8.232#implementation/unit_test/catch.hpp, 8.234#implementation/unit_test/catch.hpp, 8.235#implementation/unit_test/catch.hpp, 8.243#implementation/unit_test/catch.hpp, 8.245#implementation/unit_test/catch.hpp, 8.246#implementation/unit_test/catch.hpp, 8.247#implementation/unit_test/catch.hpp, 8.248#implementation/unit_test/catch.hpp, 8.249#implementation/unit_test/catch.hpp, 8.250#implementation/unit_test/catch.hpp, 8.251#implementation/unit_test/catch.hpp, 8.252#implementation/unit_test/catch.hpp, 8.253#implementation/unit_test/catch.hpp, 8.254#implementation/unit_test/catch.hpp, 8.255#implementation/unit_test/catch.hpp,	

ID	Node Name	Description	Children IDs	Parent IDs
			8.256#implementation/unit_test/catch.hpp, 8.257#implementation/unit_test/catch.hpp, 8.258#implementation/unit_test/catch.hpp, 8.259#implementation/unit_test/catch.hpp, 8.260#implementation/unit_test/catch.hpp, 8.261#implementation/unit_test/catch.hpp, 8.262#implementation/unit_test/catch.hpp, 8.264#implementation/unit_test/catch.hpp, 8.265#implementation/unit_test/catch.hpp, 8.266#implementation/unit_test/catch.hpp, 8.267#implementation/unit_test/catch.hpp, 8.268#implementation/unit_test/catch.hpp, 8.269#implementation/unit_test/catch.hpp, 8.270#implementation/unit_test/catch.hpp, 8.271#implementation/unit_test/catch.hpp, 8.272#implementation/unit_test/catch.hpp, 8.273#implementation/unit_test/catch.hpp, 8.274#implementation/unit_test/catch.hpp, 8.275#implementation/unit_test/catch.hpp, 8.276#implementation/unit_test/catch.hpp, 8.277#implementation/unit_test/catch.hpp, 8.278#implementation/unit_test/catch.hpp, 8.279#implementation/unit_test/catch.hpp, 8.280#implementation/unit_test/catch.hpp, 8.281#implementation/unit_test/catch.hpp, 8.282#implementation/unit_test/catch.hpp, 8.284#implementation/unit_test/catch.hpp, 8.285#implementation/unit_test/catch.hpp, 8.287#implementation/unit_test/catch.hpp, 8.288#implementation/unit_test/catch.hpp, 8.289#implementation/unit_test/catch.hpp, 8.290#implementation/unit_test/catch.hpp, 8.291#implementation/unit_test/catch.hpp, 8.292#implementation/unit_test/catch.hpp, 8.293#implementation/unit_test/catch.hpp, 8.294#implementation/unit_test/catch.hpp, 8.295#implementation/unit_test/catch.hpp, 8.296#implementation/unit_test/catch.hpp, 8.297#implementation/unit_test/catch.hpp, 8.298#implementation/unit_test/catch.hpp, 8.299#implementation/unit_test/catch.hpp,	

ID	Node Name	Description	Children IDs	Parent IDs
			8.300#implementation/unit_test/catch.hpp, 8.301#implementation/unit_test/catch.hpp, 8.302#implementation/unit_test/catch.hpp, 8.303#implementation/unit_test/catch.hpp, 8.304#implementation/unit_test/catch.hpp, 8.306#implementation/unit_test/catch.hpp, 8.308#implementation/unit_test/catch.hpp, 8.309#implementation/unit_test/catch.hpp, 8.310#implementation/unit_test/catch.hpp, 8.311#implementation/unit_test/catch.hpp, 8.312#implementation/unit_test/catch.hpp, 8.313#implementation/unit_test/catch.hpp, 8.314#implementation/unit_test/catch.hpp, 8.315#implementation/unit_test/catch.hpp, 8.316#implementation/unit_test/catch.hpp, 8.317#implementation/unit_test/catch.hpp, 8.318#implementation/unit_test/catch.hpp, 8.319#implementation/unit_test/catch.hpp, 8.320#implementation/unit_test/catch.hpp, 8.321#implementation/unit_test/catch.hpp, 8.322#implementation/unit_test/catch.hpp, 8.323#implementation/unit_test/catch.hpp, 8.324#implementation/unit_test/catch.hpp, 8.325#implementation/unit_test/catch.hpp, 8.326#implementation/unit_test/catch.hpp, 8.327#implementation/unit_test/catch.hpp, 8.328#implementation/unit_test/catch.hpp, 8.329#implementation/unit_test/catch.hpp, 8.330#implementation/unit_test/catch.hpp, 8.331#implementation/unit_test/catch.hpp, 8.332#implementation/unit_test/catch.hpp, 8.333#implementation/unit_test/catch.hpp, 8.334#implementation/unit_test/catch.hpp, 8.335#implementation/unit_test/catch.hpp, 8.336#implementation/unit_test/catch.hpp, 8.337#implementation/unit_test/catch.hpp, 8.338#implementation/unit_test/catch.hpp	
7.56#implementation/unit_test/catch.hpp	Track Test Execution Flow	This parent feature manages the state and progress of test cases and sections during execution,	8.72#implementation/unit_test/catch.hpp, 8.143#implementation/unit_test/catch.hpp, 8.144#implementation/unit_test/catch.hpp,	6.129#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		enabling features like running specific sections or handling generator-based loops across test cycles.	8.169#implementation/unit_test/catch.hpp, 8.187#implementation/unit_test/catch.hpp, 8.189#implementation/unit_test/catch.hpp, 8.208#implementation/unit_test/catch.hpp, 8.213#implementation/unit_test/catch.hpp, 8.216#implementation/unit_test/catch.hpp, 8.222#implementation/unit_test/catch.hpp, 8.223#implementation/unit_test/catch.hpp	
7.57#implementation/unit_test/catch.hpp	Report test results and messages	Includes functionalities for formatting and displaying test assertion expressions, managing test-related messages, and providing access to standard output streams for reporting.	8.105#implementation/unit_test/catch.hpp, 8.125#implementation/unit_test/catch.hpp, 8.212#implementation/unit_test/catch.hpp	6.131#implementation/unit_test/catch.hpp
8.1#implementation/unit_test/catch.hpp	Convert Data Types to Strings and Provide Utilities	Contains utilities and type traits for converting arbitrary data types into string representations suitable for display in assertion messages and reports, including handling raw memory and checking stream insertability.	CS-1.1#implementation/unit_test/catch.hpp, CS-1.1#implementation/unit_test/catch.hpp, CS-1.1#implementation/unit_test/catch.hpp, CS-1.1#implementation/unit_test/catch.hpp, CS-1.1#implementation/unit_test/catch.hpp, CS-1.1#implementation/unit_test/catch.hpp, CS-1.1#implementation/unit_test/catch.hpp	7.48#implementation/unit_test/catch.hpp, 7.54#implementation/unit_test/catch.hpp
8.2#implementation/unit_test/catch.hpp	Define Assertion Result Types and Dispositions	Enumerates the possible outcomes of an assertion (e.g., Ok, Failure, Exception) and flags that control how failures are handled (e.g., ContinueOnFailure, SuppressFail), along with helper functions to query these flags.	CS-1.2#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.3#implementation/unit_test/catch.hpp	Define Basic Utilities and Types	Introduces foundational structures and macros used throughout the library, such as `SourceLineInfo` to store code location, `NonCopyable` base class, a helper for stream operations (`StreamEndStop`), and macros for generating unique names.	CS-1.3#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
8.4#implementation/unit_test/catch.hpp	Define Test Case Registration Macros	Provides user-friendly macros (like CATCH_TESTCASE, CATCH_METHOD_AS_TEST_CASE, CATCH_TEST_CASE_METHOD) that hide the complexity of test registration, automatically creating test invokers and registrar objects.	CS-1.4#implementation/unit_test/catch.hpp, CS-1.4#implementation/unit_test/catch.hpp, CS-1.4#implementation/unit_test/catch.hpp, CS-1.4#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.5#implementation/unit_test/catch.hpp	Define Test Case Registry and Interfaces	Establishes the core infrastructure for discovering and managing test cases, including interfaces for invoking tests (<code>ITestInvoker`</code> ), registering tests ( <code>ITestCaseRegistry`</code> , <code>AutoReg`</code> ), and structures to hold test metadata ( <code>TestCase`</code> , <code>NameAndTags`</code>).	CS-1.5#implementation/unit_test/catch.hpp, CS-1.5#implementation/unit_test/catch.hpp, CS-1.5#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.6#implementation/unit_test/catch.hpp	Manage Build Configuration and Compiler Capabilities	This feature uses preprocessor defines and compiler-specific checks to configure how the Catch library is built and to detect supported compiler/platform features like <code>__COUNTER__</code> , Windows SEH, and POSIX signals, enabling conditional compilation.	CS-1.6#implementation/unit_test/catch.hpp, CS-1.6#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp
8.7#implementation/unit_test/catch.hpp	Manage Output Streams	Provides interfaces and implementations for standard output streams (<code>cout`</code> , <code>cerr`</code> , <code>clog`</code> ) and a reusable string stream utility ( <code>ReusableStringStream`</code>), used for directing test and assertion output.	CS-1.7#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.8#implementation/unit_test/catch.hpp	Prepare Expression for Assertion Output	Starts the definition of a mechanism (<code>catch_decomposer.h`</code>) to break down complex C++ expressions within assertion macros (like <code>REQUIRE</code>) into parts that can be individually evaluated and streamed for detailed failure reporting.	CS-1.8#implementation/unit_test/catch.hpp, CS-1.8#implementation/unit_test/catch.hpp, CS-1.8#implementation/unit_test/catch.hpp, CS-1.8#implementation/unit_test/catch.hpp, CS-1.8#implementation/unit_test/catch.hpp, CS-1.8#implementation/unit_test/catch.hpp, CS-1.8#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
			CS-1.13#implementation/unit_test/catch.hpp, CS-1.13#implementation/unit_test/catch.hpp, CS-1.13#implementation/unit_test/catch.hpp, CS-1.13#implementation/unit_test/catch.hpp, CS-1.13#implementation/unit_test/catch.hpp, CS-1.13#implementation/unit_test/catch.hpp, CS-1.13#implementation/unit_test/catch.hpp, CS-1.13#implementation/unit_test/catch.hpp, CS-1.13#implementation/unit_test/catch.hpp, CS-1.13#implementation/unit_test/catch.hpp, CS-1.13#implementation/unit_test/catch.hpp, CS-1.13#implementation/unit_test/catch.hpp, CS-1.13#implementation/unit_test/catch.hpp	
8.14#implementation/unit_test/catch.hpp	Decompose expressions for capture	Uses operator overloading within `ExprLhs` and `Decomposer` to enable the capture of test expressions at compile time, turning them into `ITransientExpression` objects for later evaluation and reporting.	CS-1.14#implementation/unit_test/catch.hpp, CS-1.14#implementation/unit_test/catch.hpp, CS-1.14#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.15#implementation/unit_test/catch.hpp	Define result capture interface	Defines the `IResultCapture` pure virtual interface, specifying the contract for types that handle and report test results, sections, messages, benchmarks, and errors.	CS-1.15#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.16#implementation/unit_test/catch.hpp	Handle assertion results and reactions	Provides the `AssertionHandler` class responsible for processing different types of assertion outcomes (expressions, messages, exceptions), interacting with the result capture system, and determining immediate reactions like breaking into the debugger or throwing.	CS-1.16#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.17#implementation/unit_test/catch.hpp	Represent and format transient expressions	Defines the structure (`ITransientExpression`, `BinaryExpr`, `UnaryExpr`) to	CS-1.17#implementation/unit_test/catch.hpp, CS-1.17#implementation/unit_test/catch.hpp, CS-1.17#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		represent expressions captured during test execution, facilitating their reconstruction and streaming for reporting.		
8.18#implementation/unit_test/catch.hpp	Specialize comparison operators for expression capture	Provides specialized template functions to handle comparisons, particularly for cases involving pointers and integer literals, ensuring correct behavior during expression decomposition.	CS-1.18#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.19#implementation/unit_test/catch.hpp	Control compilation based on configuration	Uses preprocessor directives to conditionally include or exclude code and define macros based on configuration flags like CATCH_CONFIG_DISABLE and CATCH_CONFIG_FAST_COMPILE, allowing customization of the library's behavior and compilation speed.	CS-1.19#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp
8.20#implementation/unit_test/catch.hpp	Define base matcher class	Provides a base class (MatcherUntypedBase) for implementing custom matchers, ensuring they have common functionality like converting their description to a string.	CS-1.20#implementation/unit_test/catch.hpp	7.50#implementation/unit_test/catch.hpp
8.21#implementation/unit_test/catch.hpp	Define core assertion macros	Provides macros like INTERNAL_CATCH_TEST, INTERNAL_CATCH_IF, and INTERNAL_CATCH_ELSE for basic value assertions and conditional execution based on assertion results. These macros capture the expression, handle potential exceptions during evaluation, and report results.	CS-1.21#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.22#implementation/unit_test/catch.hpp	Define exception assertion	Provides macros	CS-1.22#implementation/unit_test/catch.hpp,	7.47#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
nit_test/catch.hpp	macros	(INTERNAL_CATCH_NO_THROW, INTERNAL_CATCH_THROWS, INTERNAL_CATCH_THROWS_AS, INTERNAL_CATCH_THROWS_STR_MATCHES) specifically designed to assert whether a block of code throws an exception, a specific type of exception, or an exception whose message matches a criteria.	CS-1.22#implementation/unit_test/catch.hpp, CS-1.22#implementation/unit_test/catch.hpp, CS-1.22#implementation/unit_test/catch.hpp	nit_test/catch.hpp
8.23#implementation/unit_test/catch.hpp	Define message and info logging macros	Provides macros (INTERNAL_CATCH_MSG, INTERNAL_CATCH_INFO) to log messages with different levels of importance or result types within a test, separate from assertions.	CS-1.23#implementation/unit_test/catch.hpp, CS-1.23#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.24#implementation/unit_test/catch.hpp	Define registry interfaces	Defines pure virtual interfaces (IRegistryHub, IMutableRegistryHub) for accessing and modifying Catch2's internal registries, such as those for reporters, test cases, tag aliases, and exception translators.	CS-1.24#implementation/unit_test/catch.hpp	7.48#implementation/unit_test/catch.hpp, 7.52#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.25#implementation/unit_test/catch.hpp	Implement approximate floating-point comparison	Provides the Approx class to allow flexible comparison of floating-point numbers, enabling specification of absolute margin, relative epsilon, and scaling factor for tolerance.	CS-1.25#implementation/unit_test/catch.hpp	6.127#implementation/unit_test/catch.hpp
8.26#implementation/unit_test/catch.hpp	Implement benchmark looping utility	Provides the BenchmarkLooper class and the BENCHMARK macro to facilitate simple performance benchmarking by looping a section of code and measuring its execution time.	CS-1.26#implementation/unit_test/catch.hpp, CS-1.26#implementation/unit_test/catch.hpp	7.49#implementation/unit_test/catch.hpp
8.27#implementation/unit_test/catch.hpp	Implement exception translation and registration	Provides infrastructure for translating exceptions to strings, including interfaces	CS-1.27#implementation/unit_test/catch.hpp, CS-1.27#implementation/unit_test/catch.hpp, CS-1.27#implementation/unit_test/catch.hpp,	7.48#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		(IExceptionTranslator, IExceptionTranslatorRegistry) and a registrar (ExceptionTranslatorRegistrar) to hook custom exception translation functions into the framework.	CS-1.27#implementation/unit_test/catch.hpp	
8.28#implementation/unit_test/catch.hpp	Implement high-resolution timer	Provides utility functions to get the current time and clock resolution, and a Timer class to measure elapsed time with high precision in nanoseconds, microseconds, milliseconds, and seconds.	CS-1.28#implementation/unit_test/catch.hpp	7.49#implementation/unit_test/catch.hpp
8.29#implementation/unit_test/catch.hpp	Manage test counts and totals	Defines structures (Counts, Totals) to track the number of passed, failed, and failed but ok assertions and test cases, including methods for arithmetic operations on these counts.	CS-1.29#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.30#implementation/unit_test/catch.hpp	Manage test section execution	Defines the Section class which uses RAII to manage the execution state of a test section, ensuring setup and teardown logic is handled, and provides an operator bool to determine if the section should be executed.	CS-1.30#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.31#implementation/unit_test/catch.hpp	Provide string manipulation utilities	Offers a collection of helper functions for common string operations like checking prefixes/suffixes, finding substrings, case conversion, trimming, and replacement.	CS-1.31#implementation/unit_test/catch.hpp	7.54#implementation/unit_test/catch.hpp
8.32#implementation/unit_test/catch.hpp	Provide test section macro	Defines the INTERNAL_CATCH_SECTION macro to simplify the creation and use of Catch test sections, allowing code blocks to be defined as	CS-1.32#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		distinct, manageable sections within a test case.		
8.33#implementation/unit_test/catch.hpp	Store test section information	Defines structures (SectionInfo, SectionEndInfo) to hold details about a test section, including its name, description, source line information, previous assertion counts, and duration.	CS-1.33#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.34#implementation/unit_test/catch.hpp	Combine matchers with logical AND	Implements a matcher that succeeds if and only if all contained matchers succeed. It allows combining multiple matchers using the logical AND operator.	CS-1.34#implementation/unit_test/catch.hpp	7.50#implementation/unit_test/catch.hpp
8.35#implementation/unit_test/catch.hpp	Combine matchers with logical OR	Implements a matcher that succeeds if at least one of the contained matchers succeeds. It allows combining multiple matchers using the logical OR operator.	CS-1.35#implementation/unit_test/catch.hpp	7.50#implementation/unit_test/catch.hpp
8.36#implementation/unit_test/catch.hpp	Define floating point matchers	Provides matchers for comparing floating-point numbers, allowing checks for approximate equality within an absolute margin or within a specified number of ULPs (Units in the Last Place).	CS-1.36#implementation/unit_test/catch.hpp	7.50#implementation/unit_test/catch.hpp
8.37#implementation/unit_test/catch.hpp	Define fundamental matcher interfaces	Defines the base virtual interface for matchers, enabling polymorphism across different matcher types and handling both value and pointer types.	CS-1.37#implementation/unit_test/catch.hpp, CS-1.37#implementation/unit_test/catch.hpp, CS-1.37#implementation/unit_test/catch.hpp	7.50#implementation/unit_test/catch.hpp
8.38#implementation/unit_test/catch.hpp	Define Objective-C NSString matchers	Provides specific matchers for comparing Objective-C `NSString*` objects, including checks for equality, containment, prefixes, and suffixes.	CS-1.38#implementation/unit_test/catch.hpp, CS-1.38#implementation/unit_test/catch.hpp	7.50#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
8.39#implementation/unit_test/catch.hpp	Define Objective-C test case invocation	Provides mechanism (`OcMethod`, `registerTestMethods`) to discover and invoke Objective-C methods marked as test cases, including optional setup and teardown steps defined in the `OcFixture` protocol.	CS-1.39#implementation/unit_test/catch.hpp, CS-1.39#implementation/unit_test/catch.hpp, CS-1.39#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.40#implementation/unit_test/catch.hpp	Define string matchers	Provides matchers for comparing strings, including checks for equality, containment, prefixes, suffixes, and regular expression matching, with support for case sensitivity.	CS-1.40#implementation/unit_test/catch.hpp	7.50#implementation/unit_test/catch.hpp
8.41#implementation/unit_test/catch.hpp	Define vector matchers	Provides matchers for comparing vectors, including checks for element containment, subset containment, strict equality (order matters), and unordered equality (permutation).	CS-1.41#implementation/unit_test/catch.hpp, CS-1.41#implementation/unit_test/catch.hpp, CS-1.41#implementation/unit_test/catch.hpp, CS-1.41#implementation/unit_test/catch.hpp, CS-1.41#implementation/unit_test/catch.hpp	7.50#implementation/unit_test/catch.hpp
8.42#implementation/unit_test/catch.hpp	Implement exception matching assertions	Provides a macro (`INTERNAL_CATCH_THROWS_MATCHES`) to assert that a code block throws a specific exception type, and that the thrown exception object matches a provided matcher.	CS-1.42#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.43#implementation/unit_test/catch.hpp	Implement general matcher assertions	Provides a macro (`INTERNAL_CHECK_THAT`) to create and handle test assertions using a matcher, integrating with the assertion reporting mechanism.	CS-1.43#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.44#implementation/unit_test/catch.hpp	Match strings using wildcard patterns	Provides a utility class (`WildcardPattern`) to check if a given string matches a pattern containing '*' wildcards, with options for case sensitivity.	CS-1.44#implementation/unit_test/catch.hpp	6.127#implementation/unit_test/catch.hpp
8.45#implementation/unit_test/catch.hpp	Negate a matcher's result	Implements a matcher that	CS-1.45#implementation/unit_test/catch.hpp	7.50#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
nit_test/catch.hpp		succeeds if and only if the underlying matcher fails. It allows negating a matcher's result using the logical NOT operator.		nit_test/catch.hpp
8.46#implementation/unit_test/catch.hpp	Parse test specification strings	Defines a parser (`TestSpecParser`) to interpret command-line arguments or strings specifying which test cases to run, supporting names, tags, exclusions, and wildcard patterns.	CS-1.46#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.47#implementation/unit_test/catch.hpp	Represent a test case selection specification	Defines the `TestSpec` class which holds the parsed structure of filters (combinations of patterns) used to determine which test cases match the user's selection criteria.	CS-1.47#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.48#implementation/unit_test/catch.hpp	Represent matcher assertions as expressions	Defines a class to hold information about a matcher assertion, including the argument, the matcher object, and the expression string, allowing it to be evaluated and reported.	CS-1.48#implementation/unit_test/catch.hpp, CS-1.48#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp, 7.50#implementation/unit_test/catch.hpp
8.49#implementation/unit_test/catch.hpp	Define Configuration Interface	Defines an interface (IConfig) for accessing various test configuration options, ensuring consistent access regardless of the underlying configuration implementation.	CS-1.49#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.50#implementation/unit_test/catch.hpp	Define Reporting Interfaces	Specifies interfaces (IStreamingReporter, IReporterFactory, IReporterRegistry) for how test results are reported in a streaming fashion and how reporters are created and managed.	CS-1.50#implementation/unit_test/catch.hpp, CS-1.50#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.51#implementation/unit_test/catch.hpp	Handle Assertion Results	Provides structures (AssertionResultData, AssertionResult) to store details	CS-1.51#implementation/unit_test/catch.hpp, CS-1.51#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		about the outcome of a single assertion, including its expression, message, and result type.		
8.52#implementation/unit_test/catch.hpp	Implement Configuration Management	Provides a structure (ConfigData) to hold all configuration parameters and a class (Config) that implements the IConfig interface, storing and managing the configuration state.	CS-1.52#implementation/unit_test/catch.hpp, CS-1.52#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.53#implementation/unit_test/catch.hpp	Implement Optional Value Type	Provides a basic template class (Option) to wrap values that may or may not be present, and a derived struct (LazyStat) specifically for statistics that might be computed lazily.	CS-1.53#implementation/unit_test/catch.hpp, CS-1.53#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp
8.54#implementation/unit_test/catch.hpp	Implement Reporter Registration	Provides template classes (ReporterRegistrar, ListenerRegistrar) to simplify the process of registering custom reporter and listener factories with the framework's registry.	CS-1.54#implementation/unit_test/catch.hpp, CS-1.54#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.55#implementation/unit_test/catch.hpp	Manage Console Output Colour	Provides a struct (Colour) and stream operator overload to control text colour for console output, defining various colour codes for different reporting elements.	CS-1.55#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.56#implementation/unit_test/catch.hpp	Provide Base Class for Cumulative Reporters	Offers a template base class (CumulativeReporterBase) for reporters that collect and aggregate test results in a hierarchical structure before reporting the final summary.	CS-1.56#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.57#implementation/unit_test/catch.hpp	Provide Base Class for Streaming Reporters	Offers a template base class (StreamingReporterBase) for reporters that process test events	CS-1.57#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		as they occur, providing common setup and state management.		
8.58#implementation/unit_test/catch.hpp	Store Test Statistics	Defines structures (TestRunInfo, GroupInfo, AssertionStats, SectionStats, TestCaseStats, TestGroupStats, TestRunStats, BenchmarkInfo, BenchmarkStats) for accumulating and storing statistical data at different levels of the test execution hierarchy.	CS-1.58#implementation/unit_test/catch.hpp, CS-1.58#implementation/unit_test/catch.hpp, CS-1.58#implementation/unit_test/catch.hpp, CS-1.58#implementation/unit_test/catch.hpp, CS-1.58#implementation/unit_test/catch.hpp, CS-1.58#implementation/unit_test/catch.hpp, CS-1.58#implementation/unit_test/catch.hpp, CS-1.58#implementation/unit_test/catch.hpp, CS-1.58#implementation/unit_test/catch.hpp, CS-1.58#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.59#implementation/unit_test/catch.hpp	Compare floating-point numbers with tolerance	Implements approximate comparison for floating-point numbers using epsilon and margin tolerances, allowing for flexible comparison in tests.	CS-1.59#implementation/unit_test/catch.hpp, CS-1.59#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.60#implementation/unit_test/catch.hpp	Detect memory leaks	Provides a simple struct (presumably with implementation elsewhere) to facilitate memory leak detection.	CS-1.60#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp
8.61#implementation/unit_test/catch.hpp	Encode strings for XML output	Provides a utility class to encode strings for safe inclusion in XML elements or attributes, handling special characters.	CS-1.61#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.62#implementation/unit_test/catch.hpp	Generate XML output	Provides a utility class for building XML documents programmatically, with features for starting/ending elements, writing attributes and text, managing indentation, and ensuring tags are correctly closed.	CS-1.62#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.63#implementation/unit_test/catch.hpp	Handle fatal error conditions	Provides platform-specific mechanisms (like SEH on Windows or signals on POSIX) to catch and potentially report fatal errors during test execution.	CS-1.63#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
8.64#implementation/unit_test/catch.hpp	Implement Compact test reporter	Defines a test reporter that outputs results in a compact format, providing implementations for various test run events like test run end, assertion results, and section ends.	CS-1.64#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.65#implementation/unit_test/catch.hpp	Implement Console test reporter	Defines a test reporter for outputting test results to the console. It handles various test events, including assertions, sections, benchmarks, test cases, groups, and the overall test run.	CS-1.65#implementation/unit_test/catch.hpp, CS-1.65#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.66#implementation/unit_test/catch.hpp	Implement generic XML test reporter	Defines a test reporter that generates output in a general XML format. It handles various test events and uses the XmlWriter utility.	CS-1.66#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.67#implementation/unit_test/catch.hpp	Implement JUnit XML test reporter	Defines a test reporter that generates output in the JUnit XML format, suitable for integration with build systems. It uses the XmlWriter utility.	CS-1.67#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.68#implementation/unit_test/catch.hpp	Integrate with debugger	Provides functionality to detect if a debugger is attached and trigger a breakpoint programmatically, aiding in debugging test failures.	CS-1.68#implementation/unit_test/catch.hpp, CS-1.68#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp
8.69#implementation/unit_test/catch.hpp	Manage test execution context	Defines interfaces and functions (`IContext`, `IMutableContext`, `getCurrentMutableContext`) to manage the global state during a test run, providing access to components like result capture, the test runner, and configuration.	CS-1.69#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp, 7.52#implementation/unit_test/catch.hpp
8.70#implementation/unit_test/catch.hpp	Manage the test execution run	Implements the core logic for running tests (`IRunner`) and	CS-1.70#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp, 7.48

ID	Node Name	Description	Children IDs	Parent IDs
		capturing results (`IResultCapture`). It processes test groups, cases, sections, assertions, and exceptions, interacting with the context and reporter.		#implementation/unit_test/catch.hpp, 7.52#implementation/unit_test/catch.hpp
8.71#implementation/unit_test/catch.hpp	Register custom reporters and listeners	Provides macros to register custom reporter and listener types with the Catch2 framework, optionally disabled via a configuration flag.	CS-1.71#implementation/unit_test/catch.hpp, CS-1.71#implementation/unit_test/catch.hpp, CS-1.71#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.72#implementation/unit_test/catch.hpp	Track test case and section execution	Provides structures and logic to track the progress and state of test cases, sections, and loops (`ITracker`, `TrackerContext`, `SectionTracker`, `IndexTracker`) within a test run, essential for managing nested test structures and test restarts.	CS-1.72#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.56#implementation/unit_test/catch.hpp
8.73#implementation/unit_test/catch.hpp	Analyze lambda traits	Provides template traits (`UnaryLambdaTraits`) to extract the argument and return types of unary (single-argument) lambdas. This is useful for static analysis and type deduction, likely within the command-line parsing logic.	CS-1.73#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.74#implementation/unit_test/catch.hpp	Format text into columns	Provides utility classes (`Column`, `Spacer`, `Columns`) for wrapping and arranging text into formatted columns suitable for command-line output or help messages. It includes logic for breaking text at appropriate points and managing indentation and column width.	CS-1.74#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.75#implementation/unit_test/catch.hpp	Handle assertions	Implements the `AssertionHandler` class, responsible for processing assertion outcomes. It stores assertion details, interacts with a	CS-1.75#implementation/unit_test/catch.hpp, CS-1.75#implementation/unit_test/catch.hpp, CS-1.75#implementation/unit_test/catch.hpp, CS-1.75#implementation/unit_test/catch.hpp,	7.47#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		`ResultCapture` object to record results, handles different assertion types (expression, message, exceptions), checks configuration for allowed throws, and completes the assertion handling by potentially breaking into the debugger or throwing an exception.	CS-1.75#implementation/unit_test/catch.hpp, CS-1.75#implementation/unit_test/catch.hpp, CS-1.75#implementation/unit_test/catch.hpp, CS-1.75#implementation/unit_test/catch.hpp, CS-1.75#implementation/unit_test/catch.hpp, CS-1.75#implementation/unit_test/catch.hpp, CS-1.75#implementation/unit_test/catch.hpp	
8.76#implementation/unit_test/catch.hpp	Handle exception match expression	Provides helper functions to handle assertions that check if an exception's message matches a given string or a string matcher. It translates the active exception and passes it to the assertion handler.	CS-1.76#implementation/unit_test/catch.hpp, CS-1.76#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp, 7.48#implementation/unit_test/catch.hpp
8.77#implementation/unit_test/catch.hpp	Loop for benchmarking	Implements the `BenchmarkLooper` class to manage benchmark iterations. It calculates required timer resolution based on configuration, reports the start of a benchmark, and determines if more iterations are needed based on elapsed time, reporting the final result when complete.	CS-1.77#implementation/unit_test/catch.hpp, CS-1.77#implementation/unit_test/catch.hpp, CS-1.77#implementation/unit_test/catch.hpp	7.49#implementation/unit_test/catch.hpp
8.78#implementation/unit_test/catch.hpp	Manage command line arguments	Defines the `Args` class to store and provide access to command line arguments passed to the program, separating the executable name from the arguments.	CS-1.78#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.79#implementation/unit_test/catch.hpp	Manage command line parse state	Defines `ParseResultType` enum and `ParseState` class to represent the outcome and remaining tokens after a command-line parsing attempt.	CS-1.79#implementation/unit_test/catch.hpp, CS-1.79#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.80#implementation/unit_test/catch.hpp	Represent final assertion result	Defines the `AssertionResult` class which combines assertion information (`AssertionInfo`) and	CS-1.80#implementation/unit_test/catch.hpp, CS-1.80#implementation/unit_test/catch.hpp, CS-1.80#implementation/unit_test/catch.hpp,	7.47#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		result data (`AssertionResultData`) to provide a complete representation of an assertion's outcome. It offers methods to query success status, result type, presence of expression/message, retrieve original and expanded expressions, message, source info, and macro name.	CS-1.80#implementation/unit_test/catch.hpp, CS-1.80#implementation/unit_test/catch.hpp, CS-1.80#implementation/unit_test/catch.hpp, CS-1.80#implementation/unit_test/catch.hpp, CS-1.80#implementation/unit_test/catch.hpp, CS-1.80#implementation/unit_test/catch.hpp, CS-1.80#implementation/unit_test/catch.hpp, CS-1.80#implementation/unit_test/catch.hpp, CS-1.80#implementation/unit_test/catch.hpp, CS-1.80#implementation/unit_test/catch.hpp	
8.81#implementation/unit_test/catch.hpp	Represent lazy expression	Defines the `LazyExpression` class which holds an expression that is not necessarily evaluated immediately. It stores whether the expression is negated and a pointer to a transient expression object. It provides methods for construction, copying, checking validity (operator bool), and streaming (operator<<) while handling negation and potential errors.	CS-1.81#implementation/unit_test/catch.hpp, CS-1.81#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.82#implementation/unit_test/catch.hpp	Represent operation results	Defines a hierarchy of classes (`ResultBase`, `ResultValueBase`, `BasicResult`) to represent the outcome of operations (Ok, LogicError, RuntimeError). `BasicResult` can hold a value for successful outcomes and an error message for failures, providing methods to check the type and access the value or error message.	CS-1.82#implementation/unit_test/catch.hpp, CS-1.82#implementation/unit_test/catch.hpp, CS-1.82#implementation/unit_test/catch.hpp, CS-1.82#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp, 7.53#implementation/unit_test/catch.hpp
8.83#implementation/unit_test/catch.hpp	Store assertion result data	Defines `AssertionResultData` to hold the raw data for an assertion result, including the result type, the lazy expression, and the generated message. It can reconstruct the expression string from the lazy expression.	CS-1.83#implementation/unit_test/catch.hpp, CS-1.83#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
8.84#implementation/unit_test/catch.hpp	Stream transient expression	Overloads the stream insertion operator (<<) for `ITransientExpression` objects, allowing them to be directly printed to output streams by calling their `streamReconstructedExpression` method.	CS-1.84#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.85#implementation/unit_test/catch.hpp	Tokenize command line input	Implements `TokenType` enum and `Token` struct to represent parts of the command line. Defines the `TokenStream` class to abstract iterating over command line arguments, handling different option formats and splitting arguments into a stream of `Token` objects.	CS-1.85#implementation/unit_test/catch.hpp, CS-1.85#implementation/unit_test/catch.hpp, CS-1.85#implementation/unit_test/catch.hpp, CS-1.85#implementation/unit_test/catch.hpp, CS-1.85#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.86#implementation/unit_test/catch.hpp	Bind flags to boolean variable references	Implements concrete binding logic for associating boolean flags with references to boolean variables using the BoundFlagRef struct.	CS-1.86#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.87#implementation/unit_test/catch.hpp	Bind values and flags to lambdas	Implements concrete binding logic for associating parsed values (BoundLambda) or flags (BoundFlagLambda) with specific lambdas, allowing custom actions based on parsed input.	CS-1.87#implementation/unit_test/catch.hpp, CS-1.87#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.88#implementation/unit_test/catch.hpp	Bind values to variable references	Implements concrete binding logic for associating parsed values with references to variables of various types (including vectors) using the BoundRef template.	CS-1.88#implementation/unit_test/catch.hpp, CS-1.88#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.89#implementation/unit_test/catch.hpp	Build Catch2 command line parser	Creates and configures a command line parser specifically for the Catch2 test framework, defining various options and arguments and binding them to configuration variables or lambda functions that	CS-1.89#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		modify the test configuration.		
8.90#implementation/unit_test/catch.hpp	Compose and parse command line inputs	Defines the main Parser struct which composes executable name, options, and arguments, provides methods for adding parser components (using = operator), generating help text, validating the parser structure, and executing the parsing process against a stream of tokens.	CS-1.90#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.91#implementation/unit_test/catch.hpp	Convert strings to various types	Provides overloaded functions to convert string inputs into target variables of different types (generic, string, bool), returning a ParserResult indicating success or failure.	CS-1.91#implementation/unit_test/catch.hpp, CS-1.91#implementation/unit_test/catch.hpp, CS-1.91#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp, 7.54#implementation/unit_test/catch.hpp
8.92#implementation/unit_test/catch.hpp	Define help column structure	Defines a simple structure to hold text for the left and right columns when generating help output.	CS-1.92#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.93#implementation/unit_test/catch.hpp	Define parser base interfaces and composition	Provides base classes (ParserBase, ComposableParserImpl) for all parser types, defining the core interface (parse, validate, cardinality) and enabling parser composition using the bitwise OR operator (). ParserRefImpl serves as a common base for argument and option parsers. Defines the `operator ` for creating composed parsers.	CS-1.93#implementation/unit_test/catch.hpp, CS-1.93#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.94#implementation/unit_test/catch.hpp	Define result types	Defines type aliases for BasicResult used throughout the parsing library for representing operation outcomes.	CS-1.94#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.95#implementation/unit_test/catch.hpp	Handle executable name	Defines a parser component	CS-1.95#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
nit_test/catch.hpp	parsing	(ExeName) specifically for capturing and storing the executable name, optionally binding it to a variable or lambda.		nit_test/catch.hpp
8.96#implementation/unit_test/catch.hpp	Implement standard help option	Defines a standard 'Help' option struct that automatically configures common help flags (-?, -h, --help) and sets a flag to indicate help is requested, short-circuiting further parsing.	CS-1.96#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.97#implementation/unit_test/catch.hpp	Implement value and flag binding interfaces	Defines base classes (BoundRefBase, BoundValueRefBase, BoundFlagRefBase) for abstracting how parsed command line values or flags are associated with variables or lambdas, providing a common interface for setting values.	CS-1.97#implementation/unit_test/catch.hpp, CS-1.97#implementation/unit_test/catch.hpp, CS-1.97#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.98#implementation/unit_test/catch.hpp	Invoke lambdas with parsed arguments	Provides helper templates (LambdaInvoker) and a function (invokeLambda) to handle invoking lambdas with arguments converted from strings, correctly handling different lambda return types (void or ParserResult).	CS-1.98#implementation/unit_test/catch.hpp, CS-1.98#implementation/unit_test/catch.hpp, CS-1.98#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.99#implementation/unit_test/catch.hpp	Normalize option names	Helper function to standardize option names, particularly handling '/' prefixes on Windows platforms.	CS-1.99#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.100#implementation/unit_test/catch.hpp	Parse command line arguments	Defines a parser component (Arg) for processing non-option command line arguments, attempting to match and set values based on the registered binding.	CS-1.100#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.101#implementation/unit_test/catch.hpp	Parse command line options	Defines a parser component (Opt) for processing command line	CS-1.101#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		options, supporting multiple names for the same option, value options, and flags, and handling binding based on option type.		
8.102#implementation/unit_test/catch.hpp	Combine output from multiple reporters	Allows multiple reporter instances to receive the same test events simultaneously, effectively combining their output.	CS-1.102#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.103#implementation/unit_test/catch.hpp	Control console output colour	Implements platform-specific logic to change the colour of console output using either Windows API or ANSI escape codes, based on configuration.	CS-1.103#implementation/unit_test/catch.hpp, CS-1.103#implementation/unit_test/catch.hpp, CS-1.103#implementation/unit_test/catch.hpp, CS-1.103#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.104#implementation/unit_test/catch.hpp	Detect if a debugger is attached	Implements platform-specific checks to determine if the current process is being debugged.	CS-1.104#implementation/unit_test/catch.hpp, CS-1.104#implementation/unit_test/catch.hpp, CS-1.104#implementation/unit_test/catch.hpp, CS-1.104#implementation/unit_test/catch.hpp, CS-1.104#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp
8.105#implementation/unit_test/catch.hpp	Format assertion expressions for reporting	Formats the left-hand side, operator, and right-hand side of an assertion expression for display, wrapping long expressions onto multiple lines.	CS-1.105#implementation/unit_test/catch.hpp	7.57#implementation/unit_test/catch.hpp
8.106#implementation/unit_test/catch.hpp	Handle fatal signals and exceptions	Sets up platform-specific handlers (SEH on Windows, signals on POSIX) to catch fatal errors like segmentation faults or stack overflows, reporting them via the result capture mechanism before re-raising the signal/exception.	CS-1.106#implementation/unit_test/catch.hpp, CS-1.106#implementation/unit_test/catch.hpp, CS-1.106#implementation/unit_test/catch.hpp, CS-1.106#implementation/unit_test/catch.hpp, CS-1.106#implementation/unit_test/catch.hpp	7.48#implementation/unit_test/catch.hpp
8.107#implementation/unit_test/catch.hpp	Manage the global test execution context	Provides a central point to access the current test configuration, runner, and result capture, managing its lifetime.	CS-1.107#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
8.108#implementation/unit_test/catch.hpp	Register and translate active exceptions	Manages a registry of custom exception translators and provides a mechanism to iterate through them to find a translation for the currently active exception, falling back to standard exception handling.	CS-1.108#implementation/unit_test/catch.hpp, CS-1.108#implementation/unit_test/catch.hpp	7.48#implementation/unit_test/catch.hpp
8.109#implementation/unit_test/catch.hpp	Save and restore the errno value	Provides an RAII guard class that saves the current value of the global `errno` variable upon construction and restores it upon destruction.	CS-1.109#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp
8.110#implementation/unit_test/catch.hpp	Write messages to debug console	Provides a platform-dependent function to output text directly to the debugger's console or standard output.	CS-1.110#implementation/unit_test/catch.hpp, CS-1.110#implementation/unit_test/catch.hpp, CS-1.110#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.111#implementation/unit_test/catch.hpp	Define Data Structures for Reporting	Defines various structures and classes (ReporterConfig, TestRunInfo, GroupInfo, AssertionStats, SectionStats, TestCaseStats, TestGroupStats, TestRunStats) to hold configuration and statistics throughout a test run, including assertion results, section details, test case information, and overall run totals.	CS-1.111#implementation/unit_test/catch.hpp, CS-1.111#implementation/unit_test/catch.hpp, CS-1.111#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.112#implementation/unit_test/catch.hpp	Define Result Type Utilities	Provides utility functions and operators (`isOk`, `isJustInfo`, `operator `, `shouldContinueOnFailure`, `shouldSuppressFailure`) for working with the `ResultWas` and `ResultDisposition` enums to determine test outcome and behavior.	CS-1.112#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.113#implementation/unit_test/catch.hpp	Define Test Case Management Interfaces	Provides base interfaces (ITestInvoker, ITestCaseRegistry) for invoking individual test cases	CS-1.113#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		and managing the collection of registered test cases.		
8.114#implementation/unit_test/catch.hpp	Define Test Runner Interface	Provides the base interface IRunner for test execution classes.	CS-1.114#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.115#implementation/unit_test/catch.hpp	Implement Base Matcher Functionality	Provides a base class `MatcherUntypedBase` for implementing custom matchers, handling common functionality like caching the string description.	CS-1.115#implementation/unit_test/catch.hpp	7.50#implementation/unit_test/catch.hpp
8.116#implementation/unit_test/catch.hpp	Implement Floating Point Matchers	Provides matchers (`WithinAbsMatcher`, `WithinUlpMatcher`, `WithinULP`, `WithinAbs`) for comparing floating-point numbers (float and double) with a specified absolute tolerance or using the Unit in the Last Place (ULP) method, handling potential issues with floating-point precision.	CS-1.116#implementation/unit_test/catch.hpp, CS-1.116#implementation/unit_test/catch.hpp, CS-1.116#implementation/unit_test/catch.hpp	7.50#implementation/unit_test/catch.hpp
8.117#implementation/unit_test/catch.hpp	Implement Memory Leak Detection	Provides platform-specific (Windows CRTDBG) or default empty implementation for detecting memory leaks during testing.	CS-1.117#implementation/unit_test/catch.hpp, CS-1.117#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp
8.118#implementation/unit_test/catch.hpp	Implement Random Number Generation	Provides functionality to seed and use a pseudo-random number generator, used potentially for randomizing test case execution order.	CS-1.118#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp
8.119#implementation/unit_test/catch.hpp	Implement Stream Redirection	Provides a utility class `RedirectedStream` to temporarily redirect an output stream (like std::cout or std::cerr) to another stream, typically used for capturing test output.	CS-1.119#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
8.120#implementation/unit_test/catch.hpp	Implement String Matchers	Provides matchers (`Equals`, `Contains`, `StartsWith`, `EndsWith`, `Matches`) for performing various string comparisons, including options for case sensitivity and regular expressions.	CS-1.120#implementation/unit_test/catch.hpp	7.50#implementation/unit_test/catch.hpp, 7.54#implementation/unit_test/catch.hpp
8.121#implementation/unit_test/catch.hpp	List Available Reporters	Implements the `listReporters` function to output a list of all registered reporters and their descriptions.	CS-1.121#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.122#implementation/unit_test/catch.hpp	List Test Cases	Provides functions (`listTests`, `listTestsNamesOnly`) to output a list of available test cases, filtered by the configuration's test specification, including details like source location and description based on verbosity.	CS-1.122#implementation/unit_test/catch.hpp, CS-1.122#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.123#implementation/unit_test/catch.hpp	List Test Tags	Implements the `listTags` function to output a list of tags used by test cases matching the current configuration, including the count of tests using each tag.	CS-1.123#implementation/unit_test/catch.hpp, CS-1.123#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.124#implementation/unit_test/catch.hpp	Manage Global Registries	Implements a central hub (`RegistryHub`) containing registries for test cases, reporters, exception translators, tag aliases, and startup exceptions. Provides static accessors (`getRegistryHub`, `getMutableRegistryHub`) and a cleanup function.	CS-1.124#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp
8.125#implementation/unit_test/catch.hpp	Manage Test Messages	Handles test-related messages and information, including storing details about the message (macro name, source location, type) and managing scoped messages for	CS-1.125#implementation/unit_test/catch.hpp	7.57#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		capturing output.		
8.126#implementation/unit_test/catch.hpp	Manage Test Reporters	Implements interfaces (IStreamingReporter, IReporterFactory) and a registry (IReporterRegistry, ReporterRegistry) for creating and managing test reporters, allowing different output formats and listener reporters. Includes a utility function to combine multiple reporters.	CS-1.126#implementation/unit_test/catch.hpp, CS-1.126#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.127#implementation/unit_test/catch.hpp	Provide Unified Listing Functionality	Implements the `list` function which acts as a single entry point to perform various listing tasks (tests, tags, reporters) based on the configuration flags.	CS-1.127#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.128#implementation/unit_test/catch.hpp	Access command line parser	Provides a constant reference to the internal command-line parser object.	CS-1.128#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.129#implementation/unit_test/catch.hpp	Access configuration data	Provides a mutable reference to the raw configuration data object.	CS-1.129#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.130#implementation/unit_test/catch.hpp	Access configuration object	Provides a reference to the Config object, constructing it from the raw configuration data if it has not been created yet.	CS-1.130#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.131#implementation/unit_test/catch.hpp	Add registered listeners to reporters	Retrieves all registered listener reporters from the registry and adds them to the given reporter chain.	CS-1.131#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.132#implementation/unit_test/catch.hpp	Apply command line arguments	Parses the provided command-line arguments, updates the session's configuration data, and handles immediate actions like showing help or library identification. Returns an exit code.	CS-1.132#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
8.133#implementation/unit_test/catch.hpp	Apply filenames as tags to test cases	Modifies the test cases to add the base filename (derived from the SourceLineInfo) as a tag, prefixed with '#'.	CS-1.133#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.134#implementation/unit_test/catch.hpp	Check for aborting condition	Determines if the test run should be aborted based on the total number of failed assertions exceeding the configured limit.	CS-1.134#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.135#implementation/unit_test/catch.hpp	Check for missing assertions	Evaluates if a test case or section completed without executing any assertions. If configured to warn about missing assertions and no children trackers exist, it records a failed assertion.	CS-1.135#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp, 7.52#implementation/unit_test/catch.hpp
8.136#implementation/unit_test/catch.hpp	Check if last assertion passed	Returns a boolean indicating whether the previous assertion resulted in a pass.	CS-1.136#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.137#implementation/unit_test/catch.hpp	Create configured or default reporters	Creates the primary reporter(s) based on the reporter names specified in the configuration. If no names are specified, it creates the default reporter.	CS-1.137#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.138#implementation/unit_test/catch.hpp	Create reporter instance by name	Looks up and creates a reporter instance from the registry based on the provided reporter name and configuration.	CS-1.138#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.139#implementation/unit_test/catch.hpp	Define abstract stream interface	Declares the virtual destructor for the IStream abstract base class, defining the interface for custom output streams.	CS-1.139#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.140#implementation/unit_test/catch.hpp	Determine assertion reaction	Populates the AssertionReaction object based on the current configuration and the assertion's disposition, determining if a debug	CS-1.140#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp, 7.52#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		break or throwing an exception is required.		
8.141#implementation/unit_test/catch.hpp	Display help information	Prints usage instructions for the command-line interface and the library version to the standard output.	CS-1.141#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.142#implementation/unit_test/catch.hpp	Display library identification	Prints information identifying the library and its purpose in a structured key-value format.	CS-1.142#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.143#implementation/unit_test/catch.hpp	End a test section early	Handles the premature ending of a test section, typically due to an exception. Marks the section tracker as failed or closed and stores the section end information for later processing.	CS-1.143#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.56#implementation/unit_test/catch.hpp
8.144#implementation/unit_test/catch.hpp	End a test section normally	Handles the completion of a test section under normal execution (without exceptions). Calculates section-specific statistics, checks for missing assertions, closes and removes the section tracker, notifies the reporter, and clears scoped messages.	CS-1.144#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.56#implementation/unit_test/catch.hpp
8.145#implementation/unit_test/catch.hpp	Execute current test case implementation	Internal method responsible for the actual execution of the active test case body. Handles timer, stream redirection (if enabled), invokes the test case, catches exceptions, tears down sections, and reports section statistics.	CS-1.145#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.146#implementation/unit_test/catch.hpp	Execute filtered test cases	Sets up the reporter chain and RunContext, applies test specifications (filters), iterates through all registered test cases, runs those that match the filters,	CS-1.146#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		skips others, and accumulates the total results.		
8.147#implementation/unit_test/catch.hpp	Finalize test run context	Destroys the RunContext and notifies the reporter that the test run has ended, providing final statistics.	CS-1.147#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.148#implementation/unit_test/catch.hpp	Finalize test session	Destroys the Session object and performs global library cleanup.	CS-1.148#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp
8.149#implementation/unit_test/catch.hpp	Get current result capture instance	Provides access to the current result capture instance associated with the test context.	CS-1.149#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp
8.150#implementation/unit_test/catch.hpp	Get current test name	Retrieves the name of the currently active test case.	CS-1.150#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.151#implementation/unit_test/catch.hpp	Get last assertion result	Provides access to the result data of the most recently processed assertion.	CS-1.151#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.152#implementation/unit_test/catch.hpp	Get startup exceptions	Retrieves the list of exceptions that were registered during the library's startup.	CS-1.152#implementation/unit_test/catch.hpp	7.48#implementation/unit_test/catch.hpp
8.153#implementation/unit_test/catch.hpp	Handle expression assertion	Processes an assertion based on evaluating an expression. Determines if the result passes or fails (considering negation) and either increments passed count or reports the failure with expression details.	CS-1.153#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.154#implementation/unit_test/catch.hpp	Handle fatal error condition	Processes and reports a fatal error, notifying the reporter, faking an assertion result, cleaning up unfinished sections, and simulating the end of the current test case, group, and run.	CS-1.154#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp, 7.48#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
8.155#implementation/unit_test/catch.hpp	Handle incomplete assertion processing	Reports an assertion that could not be fully processed, typically due to specific configuration options like fast compile that disable exception translation.	CS-1.155#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.156#implementation/unit_test/catch.hpp	Handle message assertion	Processes an assertion based on a message (e.g., FAIL, SUCCEED). Sets the result type and message, then calls `assertionEnded` and populates the reaction if it's not ok.	CS-1.156#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.157#implementation/unit_test/catch.hpp	Handle non-expression assertion	A generic handler for assertions that do not involve evaluating an expression. Sets the result type and calls `assertionEnded`, populating the reaction if the result is not ok.	CS-1.157#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.158#implementation/unit_test/catch.hpp	Handle unexpected exception not thrown	Reports a failure when an assertion expecting an exception did not encounter one.	CS-1.158#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.159#implementation/unit_test/catch.hpp	Handle unexpected inflight exception	Reports an unexpected exception that occurred during test execution, providing the exception message and triggering the assertion ended process.	CS-1.159#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp, 7.48#implementation/unit_test/catch.hpp
8.160#implementation/unit_test/catch.hpp	Implement console output stream (cout)	Implements the IStream interface to provide output to std::cout, specifically storing the original streambuf to handle potential redirection.	CS-1.160#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.161#implementation/unit_test/catch.hpp	Implement debug output stream	Implements the IStream interface to provide output directed to the debug console using a custom stream buffer.	CS-1.161#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.162#implementation/unit_test/catch.hpp	Implement file output stream	Implements the IStream interface to provide output to a file specified by	CS-1.162#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		a filename.		
8.163#implementation/unit_test/catch.hpp	Implement stream buffer for custom writers	A template class that implements std::streambuf to buffer output and then write it using a provided function object.	CS-1.163#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.164#implementation/unit_test/catch.hpp	Increment passed assertion count	Explicitly increments the count of passed assertions and resets the last assertion info.	CS-1.164#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.165#implementation/unit_test/catch.hpp	Initialize test run context	Constructs the RunContext, setting up the environment for a test run, configuring the context with runner, config, and result capture, and notifying the reporter that the test run is starting.	CS-1.165#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp, 7.52#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.166#implementation/unit_test/catch.hpp	Initialize test session	Constructs the Session object, performing startup checks, handling any exceptions that occurred during library initialization, and setting up the command-line parser.	CS-1.166#implementation/unit_test/catch.hpp	7.48#implementation/unit_test/catch.hpp, 7.51#implementation/unit_test/catch.hpp, 7.52#implementation/unit_test/catch.hpp
8.167#implementation/unit_test/catch.hpp	Internal test execution logic	The core internal method for executing tests. It handles startup exceptions, checks for help/identify, creates config, seeds RNG, applies filename tags, handles list requests, sets up reporters and listeners, creates RunContext, filters and runs tests, and returns the final exit code.	CS-1.167#implementation/unit_test/catch.hpp	7.48#implementation/unit_test/catch.hpp, 7.51#implementation/unit_test/catch.hpp, 7.52#implementation/unit_test/catch.hpp
8.168#implementation/unit_test/catch.hpp	Invoke active test case code	Executes the code associated with the currently active test case, guarded by a fatal condition handler.	CS-1.168#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.169#implementation/	Manage test section lifecycle	Represents a test section. Its	CS-1.169#implementation/unit_test/catch.hpp,	7.52#implementation/u

ID	Node Name	Description	Children IDs	Parent IDs
unit_test/catch.hpp		constructor determines if the section is included based on filters and starts a timer. Its destructor handles section ending (normal or early due to exception) and reports statistics.	CS-1.169#implementation/unit_test/catch.hpp, CS-1.169#implementation/unit_test/catch.hpp	nit_test/catch.hpp, 7.56#implementation/unit_test/catch.hpp
8.170#implementation/unit_test/catch.hpp	Pop scoped message	Removes a specific message from the list of messages associated with the current scope.	CS-1.170#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.171#implementation/unit_test/catch.hpp	Push scoped message	Adds a message to the list of messages associated with the current scope (e.g., for INFO or WARN).	CS-1.171#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.172#implementation/unit_test/catch.hpp	Redirect standard error streams	This class redirects the standard error streams (std::cerr and std::clog) to a single internal string stream for capturing, maintaining write order.	CS-1.172#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.173#implementation/unit_test/catch.hpp	Redirect standard output	This class redirects the standard output stream (std::cout) to an internal string stream for capturing.	CS-1.173#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.174#implementation/unit_test/catch.hpp	Register startup exception	Adds an exception that occurred during the library's startup process to a registry for later reporting.	CS-1.174#implementation/unit_test/catch.hpp	7.48#implementation/unit_test/catch.hpp
8.175#implementation/unit_test/catch.hpp	Report assertion end result	Processes the result of an assertion after it has been evaluated. Updates the internal total counts (passed, failed, failed but ok) and notifies the reporter about the assertion end with relevant statistics and messages.	CS-1.175#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.176#implementation/unit_test/catch.hpp	Report expression assertion details	Formats and reports the result of an expression-based assertion, including the expression details and	CS-1.176#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		whether it was negated, before calling `assertionEnded`.		
8.177#implementation/unit_test/catch.hpp	Reset last assertion info	Clears the information stored about the last assertion that was processed.	CS-1.177#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.178#implementation/unit_test/catch.hpp	Run a single test case	Manages the execution flow for a single test case, including starting/ending notifications, tracking, potentially redirecting stdout/stderr, executing the test body (possibly multiple times if sections are involved), handling failures/exceptions, updating totals, and reporting case statistics.	CS-1.178#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.179#implementation/unit_test/catch.hpp	Run tests	Starts the test execution flow. Handles waiting for a keypress before and after the test run based on configuration, and then calls the internal run method.	CS-1.179#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.180#implementation/unit_test/catch.hpp	Run tests with command line arguments	Initiates a test run. First applies the provided command-line arguments, then calls the main run method if command line processing was successful. Includes support for wide character arguments on Windows.	CS-1.180#implementation/unit_test/catch.hpp, CS-1.180#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.53#implementation/unit_test/catch.hpp
8.181#implementation/unit_test/catch.hpp	Set command line parser	Replaces the internal command-line parser object with a new one.	CS-1.181#implementation/unit_test/catch.hpp	7.53#implementation/unit_test/catch.hpp
8.182#implementation/unit_test/catch.hpp	Signal benchmark end	Notifies the registered reporter that a benchmark section has ended, providing benchmark statistics.	CS-1.182#implementation/unit_test/catch.hpp	7.49#implementation/unit_test/catch.hpp
8.183#implementation/unit_test/catch.hpp	Signal benchmark start	Notifies the registered reporter that a benchmark section is starting, providing benchmark information.	CS-1.183#implementation/unit_test/catch.hpp	7.49#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
8.184#implementation/unit_test/catch.hpp	Signal exception early report	Sets a flag indicating that an unexpected exception has already been reported early in the process, preventing duplicate reporting.	CS-1.184#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.185#implementation/unit_test/catch.hpp	Signal test group end	Notifies the registered reporter that a test group has ended, providing statistics, test specification, group index, count, and aborting status.	CS-1.185#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.186#implementation/unit_test/catch.hpp	Signal test group start	Notifies the registered reporter that a test group is starting, providing details about the group specification, index, and count.	CS-1.186#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.187#implementation/unit_test/catch.hpp	Start a test section	Handles the initiation of a test section. Acquires or creates a tracker for the section, determines if the section should be executed based on filters, updates the context's active section list and last assertion info, and notifies the reporter.	CS-1.187#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.56#implementation/unit_test/catch.hpp
8.188#implementation/unit_test/catch.hpp	Store section information	Data structures defining the information about a test section, including its name, description, source line information, and end details such as assertions and duration.	CS-1.188#implementation/unit_test/catch.hpp, CS-1.188#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.189#implementation/unit_test/catch.hpp	Tear down unfinished sections	Processes and finalizes any test sections that ended prematurely, typically due to an exception, ensuring they are reported correctly.	CS-1.189#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.56#implementation/unit_test/catch.hpp
8.190#implementation/unit_test/catch.hpp	Use external configuration data	Sets the session's configuration data using an externally provided ConfigData object and resets the internal Config object to be rebuilt later.	CS-1.190#implementation/unit_test/catch.hpp	6.128#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
8.191#implementation/unit_test/catch.hpp	Write to debug console	A writer function object that directs output strings to a platform-specific debug console.	CS-1.191#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.192#implementation/unit_test/catch.hpp	Automatically Register Tag Aliases	Uses a class with a constructor to automatically register tag aliases in the global registry during program startup, handling potential exceptions.	CS-1.192#implementation/unit_test/catch.hpp	7.48#implementation/unit_test/catch.hpp, 7.52#implementation/unit_test/catch.hpp
8.193#implementation/unit_test/catch.hpp	Automatically Register Test Cases	Uses a class with a constructor to automatically create and register test cases in the global registry during program startup, handling potential exceptions.	CS-1.193#implementation/unit_test/catch.hpp	7.48#implementation/unit_test/catch.hpp, 7.52#implementation/unit_test/catch.hpp
8.194#implementation/unit_test/catch.hpp	Check String Containment	Checks if a string contains a specific infix string.	CS-1.194#implementation/unit_test/catch.hpp	7.54#implementation/unit_test/catch.hpp
8.195#implementation/unit_test/catch.hpp	Check String Prefixes	Checks if a string starts with a specific prefix string or character.	CS-1.195#implementation/unit_test/catch.hpp, CS-1.195#implementation/unit_test/catch.hpp	7.54#implementation/unit_test/catch.hpp
8.196#implementation/unit_test/catch.hpp	Check String Suffixes	Checks if a string ends with a specific suffix string or character.	CS-1.196#implementation/unit_test/catch.hpp, CS-1.196#implementation/unit_test/catch.hpp	7.54#implementation/unit_test/catch.hpp
8.197#implementation/unit_test/catch.hpp	Convert Characters to Lowercase	Converts a single character or an entire string to its lowercase representation.	CS-1.197#implementation/unit_test/catch.hpp, CS-1.197#implementation/unit_test/catch.hpp, CS-1.197#implementation/unit_test/catch.hpp	7.54#implementation/unit_test/catch.hpp
8.198#implementation/unit_test/catch.hpp	Create Output Streams	This function creates different types of output streams (standard output, debug output, or file streams) based on the provided filename string.	CS-1.198#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.199#implementation/unit_test/catch.hpp	Create Test Case Info and Object	Parses description and tags from a string to create a TestCaseInfo struct, and then uses it to construct a TestCase object which includes the invoker function.	CS-1.199#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.200#implementation/unit_test/catch.hpp	Define Test Specification	A struct representing a collection of	CS-1.200#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
unit_test/catch.hpp		filters and patterns used to select which test cases should be run.		nit_test/catch.hpp
8.201#implementation/unit_test/catch.hpp	Enforce Unique Test Cases	Checks a list of test cases for duplicates based on name and class name, throwing an error if duplicates are found.	CS-1.201#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.202#implementation/unit_test/catch.hpp	Extract Class Name from String	Parses a string representing a potential qualified method name (like &ClassName::MethodName) to extract just the class name.	CS-1.202#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.54#implementation/unit_test/catch.hpp
8.203#implementation/unit_test/catch.hpp	Filter Test Cases	Filters a list of test cases, returning only those that match the provided test specification and configuration.	CS-1.203#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.204#implementation/unit_test/catch.hpp	Handle Pluralization of Labels	Provides a utility to output a count followed by a label, automatically adding an 's' to the label if the count is not 1.	CS-1.204#implementation/unit_test/catch.hpp, CS-1.204#implementation/unit_test/catch.hpp	7.54#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.205#implementation/unit_test/catch.hpp	Manage Reusable String Streams	Implements a pool of std::ostringstream objects that can be reused to avoid repeated allocations, improving performance.	CS-1.205#implementation/unit_test/catch.hpp, CS-1.205#implementation/unit_test/catch.hpp	7.54#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.206#implementation/unit_test/catch.hpp	Manage Tag Alias Registry	Provides a central registry for managing tag aliases, allowing lookup, expansion of aliases in test specifications, and adding new aliases with validation.	CS-1.206#implementation/unit_test/catch.hpp, CS-1.206#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.207#implementation/unit_test/catch.hpp	Manage Test Case Registry	Stores and manages all registered test cases, handling naming of anonymous test cases and providing access to the test cases, potentially sorted.	CS-1.207#implementation/unit_test/catch.hpp, CS-1.207#implementation/unit_test/catch.hpp, CS-1.207#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.208#implementation/unit_test/catch.hpp	Manage Test Execution Tracking Context	Acts as the central manager for tracking test case execution flow	CS-1.208#implementation/unit_test/catch.hpp, CS-1.208#implementation/unit_test/catch.hpp,	7.52#implementation/unit_test/catch.hpp, 7.56

ID	Node Name	Description	Children IDs	Parent IDs
		(sections, generators), maintaining the tracker tree and the current execution state.	CS-1.208#implementation/unit_test/catch.hpp, CS-1.208#implementation/unit_test/catch.hpp, CS-1.208#implementation/unit_test/catch.hpp, CS-1.208#implementation/unit_test/catch.hpp, CS-1.208#implementation/unit_test/catch.hpp, CS-1.208#implementation/unit_test/catch.hpp	#implementation/unit_test/catch.hpp
8.209#implementation/unit_test/catch.hpp	Match Test Case to Specification	Checks if a given test case matches a specified test specification and respects configuration options like allowing exceptions.	CS-1.209#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.210#implementation/unit_test/catch.hpp	Parse Special Tags	Identifies and categorizes specific tags (e.g., hidden, throws, shouldfail) that indicate special properties for a test case.	CS-1.210#implementation/unit_test/catch.hpp, CS-1.210#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.211#implementation/unit_test/catch.hpp	Parse Test Specification String	Parses a command-line string representing the desired test cases to run into an internal TestSpec object, handling tag aliases, inclusions, and exclusions.	CS-1.211#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.212#implementation/unit_test/catch.hpp	Provide Access to Standard Streams	Offers functions to access standard C++ output streams (cout, cerr, clog).	CS-1.212#implementation/unit_test/catch.hpp	7.57#implementation/unit_test/catch.hpp
8.213#implementation/unit_test/catch.hpp	Provide Base Tracking Functionality	A base class for different tracker types, handling common properties like name, location, parent, children, and state (e.g., executing, completed, failed), as well as methods for managing children and navigation.	CS-1.213#implementation/unit_test/catch.hpp	7.56#implementation/unit_test/catch.hpp
8.214#implementation/unit_test/catch.hpp	Replace Substrings In Place	Replaces all occurrences of a specific substring with another substring within a string, modifying the string directly.	CS-1.214#implementation/unit_test/catch.hpp	7.54#implementation/unit_test/catch.hpp
8.215#implementation/unit_test/catch.hpp	Represent a Test Case	Combines the static information	CS-1.215#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
unit_test/catch.hpp	Executable	(TestCaseInfo) with the actual executable test code (ITestInvoker), allowing the test to be invoked and providing access to its metadata.		nit_test/catch.hpp
8.216#implementation/unit_test/catch.hpp	Represent Name and Location	A simple struct used to hold the name and source code location for tracking points like sections or generators.	CS-1.216#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.56#implementation/unit_test/catch.hpp
8.217#implementation/unit_test/catch.hpp	Represent String References	Provides a lightweight, non-owning view into existing string data (char const*), offering methods for manipulation and comparison. Can optionally take ownership of the data.	CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp, CS-1.217#implementation/unit_test/catch.hpp	7.54#implementation/unit_test/catch.hpp
8.218#implementation/unit_test/catch.hpp	Represent Tag Aliases	Defines a structure to hold the mapping between a tag alias and its actual tag, along with the source line information where it was defined.	CS-1.218#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.219#implementation/unit_test/catch.hpp	Represent Test Case Metadata	A struct holding all static information about a test case, such as its name, class name, description, tags, source location, and special properties derived from tags.	CS-1.219#implementation/unit_test/catch.hpp, CS-1.219#implementation/unit_test/catch.hpp, CS-1.219#implementation/unit_test/catch.hpp, CS-1.219#implementation/unit_test/catch.hpp, CS-1.219#implementation/unit_test/catch.hpp, CS-1.219#implementation/unit_test/catch.hpp, CS-1.219#implementation/unit_test/catch.hpp, CS-1.219#implementation/unit_test/catch.hpp, CS-1.219#implementation/unit_test/catch.hpp, CS-1.219#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
8.220#implementation/unit_test/catch.hpp	Sort Test Cases	Sorts a vector of test cases based on the configured run order (lexicographical, random, or declaration order).	CS-1.220#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.221#implementation/unit_test/catch.hpp	Standardize and Set Test Case Tags	Takes a vector of tags, sorts and removes duplicates, converts them to lowercase for internal storage, parses special properties, and assigns them to a TestCaseInfo object.	CS-1.221#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp
8.222#implementation/unit_test/catch.hpp	Track Index-Based Loops	A specific type of tracker used for managing the execution state of index-based loops (like those implied by generators), ensuring each iteration is run in subsequent test cycles.	CS-1.222#implementation/unit_test/catch.hpp, CS-1.222#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.56#implementation/unit_test/catch.hpp
8.223#implementation/unit_test/catch.hpp	Track Test Case Sections	A specific type of tracker used for managing the execution state of `SECTION` blocks, including handling filters to conditionally execute sections.	CS-1.223#implementation/unit_test/catch.hpp, CS-1.223#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp, 7.56#implementation/unit_test/catch.hpp
8.224#implementation/unit_test/catch.hpp	Trim Whitespace from String	Removes leading and trailing whitespace characters from a string.	CS-1.224#implementation/unit_test/catch.hpp	7.54#implementation/unit_test/catch.hpp
8.225#implementation/unit_test/catch.hpp	Wrap Function as Test Invoker	Provides an implementation of ITestInvoker that simply calls a plain function pointer, allowing free functions to be registered as tests.	CS-1.225#implementation/unit_test/catch.hpp	6.129#implementation/unit_test/catch.hpp
8.226#implementation/unit_test/catch.hpp	Wrap Pooled String Streams	Provides a wrapper around the pooled ostringstream, acquiring one from the pool upon construction and releasing it upon destruction, automatically clearing its content.	CS-1.226#implementation/unit_test/catch.hpp, CS-1.226#implementation/unit_test/catch.hpp, CS-1.226#implementation/unit_test/catch.hpp, CS-1.226#implementation/unit_test/catch.hpp	7.54#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.227#implementation/unit_test/catch.hpp	Aggregate test and assertion	Implements a struct `Totals` to	CS-1.227#implementation/unit_test/catch.hpp,	7.47#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		characters.		
8.233#implementation/unit_test/catch.hpp	Estimate clock resolution	Estimates the minimum time difference the system clock can measure by repeatedly querying the time until it changes and averaging the results.	CS-1.233#implementation/unit_test/catch.hpp, CS-1.233#implementation/unit_test/catch.hpp	7.49#implementation/unit_test/catch.hpp
8.234#implementation/unit_test/catch.hpp	Format and print assertion results	Provides a class to format and print detailed information for individual assertion outcomes, including source information, result type, original and reconstructed expressions, and associated messages.	CS-1.234#implementation/unit_test/catch.hpp, CS-1.234#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.235#implementation/unit_test/catch.hpp	Format duration string	Formats a duration value in seconds into a string with three decimal places using C-style formatting.	CS-1.235#implementation/unit_test/catch.hpp	7.54#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.236#implementation/unit_test/catch.hpp	Format floating point numbers	Formats floating-point values (float and double) into strings with a specified precision, handling NaN and removing trailing zeros.	CS-1.236#implementation/unit_test/catch.hpp, CS-1.236#implementation/unit_test/catch.hpp, CS-1.236#implementation/unit_test/catch.hpp	7.54#implementation/unit_test/catch.hpp
8.237#implementation/unit_test/catch.hpp	Get current time in nanoseconds	Retrieves the current time since the epoch with nanosecond precision.	CS-1.237#implementation/unit_test/catch.hpp	7.49#implementation/unit_test/catch.hpp
8.238#implementation/unit_test/catch.hpp	Manage test and assertion counts	Implements a struct `Counts` to track passed, failed, and failed but ok counts, providing operators for arithmetic and methods for total count and pass/fail status.	CS-1.238#implementation/unit_test/catch.hpp, CS-1.238#implementation/unit_test/catch.hpp, CS-1.238#implementation/unit_test/catch.hpp, CS-1.238#implementation/unit_test/catch.hpp, CS-1.238#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.239#implementation/unit_test/catch.hpp	Match strings using wildcards	Implements a wildcard pattern matching mechanism for strings, supporting '*' at the beginning, end, or both, with an option for case-sensitive or insensitive matching.	CS-1.239#implementation/unit_test/catch.hpp, CS-1.239#implementation/unit_test/catch.hpp, CS-1.239#implementation/unit_test/catch.hpp	7.54#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
			CS-1.245#implementation/unit_test/catch.hpp, CS-1.245#implementation/unit_test/catch.hpp, CS-1.245#implementation/unit_test/catch.hpp, CS-1.245#implementation/unit_test/catch.hpp, CS-1.245#implementation/unit_test/catch.hpp, CS-1.245#implementation/unit_test/catch.hpp	
8.246#implementation/unit_test/catch.hpp	Configure Compact Reporter preferences	Sets specific preferences for the reporter, such as disabling stdout redirection.	CS-1.246#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.247#implementation/unit_test/catch.hpp	Describe Compact Reporter purpose	Provides a brief description of the CompactReporter's output style.	CS-1.247#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.248#implementation/unit_test/catch.hpp	Format console assertion output	A helper class responsible for formatting and printing detailed information about assertion results to the console, including source location, expressions, messages, and using color.	CS-1.248#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.249#implementation/unit_test/catch.hpp	Format time durations	Provides a class and operator overload to display time durations in human-readable units (ns, μ s, ms, s, m), automatically choosing the most appropriate unit.	CS-1.249#implementation/unit_test/catch.hpp	7.54#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.250#implementation/unit_test/catch.hpp	Print data in a formatted table	A utility class for printing data to an output stream in a structured table format with columns, headers, and customizable justification and width.	CS-1.250#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.251#implementation/unit_test/catch.hpp	Print test run summary totals	Outputs the aggregate statistics for the entire test run, including total tests, assertions, etc.	CS-1.251#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.252#implementation/unit_test/catch.hpp	Process and report assertion results	Handles the conclusion of an assertion, filtering successful results based on configuration and using an AssertionPrinter to format the output.	CS-1.252#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
8.253#implementation/unit_test/catch.hpp	Register Compact Reporter	Registers the `CompactReporter` class with the name "compact" so it can be selected by the test runner.	CS-1.253#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.254#implementation/unit_test/catch.hpp	Report no matching test cases	Informs the user when the specified test case filter does not match any tests.	CS-1.254#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.255#implementation/unit_test/catch.hpp	Report test section durations	Prints the execution time for each test section if duration reporting is enabled in the configuration.	CS-1.255#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.256#implementation/unit_test/catch.hpp	Add reporter to multiple reporters	Adds a reporter instance to the list of reporters to which events will be forwarded.	CS-1.256#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.257#implementation/unit_test/catch.hpp	Construct a Console Reporter	Initializes the ConsoleReporter object, setting up the base class with the configuration and creating a TablePrinter instance for formatting benchmark output.	CS-1.257#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.258#implementation/unit_test/catch.hpp	Construct a JUnit Reporter	Initializes the JunitReporter object, setting up the base class and XML writer.	CS-1.258#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.259#implementation/unit_test/catch.hpp	Construct an XML Reporter	Initializes the XmlReporter object, setting up the base class and XML writer.	CS-1.259#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.260#implementation/unit_test/catch.hpp	Destroy Console Reporter	Default destructor for the ConsoleReporter.	CS-1.260#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.261#implementation/unit_test/catch.hpp	Destroy JUnit Reporter	Default destructor for the JunitReporter.	CS-1.261#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.262#implementation/unit_test/catch.hpp	Destroy XML Reporter	Default destructor for the XmlReporter.	CS-1.262#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.263#implementation/unit_test/catch.hpp	Extract file name tag	Finds and extracts a tag starting with '#' from a list of tags, typically	CS-1.263#implementation/unit_test/catch.hpp	7.52#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		used for deriving class names.		
8.264#implementation/unit_test/catch.hpp	Format and print header string	Prints a header string, handling potential colons for indentation on subsequent lines.	CS-1.264#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.265#implementation/unit_test/catch.hpp	Forward assertion ended event	Forwards the 'assertion ended' event to all contained reporters and returns true if any reporter requests the messages buffer to be cleared.	CS-1.265#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.266#implementation/unit_test/catch.hpp	Forward assertion starting event	Forwards the 'assertion starting' event to all contained reporters.	CS-1.266#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.267#implementation/unit_test/catch.hpp	Forward benchmark ended event	Forwards the 'benchmark ended' event to all contained reporters.	CS-1.267#implementation/unit_test/catch.hpp	7.49#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.268#implementation/unit_test/catch.hpp	Forward benchmark starting event	Forwards the 'benchmark starting' event to all contained reporters.	CS-1.268#implementation/unit_test/catch.hpp	7.49#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.269#implementation/unit_test/catch.hpp	Forward no matching test cases event	Forwards the 'no matching test cases' event to all contained reporters.	CS-1.269#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.270#implementation/unit_test/catch.hpp	Forward section ended event	Forwards the 'section ended' event to all contained reporters.	CS-1.270#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.271#implementation/unit_test/catch.hpp	Forward section starting event	Forwards the 'section starting' event to all contained reporters.	CS-1.271#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.272#implementation/unit_test/catch.hpp	Forward skip test event	Forwards the 'skip test' event to all contained reporters.	CS-1.272#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.273#implementation/unit_test/catch.hpp	Forward test case ended event	Forwards the 'test case ended' event to all contained reporters.	CS-1.273#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.274#implementation/unit_test/catch.hpp	Forward test case starting event	Forwards the 'test case starting' event to all contained reporters.	CS-1.274#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
unit_test/catch.hpp	event	event to all contained reporters.		nit_test/catch.hpp
8.275#implementation/unit_test/catch.hpp	Forward test group ended event	Forwards the 'test group ended' event to all contained reporters.	CS-1.275#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.276#implementation/unit_test/catch.hpp	Forward test group starting event	Forwards the 'test group starting' event to all contained reporters.	CS-1.276#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.277#implementation/unit_test/catch.hpp	Forward test run ended event	Forwards the 'test run ended' event to all contained reporters.	CS-1.277#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.278#implementation/unit_test/catch.hpp	Forward test run starting event	Forwards the 'test run starting' event to all contained reporters.	CS-1.278#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.279#implementation/unit_test/catch.hpp	Get current timestamp (UTC)	Retrieves the current time and formats it into a UTC timestamp string.	CS-1.279#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.280#implementation/unit_test/catch.hpp	Get multiple reporters preferences	Returns the reporter preferences from the first reporter in the list.	CS-1.280#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.281#implementation/unit_test/catch.hpp	Get stylesheet reference	Returns an empty string, allowing derived classes to specify an XML stylesheet reference.	CS-1.281#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.282#implementation/unit_test/catch.hpp	Get supported verbosity (multiple reporters)	Returns an empty set of supported verbosity for the multiple reporters.	CS-1.282#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.283#implementation/unit_test/catch.hpp	Handle assertion results (JUnit)	Tracks unexpected exceptions and calls the base class implementation when an assertion ends.	CS-1.283#implementation/unit_test/catch.hpp	7.47#implementation/unit_test/catch.hpp
8.284#implementation/unit_test/catch.hpp	Handle assertion start (Console)	Placeholder function for the start of an assertion, performs no specific action in this reporter.	CS-1.284#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.285#implementation/unit_test/catch.hpp	Handle assertion starting (XML)	Placeholder function for the start of an assertion, performs no specific action in this reporter.	CS-1.285#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.286#implementation/unit_test/catch.hpp	Handle memory leaks	Provides a global LeakDetector	CS-1.286#implementation/unit_test/catch.hpp	7.51#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
unit_test/catch.hpp		instance, likely used for tracking memory allocations.		nit_test/catch.hpp
8.287#implementation/unit_test/catch.hpp	Handle no matching test cases (JUnit)	Placeholder function for no matching test cases, performs no specific action in this reporter.	CS-1.287#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.288#implementation/unit_test/catch.hpp	Handle no matching test cases (XML)	Calls the base class implementation when no test cases match.	CS-1.288#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.289#implementation/unit_test/catch.hpp	Handle section starting	Resets the header printed flag and calls the base class implementation when a new section starts.	CS-1.289#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.290#implementation/unit_test/catch.hpp	Handle test case ended (JUnit)	Appends the test case's stdout and stderr to the suite buffers and calls the base class implementation.	CS-1.290#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.291#implementation/unit_test/catch.hpp	Handle test case starting (JUnit)	Stores the 'okToFail' status for the current test case.	CS-1.291#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.292#implementation/unit_test/catch.hpp	Identify as multi reporter	Returns true, indicating that this reporter aggregates multiple other reporters.	CS-1.292#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.293#implementation/unit_test/catch.hpp	Lazily print headers	Ensures that run, group, and test case/section headers are printed before subsequent output, closing the benchmark table if necessary.	CS-1.293#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.294#implementation/unit_test/catch.hpp	Lazily print headers (without closing benchmark table)	Ensures that run, group, and test case/section headers are printed before subsequent output.	CS-1.294#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.295#implementation/unit_test/catch.hpp	Print closed header	Prints a header string followed by a closing line separator.	CS-1.295#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.296#implementation/unit_test/catch.hpp	Print open header	Prints an opening line separator followed by a header string.	CS-1.296#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.297#implementation/unit_test/catch.hpp	Print summary divider	Prints a visual separator line above	CS-1.297#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
unit_test/catch.hpp		the test group summary totals.		nit_test/catch.hpp
8.298#implementation/unit_test/catch.hpp	Print summary row	Formats and prints a single row of the summary table (either for test cases or assertions).	CS-1.298#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.299#implementation/unit_test/catch.hpp	Print test case and section header	Prints the header for the current test case and any nested sections, including source file and line number if available.	CS-1.299#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.300#implementation/unit_test/catch.hpp	Print test group information	Prints a header for the test group if it has a name and there is more than one group.	CS-1.300#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.301#implementation/unit_test/catch.hpp	Print test run information	Prints introductory information about the test run, including the Catch version and seed if used.	CS-1.301#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.302#implementation/unit_test/catch.hpp	Print test run/group totals	Prints a summary of test cases and assertions, including counts for passed, failed, and failed as expected.	CS-1.302#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.303#implementation/unit_test/catch.hpp	Print totals divider	Prints a visual separator line below the test run totals, colored based on the success/failure ratio.	CS-1.303#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.304#implementation/unit_test/catch.hpp	Provide JUnit reporter description	Returns a string describing the JUnitReporter's output format.	CS-1.304#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.305#implementation/unit_test/catch.hpp	Provide Objective-C main entry point	Implements the main function for Objective-C, handling autorelease pool and registering test methods before running the test session.	CS-1.305#implementation/unit_test/catch.hpp	7.48#implementation/unit_test/catch.hpp
8.306#implementation/unit_test/catch.hpp	Provide reporter description	Returns a string describing the functionality of the ConsoleReporter.	CS-1.306#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.307#implementation/unit_test/catch.hpp	Provide standard main entry	Implements the standard C/C++	CS-1.307#implementation/unit_test/catch.hpp	7.48#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
unit_test/catch.hpp	point	main function (or wmain on Windows) that initializes and runs the test session with command-line arguments.		nit_test/catch.hpp
8.308#implementation/unit_test/catch.hpp	Provide XML reporter description	Returns a string describing the XMLReporter's output format.	CS-1.308#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.309#implementation/unit_test/catch.hpp	Register console reporter	Registers the ConsoleReporter implementation with the testing framework under the name 'console'.	CS-1.309#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.310#implementation/unit_test/catch.hpp	Register JUnit reporter	Registers the JunitReporter implementation with the testing framework under the name 'junit'.	CS-1.310#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.311#implementation/unit_test/catch.hpp	Register XML reporter	Registers the XmlReporter implementation with the testing framework under the name 'xml'.	CS-1.311#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.312#implementation/unit_test/catch.hpp	Report assertion results	Processes and prints the result of an assertion to the console, including messages for failures, warnings, or successful results depending on the configuration.	CS-1.312#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.313#implementation/unit_test/catch.hpp	Report assertion results (XML)	Writes failed or included assertion results to XML, including expression details, source info, and messages.	CS-1.313#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.314#implementation/unit_test/catch.hpp	Report benchmark results	Calculates and prints the iterations, elapsed time, and average time for a benchmark using the table printer.	CS-1.314#implementation/unit_test/catch.hpp	7.49#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp
8.315#implementation/unit_test/catch.hpp	Report benchmark starting	Prints necessary headers if not already printed and formats the benchmark name for the table when a benchmark starts.	CS-1.315#implementation/unit_test/catch.hpp	7.49#implementation/unit_test/catch.hpp, 7.55#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
8.316#implementation/unit_test/catch.hpp	Report no matching test cases	Prints a message to the stream when no test cases match the provided specification.	CS-1.316#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.317#implementation/unit_test/catch.hpp	Report section results	Closes the benchmark table, reports missing assertions if any, prints section duration if configured, and calls the base class implementation.	CS-1.317#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.318#implementation/unit_test/catch.hpp	Report section results (XML)	Calls the base class, writes 'OverallResults' for the section (if nested), and decreases the section depth.	CS-1.318#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.319#implementation/unit_test/catch.hpp	Report section starting (XML)	Calls the base class and starts a nested 'Section' element with name, description, and source info if not the root section.	CS-1.319#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.320#implementation/unit_test/catch.hpp	Report test case ended	Closes the benchmark table, calls the base class implementation, and resets the header printed flag when a test case finishes.	CS-1.320#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.321#implementation/unit_test/catch.hpp	Report test case ended (XML)	Calls the base class, writes 'OverallResult' for the test case with success status, optional duration, and captured stdout/stderr.	CS-1.321#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.322#implementation/unit_test/catch.hpp	Report test case starting (XML)	Calls the base class, starts a 'TestCase' element with name, description, tags, and source info, and starts a timer if durations are shown.	CS-1.322#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.323#implementation/unit_test/catch.hpp	Report test group ended (XML)	Calls the base class and writes 'OverallResults' for the test group.	CS-1.323#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.324#implementation/unit_test/catch.hpp	Report test group results	Prints a summary divider and the totals for the test group if the group was used, then calls the base class	CS-1.324#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
		implementation.		
8.325#implementation/unit_test/catch.hpp	Report test group results (JUnit)	Stops timing for the test group, calls the base class, and writes the group's information to the XML as a 'testsuite'.	CS-1.325#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.326#implementation/unit_test/catch.hpp	Report test group starting (JUnit)	Starts timing for the test group, clears stdout/stderr buffers, and calls the base class implementation.	CS-1.326#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.327#implementation/unit_test/catch.hpp	Report test group starting (XML)	Calls the base class implementation and starts a 'Group' element in the XML with the group name.	CS-1.327#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.328#implementation/unit_test/catch.hpp	Report test run ended (Cumulative - JUnit)	Closes the root 'testsuites' element in the XML output.	CS-1.328#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.329#implementation/unit_test/catch.hpp	Report test run ended (XML)	Calls the base class and writes 'OverallResults' for the entire test run.	CS-1.329#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.330#implementation/unit_test/catch.hpp	Report test run results	Prints a divider and the summary totals for the entire test run, then calls the base class implementation.	CS-1.330#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.331#implementation/unit_test/catch.hpp	Report test run starting (JUnit)	Starts the root 'testsuites' element in the XML output.	CS-1.331#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.332#implementation/unit_test/catch.hpp	Report test run starting (XML)	Calls the base class implementation and starts the root 'Catch' element in the XML, including optional stylesheet reference and name.	CS-1.332#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.333#implementation/unit_test/catch.hpp	Write assertion data to XML (JUnit)	Iterates through assertions within a section and writes them to XML.	CS-1.333#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.334#implementation/unit_test/catch.hpp	Write assertion result to XML (JUnit)	Writes a failed assertion to XML as an 'error' or 'failure' element, including message, type, and source info.	CS-1.334#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp

ID	Node Name	Description	Children IDs	Parent IDs
8.335#implementation/unit_test/catch.hpp	Write group data to XML	Writes a 'testsuite' element to the XML, including name, error/failure counts, total tests, hostname (TBD), time, timestamp, test cases, and system output/error.	CS-1.335#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.336#implementation/unit_test/catch.hpp	Write section data to XML	Recursively writes section information to XML, flattening nested sections into 'testcase' elements in the JUnit format.	CS-1.336#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.337#implementation/unit_test/catch.hpp	Write source information to XML	Writes 'filename' and 'line' attributes to the current XML element based on source line information.	CS-1.337#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp
8.338#implementation/unit_test/catch.hpp	Write test case data to XML	Writes a 'testcase' element to the XML, using section data and deriving the class name from test case info or tags.	CS-1.338#implementation/unit_test/catch.hpp	7.55#implementation/unit_test/catch.hpp

Code Snippets & Code Functions for System Features

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.1#implementation/unit_test/catch.hpp	namespace Catch {	Provides stream access, an interface, and a reusable string stream class.	8.1#implementation/unit_test/catch.hpp	Loc #612 - 645
CS-1.1#implementation/unit_test/catch.hpp	// start catch_objc_arc.hpp	Checks for Objective-C ARC compiler support via a macro and declares a function for safely releasing objects.	8.1#implementation/unit_test/catch.hpp	Loc #650 - 660
CS-1.1#implementation/unit_test/catch.hpp	#if !CATCH_ARC_ENABLED	Provides helper functions and macros for managing object memory and calling methods, adapting for Objective-C Automatic Reference Counting (ARC) enablement.	8.1#implementation/unit_test/catch.hpp	Loc #663 - 691
CS-1.1#implementation/unit_test/catch.hpp	#endif	Disables a specific MSVC compiler warning and declares a dummy type and stream operator for testing.	8.1#implementation/unit_test/catch.hpp	Loc #693 - 703
CS-1.1#implementation/unit_test/catch.hpp	using ::operator<<;	Imports global stream operator and starts a namespace for internal details.	8.1#implementation/unit_test/catch.hpp	Loc #706 - 709
CS-1.1#implementation/unit_test/catch.hpp	std::string rawMemoryToString(const T& object) {	Converts any object into a string based on its raw memory bytes.	8.1#implementation/unit_test/catch.hpp	Loc #715 - 723
CS-1.1#implementation/unit_test/catch.hpp	static const bool value = decltype(test<std::ostream, const T&>(0))::value;	Checks at compile time if type T can be output to a stream.	8.1#implementation/unit_test/catch.hpp	Loc #600 - 730
CS-1.2#implementation/unit_test/catch.hpp	namespace Catch {	Defines types for test results and flags for handling outcomes in a testing framework.	8.2#implementation/unit_test/catch.hpp	Loc #537 - 578
CS-1.3#implementation/unit_test/catch.hpp	#define INTERNAL_CATCH_UNIQUE_NAME_LINE2(name, line) name##line	Provides common utility types and macros for the Catch testing framework, handling unique names, source locations, and non-copyable objects.	8.3#implementation/unit_test/catch.hpp	Loc #209 - 278
CS-1.4#implementation/unit_test/catch.hpp	#define INTERNAL_CATCH_TEST_CASE_NO_REGISTRATION(TestName, ...) \	Define a static function with a specified name, typically for use as a test case entry point.	8.4#implementation/unit_test/catch.hpp	Loc #480 - 481

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.4#implementation/unit_test/catch.hpp	#define INTERNAL_CATCH_TEST_CASE_METHOD_NO_REGISTRATION(TestName, ClassName, ...) \	Defines a test struct within an anonymous namespace, inheriting from a base class with a test method.	8.4#implementation/unit_test/catch.hpp	Loc #482 - 488
CS-1.4#implementation/unit_test/catch.hpp	#endif	Defines a macro to declare a test case within the Catch testing framework.	8.4#implementation/unit_test/catch.hpp	Loc #490 - 493
CS-1.4#implementation/unit_test/catch.hpp	CATCH_INTERNAL_SUPPRESS_GLOBALS_WARNINGS \	Macros for registering test cases and methods with the Catch2 testing framework.	8.4#implementation/unit_test/catch.hpp	Loc #479 - 528
CS-1.5#implementation/unit_test/catch.hpp	namespace Catch {	Defines interfaces and types for managing, invoking, registering, matching, and filtering test cases within a C++ testing framework.	8.5#implementation/unit_test/catch.hpp	Loc #300 - 324
CS-1.5#implementation/unit_test/catch.hpp	namespace Catch {	Provides a non-owning view of string data. Can take ownership for C-style string access if needed.	8.5#implementation/unit_test/catch.hpp	Loc #333 - 442
CS-1.5#implementation/unit_test/catch.hpp	namespace Catch {	Handles invoking test methods within classes and automatically registering test case information.	8.5#implementation/unit_test/catch.hpp	Loc #445 - 477
CS-1.6#implementation/unit_test/catch.hpp	#if defined(CATCH_CONFIG_MAIN) defined(CATCH_CONFIG_RUNNER)	Configures Catch2 compilation flags and platform definitions based on user-defined macros.	8.6#implementation/unit_test/catch.hpp	Loc #42 - 82
CS-1.6#implementation/unit_test/catch.hpp	// CATCH_CONFIG_WINDOWS_SEH : is Windows SEH supported?	Detects compiler and environment capabilities like SEH, POSIX signals, C++ standard, and suppresses warnings, defining configuration macros.	8.6#implementation/unit_test/catch.hpp	Loc #95 - 208
CS-1.7#implementation/unit_test/catch.hpp	namespace Catch {	Provides access to standard streams and defines a reusable string stream utility within the Catch framework.	8.7#implementation/unit_test/catch.hpp	Loc #612 - 645
CS-1.8#implementation/unit_test/catch.hpp	namespace Catch {	Defines stream-related utilities for outputting data and managing reusable string streams.	8.8#implementation/unit_test/catch.hpp	Loc #612 - 645
CS-1.8#implementation/unit_test/catch.hpp	// start catch_objc_arc.hpp	Checks if Objective-C ARC is enabled and declares a function for potentially safe object	8.8#implementation/unit_test/catch.hpp	Loc #650 - 660

ID	Feature Name	Description	Feature ID	Code Lines
		release.		
CS-1.8#implementation/unit_test/catch.hpp	#if !CATCH_ARC_ENABLED	Adapts memory management and selector calls based on whether Objective-C ARC is enabled or disabled.	8.8#implementation/unit_test/catch.hpp	Loc #663 - 691
CS-1.8#implementation/unit_test/catch.hpp	#endif	Prepares stream output handling for the Catch framework, including suppressing a specific compiler warning.	8.8#implementation/unit_test/catch.hpp	Loc #693 - 703
CS-1.8#implementation/unit_test/catch.hpp	using ::operator<<;	Imports the global output stream operator into the Detail namespace.	8.8#implementation/unit_test/catch.hpp	Loc #706 - 709
CS-1.8#implementation/unit_test/catch.hpp	std::string rawMemoryToString(const T& object) {	Converts the raw memory bytes of an object into a string representation.	8.8#implementation/unit_test/catch.hpp	Loc #715 - 723
CS-1.8#implementation/unit_test/catch.hpp	static const bool value = decltype(test<std::ostream, const T&>(0))::value;	Checks if type T is compatible with std::ostream using SFINAE.	8.8#implementation/unit_test/catch.hpp	Loc #598 - 730
CS-1.9#implementation/unit_test/catch.hpp	namespace Catch {	Registers an alias for a test tag within the testing framework.	8.9#implementation/unit_test/catch.hpp	Loc #279 - 285
CS-1.9#implementation/unit_test/catch.hpp	#define CATCH_REGISTER_TAG_ALIAS(alias, spec) \	Registers a short alias for a longer tag specification string within the Catch2 framework.	8.9#implementation/unit_test/catch.hpp	Loc #287 - 290
CS-1.10#implementation/unit_test/catch.hpp	namespace Catch {	Holds data about a test assertion in Catch, like the macro, location, expression, and result flags.	8.10#implementation/unit_test/catch.hpp	Loc #581 - 595
CS-1.11#implementation/unit_test/catch.hpp	// start catch_suppress_warnings.h	Disables specific compiler warnings conditionally based on compiler (Clang, GCC, ICC) to reduce build noise.	8.11#implementation/unit_test/catch.hpp	Loc #22 - 41
CS-1.12#implementation/unit_test/catch.hpp	struct MessageInfo {	This struct stores details for a message, including its source macro name, code location, result type, and a unique sequence number for ordering.	8.12#implementation/unit_test/catch.hpp	Loc #1512 - 1527
CS-1.12#implementation/unit_test/catch.hpp	struct MessageStream {	Wraps a string stream, enabling data of different types to be added using the '<<' operator for	8.12#implementation/unit_test/catch.hpp	Loc #1529 - 1538

ID	Feature Name	Description	Feature ID	Code Lines
		stream-like writing.		
CS-1.12#implementation/unit_test/catch.hpp	struct MessageBuilder : MessageStream {	Builds a message incrementally using stream operators, initialized with context information.	8.12#implementation/unit_test/catch.hpp	Loc #1540 - 1552
CS-1.12#implementation/unit_test/catch.hpp	class ScopedMessage {	Manages a message's lifecycle using RAII, initializing info on creation and cleaning up on destruction.	8.12#implementation/unit_test/catch.hpp	Loc #1554 - 1560
CS-1.13#implementation/unit_test/catch.hpp	typename std::enable_if<!std::is_enum<T>::value, std::string>::type convertUnstreamable(T const&) {	Provides a string representation for non-enum types, returning a fixed unprintable string as a fallback.	8.13#implementation/unit_test/catch.hpp	Loc #736 - 738
CS-1.13#implementation/unit_test/catch.hpp	typename std::enable_if<std::is_enum<T>::value, std::string>::type convertUnstreamable(T const& value) {	Converts an enum value to a string using a helper function, enabled only for enums.	8.13#implementation/unit_test/catch.hpp	Loc #740 - 742
CS-1.13#implementation/unit_test/catch.hpp	template <typename T, typename = void>	Provides a way to convert types into strings using streams, enabled by checking type capabilities.	8.13#implementation/unit_test/catch.hpp	Loc #747 - 752
CS-1.13#implementation/unit_test/catch.hpp	rss << value;	Converts various data types to string representations using template specialization and streaming.	8.13#implementation/unit_test/catch.hpp	Loc #754 - 823
CS-1.13#implementation/unit_test/catch.hpp	static std::string convert(const char* str) {	Converts a C-style string to a string representation using Catch's internal stringifier.	8.13#implementation/unit_test/catch.hpp	Loc #824 - 826
CS-1.13#implementation/unit_test/catch.hpp	struct StringMaker<signed char[SZ]> {	Specializes StringMaker to convert C-style strings to std::string representation using Catch2.	8.13#implementation/unit_test/catch.hpp	Loc #829 - 833
CS-1.13#implementation/unit_test/catch.hpp	struct StringMaker<unsigned char[SZ]> {	Converts a C-style string to a string representation using an internal helper function.	8.13#implementation/unit_test/catch.hpp	Loc #835 - 839
CS-1.13#implementation/unit_test/catch.hpp	struct StringMaker<int> {	Specializes StringMaker to convert an integer value into a string.	8.13#implementation/unit_test/catch.hpp	Loc #842 - 844
CS-1.13#implementation/unit_test/catch.hpp	struct StringMaker<long> {	Defines a specialized way to convert a long integer to a string.	8.13#implementation/unit_test/catch.hpp	Loc #846 - 848
CS-1.13#implementation/unit_test/catch.hpp	struct StringMaker<long long> {	Defines a specific method for converting a long	8.13#implementation/unit_test/catch.hpp	Loc #850 - 853

ID	Feature Name	Description	Feature ID	Code Lines
on/unit_test/catch.hpp		long integer to a string.	h.hpp	
CS-1.13#implementation/unit_test/catch.hpp	struct StringMaker<unsigned int> {	Specializes a template to convert unsigned integers into strings.	8.13#implementation/unit_test/catch.hpp	Loc #854 - 856
CS-1.13#implementation/unit_test/catch.hpp	struct StringMaker<unsigned long> {	Specializes StringMaker struct for unsigned long, declares function to convert it to string.	8.13#implementation/unit_test/catch.hpp	Loc #858 - 860
CS-1.13#implementation/unit_test/catch.hpp	struct StringMaker<unsigned long long> {	Defines how to convert an unsigned long long integer into a string using a static function.	8.13#implementation/unit_test/catch.hpp	Loc #862 - 864
CS-1.13#implementation/unit_test/catch.hpp		Specifies how to convert a boolean value into a string.	8.13#implementation/unit_test/catch.hpp	Loc #865 - 868
CS-1.13#implementation/unit_test/catch.hpp	template<>	Defines how to convert a single character (`char`) into a string using a specialized template structure.	8.13#implementation/unit_test/catch.hpp	Loc #871 - 875
CS-1.13#implementation/unit_test/catch.hpp	struct StringMaker<signed char> {	Specializes StringMaker for signed char type, declaring a static method to convert a signed char value to a string.	8.13#implementation/unit_test/catch.hpp	Loc #876 - 878
CS-1.13#implementation/unit_test/catch.hpp	struct StringMaker<unsigned char> {	Specializes StringMaker struct to convert an unsigned character to string.	8.13#implementation/unit_test/catch.hpp	Loc #880 - 882
CS-1.13#implementation/unit_test/catch.hpp	struct StringMaker<std::nullptr_t> {	Provides a specialized way to convert `std::nullptr_t` to a string using `StringMaker`.	8.13#implementation/unit_test/catch.hpp	Loc #885 - 887
CS-1.13#implementation/unit_test/catch.hpp		Defines how to convert float, double, and pointer types into string representations using template specializations.	8.13#implementation/unit_test/catch.hpp	Loc #888 - 911
CS-1.13#implementation/unit_test/catch.hpp	static std::string convert(R C::* p) {	Converts a pointer to a class member into its string representation.	8.13#implementation/unit_test/catch.hpp	Loc #912 - 918
CS-1.13#implementation/unit_test/catch.hpp	namespace Detail {	This function converts a range of elements, defined by input iterators, into a comma-separated string representation enclosed in curly braces for display.	8.13#implementation/unit_test/catch.hpp	Loc #921 - 934
CS-1.13#implementation/unit_test/catch.hpp	struct StringMaker<NSString*> {	Converts an Objective-C NSString pointer to a C++ std::string, handling null and prepending	8.13#implementation/unit_test/catch.hpp	Loc #938 - 944

ID	Feature Name	Description	Feature ID	Code Lines
		"@".		
CS-1.13#implementation/unit_test/catch.hpp	struct StringMaker<NSObject*> {	Converts an Objective-C NSObject pointer to a C++ std::string using its description method.	8.13#implementation/unit_test/catch.hpp	Loc #946 - 951
CS-1.13#implementation/unit_test/catch.hpp	namespace Detail {	Converts an NSString pointer to a standard C++ string using a specialized class.	8.13#implementation/unit_test/catch.hpp	Loc #952 - 957
CS-1.13#implementation/unit_test/catch.hpp	# define CATCH_CONFIG_ENABLE_PAIR_STRINGMAKER	Provides string representations for standard C++ types like pair, tuple, range, and chrono objects for testing.	8.13#implementation/unit_test/catch.hpp	Loc #732 - 1195
CS-1.14#implementation/unit_test/catch.hpp	class ExprLhs {	Represents the left side of an expression, storing a value and enabling comparisons with other values to create expression objects.	8.14#implementation/unit_test/catch.hpp	Loc #1294 - 1335
CS-1.14#implementation/unit_test/catch.hpp	void handleExpression(ExprLhs<T> const& expr) {	Processes an expression by converting it into a unary expression and then handling that unary expression further.	8.14#implementation/unit_test/catch.hpp	Loc #1340 - 1342
CS-1.14#implementation/unit_test/catch.hpp	struct Decomposer {	Defines a struct using the '<=' operator to capture the left-hand side value into an 'ExprLhs' object, supporting various types.	8.14#implementation/unit_test/catch.hpp	Loc #1344 - 1353
CS-1.15#implementation/unit_test/catch.hpp	namespace Catch {	Interface defining methods for capturing and reporting test results within the Catch2 framework.	8.15#implementation/unit_test/catch.hpp	Loc #1366 - 1430
CS-1.16#implementation/unit_test/catch.hpp	struct TestFailureException{};	Provides components for handling test assertions, capturing results, and reacting to assertion outcomes or exceptions.	8.16#implementation/unit_test/catch.hpp	Loc #1435 - 1501
CS-1.17#implementation/unit_test/catch.hpp	struct ITransientExpression {	Base structure for expression types, storing if binary and evaluation outcome, and requiring reconstruction streaming.	8.17#implementation/unit_test/catch.hpp	Loc #1214 - 1231
CS-1.17#implementation/unit_test/catch.hpp	class BinaryExpr : public ITransientExpression {	Represents a binary expression with left/right sides, operator, and comparison result.	8.17#implementation/unit_test/catch.hpp	Loc #1236 - 1253
CS-1.17#implementation/unit_test/catch.hpp	class UnaryExpr : public ITransientExpression {	Represents a unary expression, wrapping a value for stringification and evaluating its	8.17#implementation/unit_test/catch.hpp	Loc #1256 - 1268

ID	Feature Name	Description	Feature ID	Code Lines
		truthiness.		
CS-1.18#implementation/unit_test/catch.hpp	auto compareEqual(LhsT const& lhs, RhsT const& rhs) -> bool { return lhs == rhs; };	Provides equality and inequality comparison functions supporting standard types and pointer-integer comparisons.	8.18#implementation/unit_test/catch.hpp	Loc #1270 - 1291
CS-1.19#implementation/unit_test/catch.hpp	#define CATCH_INTERNAL_STRINGIFY(...) #__VA_ARGS__	Defines macros for stringifying code and conditionally enabling try/catch blocks based on build flags.	8.19#implementation/unit_test/catch.hpp	Loc #1565 - 1586
CS-1.20#implementation/unit_test/catch.hpp	class MatcherUntypedBase {	Provides a base class for 'matcher' objects that require a string description.	8.20#implementation/unit_test/catch.hpp	Loc #2141 - 2152
CS-1.21#implementation/unit_test/catch.hpp	#define INTERNAL_CATCH_REACT(handler) handler.complete();	Defines internal macros for Catch2 test assertions, handling expression evaluation, exceptions, result reporting, and conditional test flow.	8.21#implementation/unit_test/catch.hpp	Loc #1588 - 1611
CS-1.22#implementation/unit_test/catch.hpp	#define INTERNAL_CATCH_NO_THROW(macroName, resultDisposition, ...) \	Asserts that the provided code expression does not throw any exception.	8.22#implementation/unit_test/catch.hpp	Loc #1614 - 1625
CS-1.22#implementation/unit_test/catch.hpp	#define INTERNAL_CATCH_THROWS(macroName, resultDisposition, ...) \	Macro to check if a code expression throws an exception within the Catch2 test framework.	8.22#implementation/unit_test/catch.hpp	Loc #1628 - 1642
CS-1.22#implementation/unit_test/catch.hpp	#define INTERNAL_CATCH_THROWS_AS(macroName, exceptionType, resultDisposition, expr) \	Tests if an expression throws a specific C++ exception type and reports the test result.	8.22#implementation/unit_test/catch.hpp	Loc #1645 - 1662
CS-1.22#implementation/unit_test/catch.hpp	#define INTERNAL_CATCH_THROWS_STR_MATCHES(macroName, resultDisposition, matcher, ...) \	Checks if executing a code block throws an exception and if its string representation matches a specified pattern using a test framework handler.	8.22#implementation/unit_test/catch.hpp	Loc #1678 - 1692
CS-1.23#implementation/unit_test/catch.hpp	#define INTERNAL_CATCH_MSG(macroName, messageType, resultDisposition, ...) \	Macro handles and logs messages in Catch2 testing, reacting based on type and disposition.	8.23#implementation/unit_test/catch.hpp	Loc #1665 - 1670
CS-1.23#implementation/unit_test/catch.hpp	#define INTERNAL_CATCH_INFO(	This macro logs an informational message within	8.23#implementation/unit_test/catch.hpp	Loc #1673 -

ID	Feature Name	Description	Feature ID	Code Lines
on/unit_test/catch.hpp	macroName, log) \	the Catch2 testing framework, including the source macro name and location, using a scoped object.	h.hpp	1674
CS-1.24#implementation/unit_test/catch.hpp	namespace Catch {	Defines interfaces and functions for managing test components (test cases, reporters, exceptions, tags) within a testing framework.	8.24#implementation/unit_test/catch.hpp	Loc #1861 - 1901
CS-1.25#implementation/unit_test/catch.hpp	namespace Catch {	Allows comparing floating-point numbers approximately using customizable tolerance settings (epsilon, margin, scale).	8.25#implementation/unit_test/catch.hpp	Loc #1980 - 2094
CS-1.26#implementation/unit_test/catch.hpp	namespace Catch {	Manages and times a code loop for benchmarking, determining when enough iterations are run.	8.26#implementation/unit_test/catch.hpp	Loc #1813 - 1848
CS-1.26#implementation/unit_test/catch.hpp	#define BENCHMARK(name) \	Creates a loop structure for benchmarking code with a specific name using a benchmark loop object.	8.26#implementation/unit_test/catch.hpp	Loc #1850 - 1851
CS-1.27#implementation/unit_test/catch.hpp	#define INTERNAL_CATCH_TRANSLATE_EXCEPTION_NO_REG(translatorName, signature) \	Defines a static function translating an exception to a string, without implicit registration.	8.27#implementation/unit_test/catch.hpp	Loc #1905 - 1906
CS-1.27#implementation/unit_test/catch.hpp	namespace Catch {	Provides a mechanism to register functions that translate C++ exceptions into descriptive strings using a chain of translators.	8.27#implementation/unit_test/catch.hpp	Loc #1913 - 1962
CS-1.27#implementation/unit_test/catch.hpp	#define INTERNAL_CATCH_TRANSLATE_EXCEPTION2(translatorName, signature) \	This macro defines and registers a static function used by Catch2 to translate specific exceptions into strings, aiding test reporting.	8.27#implementation/unit_test/catch.hpp	Loc #1965 - 1970
CS-1.27#implementation/unit_test/catch.hpp	#define INTERNAL_CATCH_TRANSLATE_EXCEPTION(signature) INTERNAL_CATCH_TRANSLATE_EXCEPTION2(INTERNAL_CATCH_UNIQUE_NAME(catch_internal_ExceptionTranslator), signature)	Defines a macro calling another to set up exception translation with unique ID and signature.	8.27#implementation/unit_test/catch.hpp	Loc #1904 - 1972
CS-1.28#implementation/unit_test/catch.hpp	namespace Catch {	Defines a timer class to measure elapsed time	8.28#implementation/unit_test/catch.hpp	Loc #1763 -

ID	Feature Name	Description	Feature ID	Code Lines
on/unit_test/catch.hpp		intervals.	h.hpp	1778
CS-1.29#implementation/unit_test/catch.hpp	namespace Catch {	Defines structures to manage and calculate statistics for test results, tracking passed and failed counts.	8.29#implementation/unit_test/catch.hpp	Loc #1705 - 1730
CS-1.30#implementation/unit_test/catch.hpp	namespace Catch {	Manages a specific section within a test case, handling its execution status, assertions, and timing in a testing framework.	8.30#implementation/unit_test/catch.hpp	Loc #1783 - 1802
CS-1.31#implementation/unit_test/catch.hpp	namespace Catch {	Provides utility functions for common string manipulations and a helper for generating plural labels.	8.31#implementation/unit_test/catch.hpp	Loc #2102 - 2122
CS-1.32#implementation/unit_test/catch.hpp	#define INTERNAL_CATCH_SECTION(...) \	Defines a macro for creating test sections in Catch2, using an 'if' statement to control execution flow and section state.	8.32#implementation/unit_test/catch.hpp	Loc #1804 - 1805
CS-1.33#implementation/unit_test/catch.hpp	namespace Catch {	Structures used by Catch2 to hold details about test sections during execution and completion.	8.33#implementation/unit_test/catch.hpp	Loc #1735 - 1756
CS-1.34#implementation/unit_test/catch.hpp	struct MatchAllOf : MatcherBase<ArgT> {	A matcher that passes if and only if all contained matchers pass on an argument.	8.34#implementation/unit_test/catch.hpp	Loc #2172 - 2202
CS-1.35#implementation/unit_test/catch.hpp	struct MatchAnyOf : MatcherBase<ArgT> {	Checks if an argument matches at least one of several criteria.	8.35#implementation/unit_test/catch.hpp	Loc #2204 - 2235
CS-1.36#implementation/unit_test/catch.hpp	namespace Floating {	Defines two classes for comparing floating-point numbers against a target using absolute or ULPS tolerance for testing purposes.	8.36#implementation/unit_test/catch.hpp	Loc #2283 - 2306
CS-1.37#implementation/unit_test/catch.hpp	struct MatcherMethod {	Defines an interface with a pure virtual method to check if an object matches certain criteria.	8.37#implementation/unit_test/catch.hpp	Loc #2155 - 2157
CS-1.37#implementation/unit_test/catch.hpp	struct MatcherMethod<PtrT*> {	Defines a template interface with a pure virtual method for checking if a pointer matches specific criteria.	8.37#implementation/unit_test/catch.hpp	Loc #2159 - 2161
CS-1.37#implementation/unit_test/catch.hpp	struct MatcherBase : MatcherUntypedBase, MatcherMethod<T> {	Defines a base structure for value matchers allowing logical AND, OR, and NOT combinations.	8.37#implementation/unit_test/catch.hpp	Loc #2164 - 2169

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.38#implementation/unit_test/catch.hpp	namespace Matchers {	Provides objects to match NSString content based on equality, containment, or position.	8.38#implementation/unit_test/catch.hpp	Loc #2817 - 2901
CS-1.38#implementation/unit_test/catch.hpp	#define OC_MAKE_UNIQUE_NAME(root, uniqueSuffix) root##uniqueSuffix	Macros to define Objective-C test cases with unique names for a testing framework.	8.38#implementation/unit_test/catch.hpp	Loc #2815 - 2925
CS-1.39#implementation/unit_test/catch.hpp	class OcMethod : public ITestInvoker {	Invokes a specified method on an instance of a class, typically for testing, handling optional setup and teardown.	8.39#implementation/unit_test/catch.hpp	Loc #2748 - 2767
CS-1.39#implementation/unit_test/catch.hpp	namespace Detail{	Retrieves a specific annotation value for a test case from a class.	8.39#implementation/unit_test/catch.hpp	Loc #2769 - 2782
CS-1.39#implementation/unit_test/catch.hpp	inline std::size_t registerTestMethods() {	Scans Objective-C classes and methods to find and register test cases based on a naming pattern.	8.39#implementation/unit_test/catch.hpp	Loc #2784 - 2813
CS-1.40#implementation/unit_test/catch.hpp	namespace StdString {	Defines structures for matching strings with options for case sensitivity and regular expressions.	8.40#implementation/unit_test/catch.hpp	Loc #2325 - 2372
CS-1.41#implementation/unit_test/catch.hpp	namespace Vector {	Provides reusable components (matchers) to check properties of std::vector objects, such as containing specific elements or equality with other vectors.	8.41#implementation/unit_test/catch.hpp	Loc #2394 - 2531
CS-1.41#implementation/unit_test/catch.hpp	Vector::ContainsMatcher<T> Contains(std::vector<T> const& comparator) {	Creates a matcher object to check if a vector contains elements from another vector.	8.41#implementation/unit_test/catch.hpp	Loc #2537 - 2539
CS-1.41#implementation/unit_test/catch.hpp	Vector::ContainsElementMatcher<T> VectorContains(T const& comparator) {	Creates a matcher object to check if a vector contains a specific element.	8.41#implementation/unit_test/catch.hpp	Loc #2542 - 2544
CS-1.41#implementation/unit_test/catch.hpp	Vector::EqualsMatcher<T> Equals(std::vector<T> const& comparator) {	Creates a specialized matcher object designed for comparing vectors for equality using a provided vector.	8.41#implementation/unit_test/catch.hpp	Loc #2547 - 2549
CS-1.41#implementation/unit_test/catch.hpp	Vector::UnorderedEqualsMatcher<T> UnorderedEquals(std::vector<T>	Creates a matcher object to compare vectors ignoring element order.	8.41#implementation/unit_test/catch.hpp	Loc #2552 - 2554

ID	Feature Name	Description	Feature ID	Code Lines
	const& target) {			
CS-1.42#implementation/unit_test/catch.hpp	#define INTERNAL_CATCH_THROWS_MATCHES(macroName, exceptionType, resultDisposition, matcher, ...) \	Asserts that a code block throws an exception of a specified type and that the thrown exception matches a given condition.	8.42#implementation/unit_test/catch.hpp	Loc #2607 - 2624
CS-1.43#implementation/unit_test/catch.hpp	#define INTERNAL_CHECK_THAT(macroName, matcher, resultDisposition, arg) \	Implements an assertion checking an argument against a matcher using Catch2 testing framework, handling evaluation and reporting.	8.43#implementation/unit_test/catch.hpp	Loc #2597 - 2604
CS-1.44#implementation/unit_test/catch.hpp	namespace Catch	Represents a string pattern with optional wildcards (*). Checks if other strings match the pattern based on case sensitivity and wildcard location.	8.44#implementation/unit_test/catch.hpp	Loc #2952 - 2974
CS-1.45#implementation/unit_test/catch.hpp	struct MatchNotOf : MatcherBase<ArgT> {	Creates a matcher that negates the result of another matcher.	8.45#implementation/unit_test/catch.hpp	Loc #2238 - 2250
CS-1.46#implementation/unit_test/catch.hpp	class TestSpecParser {	Parses a string specification, creating test filters and patterns including handling names, tags, and exclusion rules.	8.46#implementation/unit_test/catch.hpp	Loc #3061 - 3105
CS-1.47#implementation/unit_test/catch.hpp	namespace Catch {	Defines how to specify which test cases to run using patterns for names, tags, and exclusions, combined into filters.	8.47#implementation/unit_test/catch.hpp	Loc #2981 - 3032
CS-1.48#implementation/unit_test/catch.hpp	class MatchExpr : public ITransientExpression {	This class represents a condition check, storing an argument, a matcher, the match result, and information to reconstruct the expression string.	8.48#implementation/unit_test/catch.hpp	Loc #2563 - 2583
CS-1.48#implementation/unit_test/catch.hpp	using StringMatcher = Matchers::Impl::MatcherBase<std::string>;	Provides types and functions to handle matching expressions for testing or assertions.	8.48#implementation/unit_test/catch.hpp	Loc #2562 - 2592
CS-1.49#implementation/unit_test/catch.hpp	struct IConfig : NonCopyable {	Defines an interface for accessing test configuration settings, preventing copying.	8.49#implementation/unit_test/catch.hpp	Loc #3159 - 3179
CS-1.50#implementation/unit_test/catch.hpp	struct IStreamingReporter {	Defines an interface for reporting test run events like starts, ends, assertions, and sections.	8.50#implementation/unit_test/catch.hpp	Loc #3543 - 3582
CS-1.50#implementation/unit_test/catch.hpp	using IStreamingReporterPtr = std::	Defines blueprints for creating and managing	8.50#implementation/unit_test/catch.hpp	Loc #3543 -

ID	Feature Name	Description	Feature ID	Code Lines
on/unit_test/catch.hpp	unique_ptr<IStreamingReporter>;	different types of software report generators.	h.hpp	3600
CS-1.51#implementation/unit_test/catch.hpp	struct AssertionResultData	Stores data about an assertion result, including type, message, and the original evaluated expression.	8.51#implementation/unit_test/catch.hpp	Loc #3290 - 3302
CS-1.51#implementation/unit_test/catch.hpp	class AssertionResult {	Represents the outcome and details of a single assertion evaluation within a testing framework.	8.51#implementation/unit_test/catch.hpp	Loc #3304 - 3325
CS-1.52#implementation/unit_test/catch.hpp	struct ConfigData {	Stores configuration settings for a test runner or application, controlling execution, reporting, and output options.	8.52#implementation/unit_test/catch.hpp	Loc #3199 - 3231
CS-1.52#implementation/unit_test/catch.hpp	class Config : public IConfig {	Stores and provides access to configuration settings for a testing framework, including run options and output streams.	8.52#implementation/unit_test/catch.hpp	Loc #3233 - 3279
CS-1.53#implementation/unit_test/catch.hpp	class Option {	A class template that can hold a value of type T or no value, similar to std::optional.	8.53#implementation/unit_test/catch.hpp	Loc #3336 - 3390
CS-1.53#implementation/unit_test/catch.hpp	struct LazyStat : Option<T> {	A struct wrapping Option<T>, resetting a 'used' flag when the value is assigned or cleared.	8.53#implementation/unit_test/catch.hpp	Loc #3421 - 3432
CS-1.54#implementation/unit_test/catch.hpp	class ReporterRegistrar {	Registers a factory for creating reporter objects with a global registry.	8.54#implementation/unit_test/catch.hpp	Loc #3931 - 3949
CS-1.54#implementation/unit_test/catch.hpp	class ListenerRegistrar {	Registers a factory that creates specific listener or reporter objects with a central registry.	8.54#implementation/unit_test/catch.hpp	Loc #3952 - 3969
CS-1.55#implementation/unit_test/catch.hpp	struct Colour {	Defines text color codes and manages console text color using RAII or static method.	8.55#implementation/unit_test/catch.hpp	Loc #3871 - 3918
CS-1.56#implementation/unit_test/catch.hpp	struct CumulativeReporterBase : IStreamingReporter {	Base class for test reporters accumulating results in a tree structure, handling sections, assertions, and test cases.	8.56#implementation/unit_test/catch.hpp	Loc #3691 - 3845
CS-1.57#implementation/unit_test/catch.hpp	struct StreamingReporterBase : IStreamingReporter {	Base class for streaming test reporters, managing configuration, stream, and test lifecycle state.	8.57#implementation/unit_test/catch.hpp	Loc #3622 - 3688
CS-1.58#implementation/unit_test/catch.hpp	struct TestRunInfo {	Defines a structure to hold information about a test run, including its name.	8.58#implementation/unit_test/catch.hpp	Loc #3434 - 3437

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.58#implementation/unit_test/catch.hpp	struct GroupInfo {	Defines a structure to hold information about a group including name, index, and total count.	8.58#implementation/unit_test/catch.hpp	Loc #3438 - 3446
CS-1.58#implementation/unit_test/catch.hpp	struct AssertionStats {	Stores the result of a test assertion, related messages, and overall test counts.	8.58#implementation/unit_test/catch.hpp	Loc #3448 - 3462
CS-1.58#implementation/unit_test/catch.hpp	struct SectionStats {	Holds statistics for a code or test section, including info, assertion counts, duration, and missing assertions.	8.58#implementation/unit_test/catch.hpp	Loc #3464 - 3479
CS-1.58#implementation/unit_test/catch.hpp	struct TestCaseStats {	Stores statistics and results for a single test run, including test info, totals, output, error streams, and abort status.	8.58#implementation/unit_test/catch.hpp	Loc #3481 - 3499
CS-1.58#implementation/unit_test/catch.hpp	struct TestGroupStats {	Stores statistics and information related to a specific test group.	8.58#implementation/unit_test/catch.hpp	Loc #3501 - 3516
CS-1.58#implementation/unit_test/catch.hpp	struct TestRunStats {	Holds information and statistics for a test run, including totals and its abort status.	8.58#implementation/unit_test/catch.hpp	Loc #3518 - 3532
CS-1.58#implementation/unit_test/catch.hpp	struct BenchmarkInfo {	Defines a structure to hold basic information about a benchmark.	8.58#implementation/unit_test/catch.hpp	Loc #3534 - 3536
CS-1.58#implementation/unit_test/catch.hpp	struct BenchmarkStats {	Stores statistics for a benchmark run including info, iterations, and elapsed time.	8.58#implementation/unit_test/catch.hpp	Loc #3537 - 3541
CS-1.59#implementation/unit_test/catch.hpp	namespace {	Checks if two double numbers are within a specified margin, designed to handle infinity.	8.59#implementation/unit_test/catch.hpp	Loc #4485 - 4493
CS-1.59#implementation/unit_test/catch.hpp	namespace Catch {	Defines Approx class for comparing doubles approximately, accounting for tolerance and scale within Catch.	8.59#implementation/unit_test/catch.hpp	Loc #4495 - 4527
CS-1.60#implementation/unit_test/catch.hpp	namespace Catch {	Defines a structure likely used for detecting memory leaks within the Catch testing framework.	8.60#implementation/unit_test/catch.hpp	Loc #4471 - 4477
CS-1.61#implementation/unit_test/catch.hpp	class XmlEncode {	Encodes a string for use in XML text nodes or attributes.	8.61#implementation/unit_test/catch.hpp	Loc #4099 - 4112
CS-1.62#implementation/unit_test/catch.hpp	class XmlWriter {	Writes XML content to an output stream, managing elements, attributes, text, and	8.62#implementation/unit_test/catch.hpp	Loc #4114 - 4182

ID	Feature Name	Description	Feature ID	Code Lines
		comments.		
CS-1.63#implementation/unit_test/catch.hpp	# define CATCH_DEFINED_NOMINMAX	Configures platform-specific handlers for fatal errors or signals, mainly for testing frameworks like Catch2.	8.63#implementation/unit_test/catch.hpp	Loc #4624 - 4719
CS-1.64#implementation/unit_test/catch.hpp	namespace Catch {	Defines a test reporter for Catch framework to generate compact output for test results.	8.64#implementation/unit_test/catch.hpp	Loc #3994 - 4018
CS-1.65#implementation/unit_test/catch.hpp	std::unique_ptr<TablePrinter> m_tablePrinter;	Handles test events and reports detailed results (assertions, sections, benchmarks, cases, runs) to the console.	8.65#implementation/unit_test/catch.hpp	Loc #4023 - 4070
CS-1.65#implementation/unit_test/catch.hpp	bool m_headerPrinted = false;	Declares a class member flag, closes class/namespace, and manages compiler warnings.	8.65#implementation/unit_test/catch.hpp	Loc #4072 - 4088
CS-1.66#implementation/unit_test/catch.hpp	namespace Catch {	Formats Catch test results into XML output.	8.66#implementation/unit_test/catch.hpp	Loc #4236 - 4279
CS-1.67#implementation/unit_test/catch.hpp	namespace Catch {	Report test execution results in the standard JUnit XML format.	8.67#implementation/unit_test/catch.hpp	Loc #4187 - 4231
CS-1.68#implementation/unit_test/catch.hpp	namespace Catch {	This code declares a function to check if a debugger is currently active and attached to the running program, returning a boolean result.	8.68#implementation/unit_test/catch.hpp	Loc #4585 - 4587
CS-1.68#implementation/unit_test/catch.hpp	#define CATCH_TRAP() __asm__ ("int \$3\n" : :) /* NOLINT */	Defines macros to trigger a debugger breakpoint based on the operating system, compiler, and whether a debugger is attached.	8.68#implementation/unit_test/catch.hpp	Loc #4585 - 4615
CS-1.69#implementation/unit_test/catch.hpp	namespace Catch {	Defines interfaces for the test context and manages a global mutable context instance.	8.69#implementation/unit_test/catch.hpp	Loc #4535 - 4580
CS-1.70#implementation/unit_test/catch.hpp	namespace Catch {	Manages test execution, runs test cases, and captures/reports assertion results using a reporter.	8.70#implementation/unit_test/catch.hpp	Loc #4724 - 4841
CS-1.71#implementation/unit_test/catch.hpp	#define CATCH_REGISTER_REPORTER(name, reporterType)\	Registers a custom test reporter type with the Catch2 framework using a specific name.	8.71#implementation/unit_test/catch.hpp	Loc #3974 - 3977

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.71#implementation/unit_test/catch.hpp	#define CATCH_REGISTER_LISTENER(listenerType) \	Registers a custom listener type with the Catch2 framework.	8.71#implementation/unit_test/catch.hpp	Loc #3979 - 3982
CS-1.71#implementation/unit_test/catch.hpp	#define CATCH_REGISTER_REPORTER(name, reporterType)	These preprocessor macros define empty registration points for Catch2 reporters and listeners.	8.71#implementation/unit_test/catch.hpp	Loc #3972 - 3988
CS-1.72#implementation/unit_test/catch.hpp	namespace Catch {	Manages the execution state and hierarchy of test sections and loops during a test run.	8.72#implementation/unit_test/catch.hpp	Loc #4303 - 4465
CS-1.73#implementation/unit_test/catch.hpp	struct UnaryLambdaTraits : UnaryLambdaTraits<decltype(&L::operator())> {};	Analyzes types to determine if they are const unary lambdas and extracts argument/return types.	8.73#implementation/unit_test/catch.hpp	Loc #5448 - 5462
CS-1.74#implementation/unit_test/catch.hpp	namespace Catch { namespace clara { namespace TextFlow {	Wraps text into columns, handling indentation and line breaks, and combines columns side-by-side.	8.74#implementation/unit_test/catch.hpp	Loc #5129 - 5432
CS-1.75#implementation/unit_test/catch.hpp	AssertionHandler::AssertionHandler	Constructs an assertion handler object, storing details and obtaining a result capture.	8.75#implementation/unit_test/catch.hpp	Loc #4877 - 4884
CS-1.75#implementation/unit_test/catch.hpp	void AssertionHandler::handleExpr(ITransientExpression const& expr) {	Handles an expression within an assertion context by processing it with an internal handler.	8.75#implementation/unit_test/catch.hpp	Loc #4886 - 4888
CS-1.75#implementation/unit_test/catch.hpp	void AssertionHandler::handleMessage(ResultWas::OfType resultType, StringRef const& message) {	Forwards assertion result details and a message to an internal result capturing object for handling.	8.75#implementation/unit_test/catch.hpp	Loc #4889 - 4891
CS-1.75#implementation/unit_test/catch.hpp	auto AssertionHandler::allowThrows() const -> bool {	Gets the setting from the current configuration determining if throwing exceptions is allowed.	8.75#implementation/unit_test/catch.hpp	Loc #4893 - 4895
CS-1.75#implementation/unit_test/catch.hpp	void AssertionHandler::complete() {	Finalizes assertion handling, potentially breaking into debugger or throwing exception.	8.75#implementation/unit_test/catch.hpp	Loc #4897 - 4909
CS-1.75#implementation/unit_test/catch.hpp	void AssertionHandler::setCompleted() {	Sets the completion status of an assertion handler to true.	8.75#implementation/unit_test/catch.hpp	Loc #4910 - 4912
CS-1.75#implementation/unit_test/catch.hpp	void AssertionHandler::handleUnexpectedInflightException() {	Handles an unexpected exception during assertion evaluation by reporting it to a result	8.75#implementation/unit_test/catch.hpp	Loc #4914 - 4916

ID	Feature Name	Description	Feature ID	Code Lines
		capture object.		
CS-1.75#implementation/unit_test/catch.hpp	void AssertionHandler::handleExceptionThrownAsExpected() {	Handles the case where an expected exception was thrown during an assertion.	8.75#implementation/unit_test/catch.hpp	Loc #4918 - 4920
CS-1.75#implementation/unit_test/catch.hpp	void AssertionHandler::handleExceptionNotThrownAsExpected() {	Handles assertions where an exception was expected but not thrown, reporting the result via a capture object.	8.75#implementation/unit_test/catch.hpp	Loc #4921 - 4923
CS-1.75#implementation/unit_test/catch.hpp	void AssertionHandler::handleUnexpectedExceptionNotThrown() {	Handles when an expected exception wasn't thrown, informing the result capture object.	8.75#implementation/unit_test/catch.hpp	Loc #4925 - 4927
CS-1.75#implementation/unit_test/catch.hpp	void AssertionHandler::handleThrowingCallSkipped() {	Records that a potentially throwing function call was skipped, treating it as a successful assertion.	8.75#implementation/unit_test/catch.hpp	Loc #4929 - 4931
CS-1.76#implementation/unit_test/catch.hpp	void handleExceptionMatchExpr(AssertionHandler& handler, std::string const& str, StringRef matcherString) {	Calls another handler function to match an exception using a string equality matcher.	8.76#implementation/unit_test/catch.hpp	Loc #4935 - 4937
CS-1.76#implementation/unit_test/catch.hpp	using StringMatcher = Matchers::Impl::MatcherBase<std::string>;	Catches the active exception message, checks if it matches a given string pattern using a matcher, and processes the assertion result.	8.76#implementation/unit_test/catch.hpp	Loc #5062 - 5071
CS-1.77#implementation/unit_test/catch.hpp	auto BenchmarkLooper::getResolution() -> uint64_t {	Calculates a benchmark resolution by multiplying estimated clock resolution with a configured multiple.	8.77#implementation/unit_test/catch.hpp	Loc #5036 - 5038
CS-1.77#implementation/unit_test/catch.hpp	void BenchmarkLooper::reportStart() {	Reports the start of a benchmark using a result capture object.	8.77#implementation/unit_test/catch.hpp	Loc #5040 - 5042
CS-1.77#implementation/unit_test/catch.hpp	auto BenchmarkLooper::needsMoreIterations() -> bool {	Determines if more benchmark iterations are needed based on elapsed time compared to timer resolution.	8.77#implementation/unit_test/catch.hpp	Loc #5043 - 5054
CS-1.78#implementation/unit_test/catch.hpp	class Args {	Stores the program's executable name and command-line arguments for later access.	8.78#implementation/unit_test/catch.hpp	Loc #5467 - 5487
CS-1.79#implementation/unit_test/catch.hpp	enum class ParseResultType {	Defines possible results of a parsing operation, indicating success or failure modes.	8.79#implementation/unit_test/catch.hpp	Loc #5687 - 5689

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.79#implementation/unit_test/catch.hpp	class ParseState {	Holds parsing outcome status and the remaining tokens after parsing attempt.	8.79#implementation/unit_test/catch.hpp	Loc #5691 - 5705
CS-1.80#implementation/unit_test/catch.hpp	AssertionResult::AssertionResult(AssertionInfo const& info, AssertionResultData const& data)	Creates an AssertionResult object storing assertion details and outcome data.	8.80#implementation/unit_test/catch.hpp	Loc #4960 - 4963
CS-1.80#implementation/unit_test/catch.hpp	bool AssertionResult::succeeded() const {	Checks if an assertion result object represents a successful outcome.	8.80#implementation/unit_test/catch.hpp	Loc #4966 - 4968
CS-1.80#implementation/unit_test/catch.hpp	bool AssertionResult::isOk() const {	Checks if the assertion result type is okay or failure is suppressed.	8.80#implementation/unit_test/catch.hpp	Loc #4971 - 4973
CS-1.80#implementation/unit_test/catch.hpp	ResultWas::OfType AssertionResult::getResultType() const {	Gets the specific type of the assertion result stored internally in the object.	8.80#implementation/unit_test/catch.hpp	Loc #4975 - 4977
CS-1.80#implementation/unit_test/catch.hpp	bool AssertionResult::hasExpression() const {	Checks if an assertion result contains a captured expression string.	8.80#implementation/unit_test/catch.hpp	Loc #4979 - 4981
CS-1.80#implementation/unit_test/catch.hpp	bool AssertionResult::hasMessage() const {	Checks if the assertion result object holds a non-empty message string.	8.80#implementation/unit_test/catch.hpp	Loc #4983 - 4985
CS-1.80#implementation/unit_test/catch.hpp	std::string AssertionResult::getExpression() const {	Gets the formatted expression string for an assertion, adding negation if the test was for a false condition.	8.80#implementation/unit_test/catch.hpp	Loc #4987 - 4992
CS-1.80#implementation/unit_test/catch.hpp	std::string AssertionResult::getExpressionInMacro() const {	Creates a string representing the assertion expression, either directly or formatted within its surrounding macro.	8.80#implementation/unit_test/catch.hpp	Loc #4994 - 5006
CS-1.80#implementation/unit_test/catch.hpp	bool AssertionResult::hasExpandedExpression() const {	Checks if an expanded expression exists and is different from the original expression.	8.80#implementation/unit_test/catch.hpp	Loc #5008 - 5010
CS-1.80#implementation/unit_test/catch.hpp	std::string AssertionResult::getExpandedExpression() const {	This function attempts to reconstruct an expression; if unsuccessful, it returns the original expression.	8.80#implementation/unit_test/catch.hpp	Loc #5012 - 5017
CS-1.80#implementation/unit_test/catch.hpp	std::string AssertionResult::getMessage()	This method retrieves the assertion message stored within the result data. It provides external	8.80#implementation/unit_test/catch.hpp	Loc #5019 - 5021

ID	Feature Name	Description	Feature ID	Code Lines
	const {	access to the message.		
CS-1.80#implementation/unit_test/catch.hpp	SourceLineInfo AssertionResult::getSourceInfo() const {	Retrieves the source line information associated with an assertion result.	8.80#implementation/unit_test/catch.hpp	Loc #5022 - 5024
CS-1.80#implementation/unit_test/catch.hpp	StringRef AssertionResult::getTestMacroName() const {	Retrieves the name of the test macro associated with the assertion result.	8.80#implementation/unit_test/catch.hpp	Loc #5026 - 5028
CS-1.81#implementation/unit_test/catch.hpp	LazyExpression::LazyExpression(bool isNegated)	Initializes a LazyExpression object, setting its negation flag.	8.81#implementation/unit_test/catch.hpp	Loc #4851 - 4853
CS-1.81#implementation/unit_test/catch.hpp	LazyExpression::LazyExpression(LazyExpression const& other) : m_isNegated(other.m_isNegated) {}	Defines how to copy, convert to boolean, and print a lazy expression object.	8.81#implementation/unit_test/catch.hpp	Loc #4851 - 4875
CS-1.82#implementation/unit_test/catch.hpp	class ResultBase {	Defines a base class for operation results, indicating success, logic errors, or runtime errors using an enumeration.	8.82#implementation/unit_test/catch.hpp	Loc #5580 - 5593
CS-1.82#implementation/unit_test/catch.hpp	class ResultValueBase : public ResultBase {	Base class wrapping a value of type T that is only valid when the result type is 'Ok'.	8.82#implementation/unit_test/catch.hpp	Loc #5596 - 5632
CS-1.82#implementation/unit_test/catch.hpp	class ResultValueBase<void> : public ResultBase {	Provides a base class for results without a value, inheriting error status.	8.82#implementation/unit_test/catch.hpp	Loc #5635 - 5638
CS-1.82#implementation/unit_test/catch.hpp	class BasicResult : public ResultValueBase<T> {	Represents operation outcome: success with value (from base class) or error with message.	8.82#implementation/unit_test/catch.hpp	Loc #5641 - 5685
CS-1.83#implementation/unit_test/catch.hpp	AssertionResultData::AssertionResultData(ResultWas::OfType _resultType, LazyExpression const& _lazyExpression):	Initializes an AssertionResultData object by setting its result type and associated lazy expression from the provided inputs.	8.83#implementation/unit_test/catch.hpp	Loc #4944 - 4946
CS-1.83#implementation/unit_test/catch.hpp	std::string AssertionResultData::reconstructExpression() const {	Reconstructs an expression string from a lazy source, caching the result for future calls.	8.83#implementation/unit_test/catch.hpp	Loc #4948 - 4958
CS-1.84#implementation/unit_test/catch.hpp	auto operator <<(std::ostream& os, ITransientExpression const& expr) -> std::ostream& {	Enables printing an expression object's reconstructed form to an output stream.	8.84#implementation/unit_test/catch.hpp	Loc #4846 - 4849

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.85#implementation/unit_test/catch.hpp	enum class TokenType {	Defines a type for tokens, specifying if it's an option or an argument.	8.85#implementation/unit_test/catch.hpp	Loc #5491 - 5493
CS-1.85#implementation/unit_test/catch.hpp	struct Token {	Represents a single token with its type and string value.	8.85#implementation/unit_test/catch.hpp	Loc #5494 - 5497
CS-1.85#implementation/unit_test/catch.hpp	inline auto isOptPrefix(char c) -> bool {	Checks if a character is a standard command-line option prefix like '-' or '/' on Windows.	8.85#implementation/unit_test/catch.hpp	Loc #5499 - 5505
CS-1.85#implementation/unit_test/catch.hpp	using Iterator = std::vector<std::string>::const_iterator;	Clears a token buffer, skips empty input strings, and checks the prefix of the next non-empty string.	8.85#implementation/unit_test/catch.hpp	Loc #5489 - 5523
CS-1.85#implementation/unit_test/catch.hpp	explicit TokenStream(Args const &args) : TokenStream(args.m_args.begin(), args.m_args.end()) {}	Manages a stream of tokens using a buffer, providing iteration and access.	8.85#implementation/unit_test/catch.hpp	Loc #5525 - 5578
CS-1.86#implementation/unit_test/catch.hpp	struct BoundFlagRef : BoundFlagRefBase {	Wraps a boolean reference to allow setting its value using a standard interface or parser result.	8.86#implementation/unit_test/catch.hpp	Loc #5805 - 5814
CS-1.87#implementation/unit_test/catch.hpp	struct BoundLambda : BoundValueRefBase {	Wrap a unary lambda to invoke it with a string input after converting the string to the lambda's argument type.	8.87#implementation/unit_test/catch.hpp	Loc #5845 - 5854
CS-1.87#implementation/unit_test/catch.hpp	struct BoundFlagLambda : BoundFlagRefBase {	Adapts a lambda taking one boolean argument to a standard flag-setting interface for parsing.	8.87#implementation/unit_test/catch.hpp	Loc #5857 - 5868
CS-1.88#implementation/unit_test/catch.hpp	struct BoundRef : BoundValueRefBase {	Binds a reference to a variable and allows setting its value from a string.	8.88#implementation/unit_test/catch.hpp	Loc #5778 - 5786
CS-1.88#implementation/unit_test/catch.hpp	struct BoundRef<std::vector<T>> : BoundValueRefBase {	Helper struct storing reference to a vector, allowing string inputs to be converted and appended as new elements to the referenced vector.	8.88#implementation/unit_test/catch.hpp	Loc #5789 - 5803
CS-1.89#implementation/unit_test/catch.hpp	clara::Parser makeCommandLineParser(ConfigData& config) {	Creates a command-line parser to configure test execution based on user inputs.	8.89#implementation/unit_test/catch.hpp	Loc #6337 - 6501

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.90#implementation/unit_test/catch.hpp	struct Parser : ParserBase {	Defines a command-line parser structure. It holds options, arguments, executable name, and handles validation, help generation, and parsing.	8.90#implementation/unit_test/catch.hpp	Loc #6132 - 6278
CS-1.91#implementation/unit_test/catch.hpp	inline auto convertInto(std::string const &source, T& target) -> ParserResult {	Converts a string to a specified data type using stringstream.	8.91#implementation/unit_test/catch.hpp	Loc #5717 - 5725
CS-1.91#implementation/unit_test/catch.hpp	inline auto convertInto(std::string const &source, std::string& target) -> ParserResult {	Copies content from a source string to a target string and returns success.	8.91#implementation/unit_test/catch.hpp	Loc #5726 - 5729
CS-1.91#implementation/unit_test/catch.hpp	inline auto convertInto(std::string const &source, bool &target) -> ParserResult {	Converts specific string values representing boolean (true/false) into a boolean variable.	8.91#implementation/unit_test/catch.hpp	Loc #5730 - 5740
CS-1.92#implementation/unit_test/catch.hpp	struct HelpColumns {	Defines a structure to group two text strings, likely representing paired data for columns.	8.92#implementation/unit_test/catch.hpp	Loc #5711 - 5714
CS-1.93#implementation/unit_test/catch.hpp	enum class Optionality { Optional, Required };	Base classes for creating composable parsers binding values, managing optionality, hints, and descriptions.	8.93#implementation/unit_test/catch.hpp	Loc #5870 - 5944
CS-1.93#implementation/unit_test/catch.hpp	auto ComposableParserImpl<DerivedT>::operator (T const &other) const -> Parser {	Implements the ' ' operator to combine the current parser implementation with a distinct element, producing a new composed parser object representing their sequence.	8.93#implementation/unit_test/catch.hpp	Loc #6282 - 6284
CS-1.94#implementation/unit_test/catch.hpp	using Result = BasicResult<void>;	Define easy-to-use names for different types of operation results.	8.94#implementation/unit_test/catch.hpp	Loc #5707 - 5709
CS-1.95#implementation/unit_test/catch.hpp	class ExeName : public ComposableParserImpl<ExeName> {	Manages program executable name, extracts filename from path, and optionally binds it to an external variable or lambda.	8.95#implementation/unit_test/catch.hpp	Loc #5946 - 5986
CS-1.96#implementation/unit_test/catch.hpp	struct Help : Opt {	Defines a help option that shows usage and stops parsing.	8.96#implementation/unit_test/catch.hpp	Loc #6118 - 6130
CS-1.97#implementation/unit_test/catch.hpp	struct BoundRefBase {	Defines an abstract base interface for command-line argument or config value binding, preventing copying.	8.97#implementation/unit_test/catch.hpp	Loc #5742 - 5755

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.97#implementation/unit_test/catch.hpp	struct BoundValueRefBase : BoundRefBase {	Base class for references to non-flag bound values, preventing flag operations.	8.97#implementation/unit_test/catch.hpp	Loc #5757 - 5763
CS-1.97#implementation/unit_test/catch.hpp	struct BoundFlagRefBase : BoundRefBase {	Base struct for command-line arguments representing boolean flags, converting strings to boolean state.	8.97#implementation/unit_test/catch.hpp	Loc #5765 - 5775
CS-1.98#implementation/unit_test/catch.hpp	struct LambdaInvoker {	Helper struct to invoke a lambda function and ensure it returns a specific type (ParserResult).	8.98#implementation/unit_test/catch.hpp	Loc #5817 - 5824
CS-1.98#implementation/unit_test/catch.hpp	struct LambdaInvoker<void> {	Calls a void-returning lambda with an argument and returns a successful parse result.	8.98#implementation/unit_test/catch.hpp	Loc #5827 - 5833
CS-1.98#implementation/unit_test/catch.hpp	inline auto invokeLambda(L const &lambda, std::string const &arg) -> ParserResult {	Converts string to argument type and invokes lambda, handling conversion errors.	8.98#implementation/unit_test/catch.hpp	Loc #5836 - 5842
CS-1.99#implementation/unit_test/catch.hpp	inline auto normaliseOpt(std::string const &optName) -> std::string {	Standardizes command-line option prefixes, changing '/' to '-' on Windows.	8.99#implementation/unit_test/catch.hpp	Loc #6010 - 6017
CS-1.100#implementation/unit_test/catch.hpp	class Arg : public ParserRefImpl<Arg> {	Parses an argument token from a stream, setting its value if found.	8.100#implementation/unit_test/catch.hpp	Loc #5988 - 6008
CS-1.101#implementation/unit_test/catch.hpp	class Opt : public ParserRefImpl<Opt> {	Represents a command-line option with names and logic to parse its value or presence from input tokens.	8.101#implementation/unit_test/catch.hpp	Loc #6019 - 6116
CS-1.102#implementation/unit_test/catch.hpp	class MultipleReporters : public IStreamingReporter {	Manages multiple test reporters and forwards streaming reporter events to each one.	8.102#implementation/unit_test/catch.hpp	Loc #7318 - 7352
CS-1.103#implementation/unit_test/catch.hpp	namespace Catch {	Defines a colour interface and a do-nothing default implementation using a singleton pattern within the Catch namespace.	8.103#implementation/unit_test/catch.hpp	Loc #6635 - 6653
CS-1.103#implementation/unit_test/catch.hpp	# define CATCH_CONFIG_COLOUR_WINDOWS	Provides platform-specific console colour handling using Windows API or ANSI escape codes.	8.103#implementation/unit_test/catch.hpp	Loc #6633 - 6790
CS-1.103#implementation/unit_test/catch.hpp	namespace Catch {	Provides a no-color implementation instance for platform color in Catch framework.	8.103#implementation/unit_test/catch.hpp	Loc #6794 - 6798
CS-1.103#implementation/unit_test/catch.hpp	namespace Catch {	Manages console output colors using a	8.103#implementation/unit_test/catch.hpp	Loc #6802 -

ID	Feature Name	Description	Feature ID	Code Lines
ion/unit_test/catch.hpp		platform-specific backend, handling object lifecycle and move semantics.	ch.hpp	6826
CS-1.104#implementation/unit_test/catch.hpp	namespace Catch {	Checks if the current process is attached to or running under a debugger on macOS.	8.104#implementation/unit_test/cat ch.hpp	Loc #6927 - 6965
CS-1.104#implementation/unit_test/catch.hpp	namespace Catch{	Checks if the current process is being debugged by reading its /proc file system status on Linux.	8.104#implementation/unit_test/cat ch.hpp	Loc #6971 - 6996
CS-1.104#implementation/unit_test/catch.hpp	namespace Catch {	Checks if a debugger is attached to the current process.	8.104#implementation/unit_test/cat ch.hpp	Loc #6999 - 7003
CS-1.104#implementation/unit_test/catch.hpp	namespace Catch {	Checks if a debugger is currently attached to the running program using the Windows API.	8.104#implementation/unit_test/cat ch.hpp	Loc #7006 - 7010
CS-1.104#implementation/unit_test/catch.hpp	namespace Catch {	Provides a function that reports if a debugger is active, always returning false.	8.104#implementation/unit_test/cat ch.hpp	Loc #7012 - 7014
CS-1.105#implementation/unit_test/catch.hpp	void formatReconstructedExpression(std::ostream &os, std::string const& lhs, StringRef op, std::string const& rhs) {	Formats and prints expression parts, using one or multiple lines based on size and content.	8.105#implementation/unit_test/cat ch.hpp	Loc #7023 - 7030
CS-1.106#implementation/unit_test/catch.hpp	namespace {	Reports a fatal error condition using the Catch2 test framework.	8.106#implementation/unit_test/cat ch.hpp	Loc #7124 - 7129
CS-1.106#implementation/unit_test/catch.hpp	namespace Catch {	Defines an empty method within a fatal condition handler class.	8.106#implementation/unit_test/cat ch.hpp	Loc #7135 - 7137
CS-1.106#implementation/unit_test/catch.hpp	namespace Catch {	Sets up a handler for Windows exceptions like stack overflow/access violation to report them and ensures stack space.	8.106#implementation/unit_test/cat ch.hpp	Loc #7141 - 7195
CS-1.106#implementation/unit_test/catch.hpp	namespace Catch {	Defines a reset method for a fatal condition handler; currently, this method performs no action.	8.106#implementation/unit_test/cat ch.hpp	Loc #7203 - 7205
CS-1.106#implementation/unit_test/catch.hpp	namespace Catch {	Sets up signal handlers to catch fatal system errors like crashes, report them, and restore original handlers.	8.106#implementation/unit_test/cat ch.hpp	Loc #7211 - 7276

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.107#implementation/unit_test/catch.hpp	namespace Catch {	Manages central components (config, runner, results) for Catch testing framework, providing access and global instance management.	8.107#implementation/unit_test/catch.hpp	Loc #6835 - 6886
CS-1.108#implementation/unit_test/catch.hpp	namespace Catch {	Manages exception translators to convert C++ exceptions into human-readable strings.	8.108#implementation/unit_test/catch.hpp	Loc #7050 - 7062
CS-1.108#implementation/unit_test/catch.hpp	namespace Catch {	Manages and uses registered objects to convert active exceptions into human-readable string messages.	8.108#implementation/unit_test/catch.hpp	Loc #7069 - 7115
CS-1.109#implementation/unit_test/catch.hpp	namespace Catch {	Saves and restores the global 'errno' value automatically.	8.109#implementation/unit_test/catch.hpp	Loc #7037 - 7040
CS-1.110#implementation/unit_test/catch.hpp	namespace Catch {	Declares a function to write given text to the debug console.	8.110#implementation/unit_test/catch.hpp	Loc #6894 - 6896
CS-1.110#implementation/unit_test/catch.hpp	namespace Catch {	Writes a string to the Windows debug console using the system API.	8.110#implementation/unit_test/catch.hpp	Loc #6901 - 6905
CS-1.110#implementation/unit_test/catch.hpp	namespace Catch {	Writes given text to the debug console using the Catch framework's output stream.	8.110#implementation/unit_test/catch.hpp	Loc #6907 - 6912
CS-1.111#implementation/unit_test/catch.hpp	ReporterConfig::ReporterConfig(IConfigPtr const& _fullConfig)	Initializes ReporterConfig using a full configuration object, storing it and a reference to its stream.	8.111#implementation/unit_test/catch.hpp	Loc #7359 - 7360
CS-1.111#implementation/unit_test/catch.hpp	ReporterConfig::ReporterConfig(IConfigPtr const& _fullConfig, std::ostream& _stream)	Initializes a ReporterConfig object with a shared pointer to configuration data and a reference to an output stream.	8.111#implementation/unit_test/catch.hpp	Loc #7362 - 7363
CS-1.111#implementation/unit_test/catch.hpp	std::ostream& ReporterConfig::stream() const { return *m_stream; }	Defines constructors and destructors for various classes representing test run statistics and information at different levels.	8.111#implementation/unit_test/catch.hpp	Loc #7359 - 7449
CS-1.112#implementation/unit_test/catch.hpp	namespace Catch {	Provides utility functions for checking test result types and flags.	8.112#implementation/unit_test/catch.hpp	Loc #8308 - 8324
CS-1.113#implementation/unit_test/catch.hpp	namespace Catch {	Defines default destructors for two testing interfaces.	8.113#implementation/unit_test/catch.hpp	Loc #7488 - 7491
CS-1.114#implementation/unit_test/catch.hpp	namespace Catch {	Defines the default destructor for the IRunner	8.114#implementation/unit_test/catch.hpp	Loc #7482 -

ID	Feature Name	Description	Feature ID	Code Lines
ion/unit_test/catch.hpp		class.	ch.hpp	7484
CS-1.115#implementation/unit_test/catch.hpp	namespace Matchers {	Provides a cached string representation for a base matcher object, computing it only once. Includes a standard destructor.	8.115#implementation/unit_test/cat ch.hpp	Loc #7702 - 7714
CS-1.116#implementation/unit_test/catch.hpp	namespace Catch {	Defines an enum class to distinguish between float and double floating-point types.	8.116#implementation/unit_test/cat ch.hpp	Loc #7728 - 7737
CS-1.116#implementation/unit_test/catch.hpp	namespace {	Compares floating-point numbers for approximate equality using their bit representations (ULP difference).	8.116#implementation/unit_test/cat ch.hpp	Loc #7739 - 7787
CS-1.116#implementation/unit_test/catch.hpp	namespace Catch {	Provides matchers to compare floating-point numbers with tolerance, based on absolute difference or Units in the Last Place (ULPs).	8.116#implementation/unit_test/cat ch.hpp	Loc #7789 - 7846
CS-1.117#implementation/unit_test/catch.hpp	namespace Catch {	Configures C runtime library for memory leak detection and reporting setup.	8.117#implementation/unit_test/cat ch.hpp	Loc #7498 - 7510
CS-1.117#implementation/unit_test/catch.hpp	Catch::LeakDetector::LeakDetector() {}	Creates an instance of the Catch framework's memory leak detector object.	8.117#implementation/unit_test/cat ch.hpp	Loc #7495 - 7516
CS-1.118#implementation/unit_test/catch.hpp	namespace Catch {	Manages and uses a random number generator, allowing seeding and generating random values based on configuration.	8.118#implementation/unit_test/cat ch.hpp	Loc #8041 - 8058
CS-1.119#implementation/unit_test/catch.hpp	class RedirectedStream {	Temporarily redirects output from one C++ stream to another, restoring original state when destroyed.	8.119#implementation/unit_test/cat ch.hpp	Loc #8334 - 8350
CS-1.120#implementation/unit_test/catch.hpp	namespace Catch {	Provides string matchers for testing equality, containment, prefixes, suffixes, and regex with case sensitivity.	8.120#implementation/unit_test/cat ch.hpp	Loc #7853 - 7949
CS-1.121#implementation/unit_test/catch.hpp	std::size_t listReporters(Config const& /*config*/) {	Lists available Catch2 test reporters with descriptions. Helps users choose an output format.	8.121#implementation/unit_test/cat ch.hpp	Loc #7662 - 7682
CS-1.122#implementation/unit_test/catch.hpp	std::size_t listTests(Config const& config) {	This function lists and prints details for test cases matching the configuration filters or all available	8.122#implementation/unit_test/cat ch.hpp	Loc #7560 - 7593

ID	Feature Name	Description	Feature ID	Code Lines
		tests if no filters are specified.		
CS-1.122#implementation/unit_test/catch.hpp	std::size_t listTestsNamesOnly(Config const& config) {	Lists names of test cases matching configuration filters, printing detailed info if verbosity is high.	8.122#implementation/unit_test/catch.hpp	Loc #7595 - 7612
CS-1.123#implementation/unit_test/catch.hpp	struct TagInfo {	Stores unique tag spellings in a set and an associated count.	8.123#implementation/unit_test/catch.hpp	Loc #7530 - 7536
CS-1.123#implementation/unit_test/catch.hpp	std::size_t listTags(Config const& config) {	Lists and counts tags from filtered or all test cases, displaying counts and tag names formatted.	8.123#implementation/unit_test/catch.hpp	Loc #7626 - 7660
CS-1.124#implementation/unit_test/catch.hpp	namespace Catch {	Manages global registries for tests, reporters, exceptions, and tag aliases in Catch framework, providing read and write access via a singleton.	8.124#implementation/unit_test/catch.hpp	Loc #8197 - 8275
CS-1.125#implementation/unit_test/catch.hpp	namespace Catch {	Handles test messages and assertions, managing their information, construction, and lifecycle.	8.125#implementation/unit_test/catch.hpp	Loc #7953 - 8004
CS-1.126#implementation/unit_test/catch.hpp	void IStreamingReporter::fatalError Encountered(StringRef) {	Adds an additional reporter to an existing one, creating a multi-reporter if needed.	8.126#implementation/unit_test/catch.hpp	Loc #7451 - 7478
CS-1.126#implementation/unit_test/catch.hpp	namespace Catch {	Manages reporter and listener factories, allowing creation of reporters by name and access to registered factories.	8.126#implementation/unit_test/catch.hpp	Loc #8279 - 8304
CS-1.127#implementation/unit_test/catch.hpp	Option<std::size_t> list(Config const& config) {	Lists items (tests, names, tags, reporters) based on configuration flags, summing and returning the total count listed.	8.127#implementation/unit_test/catch.hpp	Loc #7684 - 7695
CS-1.128#implementation/unit_test/catch.hpp	clara::Parser const& Session::cli() const {	Provides access to the session's command-line parser object.	8.128#implementation/unit_test/catch.hpp	Loc #9116 - 9118
CS-1.129#implementation/unit_test/catch.hpp	ConfigData& Session::configData() {	Provides read/write access to the session's configuration data.	8.129#implementation/unit_test/catch.hpp	Loc #9122 - 9124
CS-1.130#implementation/unit_test/catch.hpp	Config& Session::config() {	Provides access to a configuration object, creating it on first request if needed.	8.130#implementation/unit_test/catch.hpp	Loc #9125 - 9129
CS-1.131#implementation/unit_test/catch.hpp	void addListeners(IStreamingReporterPtr& reporters, IConfigPtr const&	Iterates through registered listeners and adds corresponding reporter instances to a collection	8.131#implementation/unit_test/catch.hpp	Loc #8944 - 8948

ID	Feature Name	Description	Feature ID	Code Lines
	config) {	based on configuration.		
CS-1.132#implementation/unit_test/catch.hpp	int Session::applyCommandLine(int argc, char* argv[]) {	Processes command line arguments and applies configuration based on flags.	8.132#implementation/unit_test/catch.hpp	Loc #9043 - 9064
CS-1.133#implementation/unit_test/catch.hpp	void applyFileNamesAsTags(Catch::IConfig const& config) {	Adds modified base filename (prefixed #, no extension) as a tag for each test case.	8.133#implementation/unit_test/catch.hpp	Loc #8976 - 8996
CS-1.134#implementation/unit_test/catch.hpp	bool RunContext::aborting() const {	Determines if the test run should stop based on the number of failed assertions compared to a configured limit.	8.134#implementation/unit_test/catch.hpp	Loc #8603 - 8605
CS-1.135#implementation/unit_test/catch.hpp	bool RunContext::testForMissingAssertions(Counts& assertions) {	Checks for missing assertions in the current test scope. Updates failure counts if missing assertions found and warnings are enabled.	8.135#implementation/unit_test/catch.hpp	Loc #8492 - 8502
CS-1.136#implementation/unit_test/catch.hpp	bool RunContext::lastAssertionPassed() {	Checks if the previous assertion evaluation was successful.	8.136#implementation/unit_test/catch.hpp	Loc #8593 - 8595
CS-1.137#implementation/unit_test/catch.hpp	IStreamingReporterPtr makeReporter(std::shared_ptr<Config> const& config) {	Creates test reporters based on config names, chaining multiple if needed.	8.137#implementation/unit_test/catch.hpp	Loc #8931 - 8940
CS-1.138#implementation/unit_test/catch.hpp	IStreamingReporterPtr createReporter(std::string const& reporterName, IConfigPtr const& config) {	Creates a test reporter instance using a registry lookup based on name and configuration.	8.138#implementation/unit_test/catch.hpp	Loc #8920 - 8925
CS-1.139#implementation/unit_test/catch.hpp	Catch::IStream::~IStream() = default;	Defines the destructor for the IStream class using the default compiler implementation.	8.139#implementation/unit_test/catch.hpp	Loc #9196 - 9196
CS-1.140#implementation/unit_test/catch.hpp	void RunContext::populateReaction(AssertionReaction& reaction) {	Populates an assertion reaction object based on the current run context configuration and state.	8.140#implementation/unit_test/catch.hpp	Loc #8746 - 8749
CS-1.141#implementation/unit_test/catch.hpp	void Session::showHelp() const {	Prints help information about the Catch testing framework, including version and CLI options.	8.141#implementation/unit_test/catch.hpp	Loc #9029 - 9034
CS-1.142#implementation/unit_test/catch.hpp	void Session::libIdentify() {	Prints identification details about the executable to the console.	8.142#implementation/unit_test/catch.hpp	Loc #9035 - 9041
CS-1.143#implementation/unit_test/catch.hpp	void RunContext::sectionEndedEarly	Handles a section ending before expected, either	8.143#implementation/unit_test/catch.hpp	Loc #8517 -

ID	Feature Name	Description	Feature ID	Code Lines
ion/unit_test/catch.hpp	y(SectionEndInfo const & endInfo) {	failing or closing the last active section and marking it as unfinished.	ch.hpp	8525
CS-1.144#implementation/unit_test/catch.hpp	void RunContext::sectionEnded(SectionEndInfo const & endInfo) {	Handles the end of a test section by calculating stats, closing it, reporting results, and clearing messages.	8.144#implementation/unit_test/catch.hpp	Loc #8504 - 8515
CS-1.145#implementation/unit_test/catch.hpp	void RunContext::runCurrentTest(std::string & redirectedCout, std::string & redirectedCerr) {	Runs a single test case, handles output redirection, timing, exceptions, and reports results.	8.145#implementation/unit_test/catch.hpp	Loc #8607 - 8651
CS-1.146#implementation/unit_test/catch.hpp	Catch::Totals runTests(std::shared_ptr<Config> const& config) {	Runs tests based on configuration and specification, collects results, reports progress.	8.146#implementation/unit_test/catch.hpp	Loc #8950 - 8974
CS-1.147#implementation/unit_test/catch.hpp	RunContext::~~RunContext() {	Signals the end of a test run to a reporter object, providing final statistics and status.	8.147#implementation/unit_test/catch.hpp	Loc #8389 - 8391
CS-1.148#implementation/unit_test/catch.hpp	Session::~~Session() {	Destructor for the Session class. It cleans up resources when a Session object is destroyed, specifically calling a cleanup function.	8.148#implementation/unit_test/catch.hpp	Loc #9025 - 9027
CS-1.149#implementation/unit_test/catch.hpp	IResultCapture& getResultCapture() {	Gets the result capture object from the current testing context, ensuring it exists or reporting an internal error if not found. Provides access for test result reporting.	8.149#implementation/unit_test/catch.hpp	Loc #8776 - 8781
CS-1.150#implementation/unit_test/catch.hpp	std::string RunContext::getCurrentTestName() const {	Retrieves the name of the currently active test case, returning empty string if none is active.	8.150#implementation/unit_test/catch.hpp	Loc #8541 - 8545
CS-1.151#implementation/unit_test/catch.hpp	const AssertionResult * RunContext::getLastResult() const {	Returns a pointer to the last assertion result stored internally.	8.151#implementation/unit_test/catch.hpp	Loc #8547 - 8549
CS-1.152#implementation/unit_test/catch.hpp	std::vector<std::exception_ptr> const& StartupExceptionRegistry::getExceptions() const noexcept {	Provides read-only access to a list of captured startup exceptions.	8.152#implementation/unit_test/catch.hpp	Loc #9174 - 9176
CS-1.153#implementation/unit_test/catch.hpp	void RunContext::handleExpr(	Handles assertion evaluation, reports start, checks result, and reports pass or fail.	8.153#implementation/unit_test/catch.hpp	Loc #8670 - 8692

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.154#implementation/unit_test/catch.hpp	void RunContext::handleFatalErrorCondition(StringRef message) {	Handles a fatal error during a test run, reports it, and gracefully shuts down the current test case and run.	8.154#implementation/unit_test/catch.hpp	Loc #8555 - 8591
CS-1.155#implementation/unit_test/catch.hpp	void RunContext::handleIncomplete(	Handles incomplete test assertions due to exceptions when translation is disabled, records info, creates a result, and ends assertion.	8.155#implementation/unit_test/catch.hpp	Loc #8751 - 8760
CS-1.156#implementation/unit_test/catch.hpp	void RunContext::handleMessage(	Processes a test assertion result, reports start and end, and handles failures.	8.156#implementation/unit_test/catch.hpp	Loc #8708 - 8724
CS-1.157#implementation/unit_test/catch.hpp	void RunContext::handleNonExpr(	Handles assertions without evaluated expressions, records the assertion result, and populates a reaction object if the result indicates a failure.	8.157#implementation/unit_test/catch.hpp	Loc #8761 - 8774
CS-1.158#implementation/unit_test/catch.hpp	void RunContext::handleUnexpectedExceptionNotThrown(	Handles the scenario where an expected exception was not thrown during assertion evaluation.	8.158#implementation/unit_test/catch.hpp	Loc #8725 - 8730
CS-1.159#implementation/unit_test/catch.hpp	void RunContext::handleUnexpectedInflightException(	Handles an exception occurring during an assertion, recording the failure, creating a result, and updating the assertion's reaction.	8.159#implementation/unit_test/catch.hpp	Loc #8732 - 8744
CS-1.160#implementation/unit_test/catch.hpp	class CoutStream : public IStream {	Wraps the standard output stream (cout) in an IStream interface for testability.	8.160#implementation/unit_test/catch.hpp	Loc #9262 - 9272
CS-1.161#implementation/unit_test/catch.hpp	class DebugOutputStream : public IStream {	Creates an output stream that writes data to a debug output channel using a custom buffer and stream implementation.	8.161#implementation/unit_test/catch.hpp	Loc #9276 - 9289
CS-1.162#implementation/unit_test/catch.hpp	class FileStream : public IStream {	Opens a file for writing and provides access to the output file stream object.	8.162#implementation/unit_test/catch.hpp	Loc #9246 - 9258
CS-1.163#implementation/unit_test/catch.hpp	class StreamBufImpl : public std::streambuf {	Buffers output for a stream and writes buffered data using a custom function when full or synced.	8.163#implementation/unit_test/catch.hpp	Loc #9200 - 9233
CS-1.164#implementation/unit_test/catch.hpp	void RunContext::assertionPassed() {	Records that an assertion has passed during a test run, updating counters and state.	8.164#implementation/unit_test/catch.hpp	Loc #8597 - 8601

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.165#implementation/unit_test/catch.hpp	RunContext::RunContext(IConfigPtr const& _config, IStreamingReporterPtr&& reporter)	Initializes the test run context with configuration, reporter, and sets up the current test context.	8.165#implementation/unit_test/catch.hpp	Loc #8375 - 8387
CS-1.166#implementation/unit_test/catch.hpp	Session::Session() {	Initializes a test session object, ensuring only one instance and reporting any errors found during startup.	8.166#implementation/unit_test/catch.hpp	Loc #9000 - 9024
CS-1.167#implementation/unit_test/catch.hpp	int Session::runInternal() {	Runs a testing session, handling configuration, seeding, listing, and executing tests with error handling.	8.167#implementation/unit_test/catch.hpp	Loc #9131 - 9157
CS-1.168#implementation/unit_test/catch.hpp	void RunContext::invokeActiveTest Case() {	Executes the active test case while managing potential fatal conditions during its run.	8.168#implementation/unit_test/catch.hpp	Loc #8653 - 8657
CS-1.169#implementation/unit_test/catch.hpp	Section::Section(SectionInfo const& info)	Initializes a test section, records its start, and starts a timer.	8.169#implementation/unit_test/catch.hpp	Loc #8788 - 8793
CS-1.169#implementation/unit_test/catch.hpp	Section::~~Section() {	Finalizes a test section, recording results and handling uncaught exceptions.	8.169#implementation/unit_test/catch.hpp	Loc #8799 - 8807
CS-1.169#implementation/unit_test/catch.hpp	Section::operator bool() const {	Checks if the section is included and returns true or false.	8.169#implementation/unit_test/catch.hpp	Loc #8813 - 8815
CS-1.170#implementation/unit_test/catch.hpp	void RunContext::popScopedMessage(MessageInfo const & message) {	Removes all occurrences of a given message object from the internal list of messages.	8.170#implementation/unit_test/catch.hpp	Loc #8537 - 8539
CS-1.171#implementation/unit_test/catch.hpp	void RunContext::pushScopedMessage(MessageInfo const & message) {	Adds a constant message reference to the internal list of messages for context tracking.	8.171#implementation/unit_test/catch.hpp	Loc #8533 - 8535
CS-1.172#implementation/unit_test/catch.hpp	class RedirectedStdErr {	This class captures output from standard error and logging streams (cerr, clog) into an internal string stream, allowing the output to be retrieved later as a single string.	8.172#implementation/unit_test/catch.hpp	Loc #8363 - 8373
CS-1.173#implementation/unit_test/catch.hpp	class RedirectedStdOut {	Captures standard output into a string stream.	8.173#implementation/unit_test/catch.hpp	Loc #8352 - 8358
CS-1.174#implementation/unit_test/catch.hpp	void StartupExceptionRegistry::add(std::exception_ptr const& exception	Adds an exception pointer to a list of startup errors, terminating if memory allocation fails.	8.174#implementation/unit_test/catch.hpp	Loc #9164 - 9172

ID	Feature Name	Description	Feature ID	Code Lines
	) noexcept {			
CS-1.175#implementation/unit_test/catch.hpp	void RunContext::assertionEnded(AssertionResult const& result) {	Processes an assertion result, updating total pass/fail counts, tracking last result status, notifying the test reporter, and resetting internal state.	8.175#implementation/unit_test/catch.hpp	Loc #8449 - 8471
CS-1.176#implementation/unit_test/catch.hpp	void RunContext::reportExpr(	Processes and reports the outcome of an assertion, including its details and expression, then signals its completion.	8.176#implementation/unit_test/catch.hpp	Loc #8693 - 8706
CS-1.177#implementation/unit_test/catch.hpp	void RunContext::resetAssertionInfo() {	Resets stored information about the last assertion, clearing macro name and setting a default expression message.	8.177#implementation/unit_test/catch.hpp	Loc #8472 - 8475
CS-1.178#implementation/unit_test/catch.hpp	Totals RunContext::runTest(TestCase const& testCase) {	Runs a single test case, tracking progress, capturing output, reporting results, and updating overall totals.	8.178#implementation/unit_test/catch.hpp	Loc #8401 - 8439
CS-1.179#implementation/unit_test/catch.hpp	int Session::run() {	Manages session execution flow, optionally pausing for user input before starting and before exiting based on configuration.	8.179#implementation/unit_test/catch.hpp	Loc #9103 - 9114
CS-1.180#implementation/unit_test/catch.hpp	int Session::run(int argc, char* argv[]) {	Runs the session, processing command line arguments and executing main logic.	8.180#implementation/unit_test/catch.hpp	Loc #9071 - 9078
CS-1.180#implementation/unit_test/catch.hpp	int Session::run(int argc, wchar_t* const argv[]) {	Converts wide character command-line arguments to UTF-8, calls another method, and cleans up memory.	8.180#implementation/unit_test/catch.hpp	Loc #9081 - 9101
CS-1.181#implementation/unit_test/catch.hpp	void Session::cli(clara::Parser const& newParser) {	Sets or updates the command-line parser object for the session.	8.181#implementation/unit_test/catch.hpp	Loc #9119 - 9121
CS-1.182#implementation/unit_test/catch.hpp	void RunContext::benchmarkEnded(BenchmarkStats const& stats) {	Notifies a reporter object that a benchmark has finished, providing statistics.	8.182#implementation/unit_test/catch.hpp	Loc #8529 - 8531
CS-1.183#implementation/unit_test/catch.hpp	void RunContext::benchmarkStarting(BenchmarkInfo const& info) {	Notifies the attached reporter object that a benchmark test is starting, passing its information.	8.183#implementation/unit_test/catch.hpp	Loc #8526 - 8528

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.184#implementation/unit_test/catch.hpp	void RunContext::exceptionEarlyReported() {	Disables reporting of unexpected events by setting a flag to false.	8.184#implementation/unit_test/catch.hpp	Loc #8551 - 8553
CS-1.185#implementation/unit_test/catch.hpp	void RunContext::testGroupEnded(std::string const& testSpec, Totals const& totals, std::size_t groupIndex, std::size_t groupsCount) {	Reports the completion and results of a test group to a reporter.	8.185#implementation/unit_test/catch.hpp	Loc #8397 - 8399
CS-1.186#implementation/unit_test/catch.hpp	void RunContext::testGroupStarting(std::string const& testSpec, std::size_t groupIndex, std::size_t groupsCount) {	Notifies a reporter object that a new test group is starting.	8.186#implementation/unit_test/catch.hpp	Loc #8393 - 8395
CS-1.187#implementation/unit_test/catch.hpp	bool RunContext::sectionStarted(SectionInfo const & sectionInfo, Counts & assertions) {	Starts a test section, checks its status, updates internal state, and reports the start.	8.187#implementation/unit_test/catch.hpp	Loc #8477 - 8490
CS-1.188#implementation/unit_test/catch.hpp	SectionInfo::SectionInfo	Creates a SectionInfo object storing line info, name, and description.	8.188#implementation/unit_test/catch.hpp	Loc #8823 - 8830
CS-1.188#implementation/unit_test/catch.hpp	SectionEndInfo::SectionEndInfo(SectionInfo const& _sectionInfo, Counts const& _prevAssertions, double _durationInSeconds)	Create an object holding section end details like info, previous counts, and duration.	8.188#implementation/unit_test/catch.hpp	Loc #8832 - 8834
CS-1.189#implementation/unit_test/catch.hpp	void RunContext::handleUnfinishedSections() {	Processes prematurely ended sections by calling cleanup and clearing the list.	8.189#implementation/unit_test/catch.hpp	Loc #8659 - 8668
CS-1.190#implementation/unit_test/catch.hpp	void Session::useConfigData(ConfigData const& configData) {	Updates the session's configuration data and resets its internal configuration object.	8.190#implementation/unit_test/catch.hpp	Loc #9066 - 9069
CS-1.191#implementation/unit_test/catch.hpp	struct OutputDebugWriter {	Writes a given string to the debug console using a helper function.	8.191#implementation/unit_test/catch.hpp	Loc #9237 - 9242
CS-1.192#implementation/unit_test/catch.hpp	namespace Catch {	Registers a tag alias with the test framework's internal registry, catching and deferring exceptions during startup.	8.192#implementation/unit_test/catch.hpp	Loc #9566 - 9577
CS-1.193#implementation/unit_test/catch.hpp	namespace Catch {	Functions and classes to help automatically register tests in a testing framework.	8.193#implementation/unit_test/catch.hpp	Loc #10164 - 10189

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.194#implementation/unit_test/catch.hpp	bool contains(std::string const& s, std::string const& infix) {	Checks if a main string contains a smaller substring within it, returning true if found.	8.194#implementation/unit_test/catch.hpp	Loc #9399 - 9401
CS-1.195#implementation/unit_test/catch.hpp	bool startsWith(std::string const& s, std::string const& prefix) {	Checks if a string starts with a given prefix string by comparing initial characters and lengths.	8.195#implementation/unit_test/catch.hpp	Loc #9387 - 9389
CS-1.195#implementation/unit_test/catch.hpp	bool startsWith(std::string const& s, char prefix) {	Checks if a string starts with a specific character, returning true if it does and false otherwise.	8.195#implementation/unit_test/catch.hpp	Loc #9390 - 9392
CS-1.196#implementation/unit_test/catch.hpp	bool endsWith(std::string const& s, std::string const& suffix) {	Checks if the first string ends with the second string.	8.196#implementation/unit_test/catch.hpp	Loc #9393 - 9395
CS-1.196#implementation/unit_test/catch.hpp	bool endsWith(std::string const& s, char suffix) {	Checks if a string ends with a specific character, returning true or false.	8.196#implementation/unit_test/catch.hpp	Loc #9396 - 9398
CS-1.197#implementation/unit_test/catch.hpp	char toLowerCh(char c) {	Converts a character to its lowercase equivalent using C locale.	8.197#implementation/unit_test/catch.hpp	Loc #9402 - 9404
CS-1.197#implementation/unit_test/catch.hpp	void toLowerInPlace(std::string& s) {	Converts all characters in the input string to lowercase directly within the original string.	8.197#implementation/unit_test/catch.hpp	Loc #9405 - 9407
CS-1.197#implementation/unit_test/catch.hpp	std::string toLower(std::string const& s) {	Creates and returns a lowercase copy of the input string.	8.197#implementation/unit_test/catch.hpp	Loc #9408 - 9412
CS-1.198#implementation/unit_test/catch.hpp	auto makeStream(StringRef const& filename) -> IStream const* {	Creates output streams for console, debug, or file based on filename.	8.198#implementation/unit_test/catch.hpp	Loc #9295 - 9306
CS-1.199#implementation/unit_test/catch.hpp	TestCase makeTestCase(ITestInvoker* _testCase,	Creates a test case by parsing description and tags, identifying special properties like 'hidden', and packaging test details.	8.199#implementation/unit_test/catch.hpp	Loc #9662 - 9703
CS-1.200#implementation/unit_test/catch.hpp	namespace Catch {	Defines structures to match test cases by name or tag, combining criteria into filters and specifications for selection.	8.200#implementation/unit_test/catch.hpp	Loc #10198 - 10241
CS-1.201#implementation/unit_test/catch.hpp	void enforceNoDuplicateTestCases(std::vector<TestCase> const& functions) {	Checks a list of test cases for duplicates, stopping and reporting an error if any test case name is found more than once.	8.201#implementation/unit_test/catch.hpp	N/A
CS-1.202#implementation/unit_test/catch.hpp	std::string extractClassName(std::string const& classOrQualifiedMethodName) {	Extracts class name from a qualified method/class name string, handling '&' prefix and namespaces.	8.202#implementation/unit_test/catch.hpp	Loc #9873 - 9884

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.203#implementation/unit_test/catch.hpp	std::vector<TestCase> filterTests(std::vector<TestCase> const& testCases, TestSpec const& testSpec, IConfig const& config) {	Filter a list of test cases based on a test specification and configuration settings.	8.203#implementation/unit_test/catch.hpp	Loc #9830 - 9837
CS-1.204#implementation/unit_test/catch.hpp	pluralise::pluralise(std::size_t count, std::string const& label)	Creates a pluralise object, storing a count and a label for later use.	8.204#implementation/unit_test/catch.hpp	Loc #9435 - 9438
CS-1.204#implementation/unit_test/catch.hpp	std::ostream& operator << (std::ostream& os, pluralise const& pluraliser) {	Formats an object's count and label to an output stream, adding 's' for pluralization.	8.204#implementation/unit_test/catch.hpp	Loc #9440 - 9445
CS-1.205#implementation/unit_test/catch.hpp	struct StringStreams {	Manages a pool of string streams for reuse, reducing overhead of creating new streams frequently.	8.205#implementation/unit_test/catch.hpp	Loc #9309 - 9342
CS-1.205#implementation/unit_test/catch.hpp	StringStreams* StringStreams::s_instance = nullptr;	Defines and initializes a static pointer instance for the StringStreams class.	8.205#implementation/unit_test/catch.hpp	Loc #9308 - 9344
CS-1.206#implementation/unit_test/catch.hpp	TagAliasRegistry::~~TagAliasRegistry() {}	Manages a registry of aliases for tags, allowing finding, adding, and expanding aliases in strings with their corresponding tags.	8.206#implementation/unit_test/catch.hpp	Loc #9583 - 9609
CS-1.206#implementation/unit_test/catch.hpp	ITagAliasRegistry::~~ITagAliasRegistry() {}	Provides global access to the tag alias registry instance for managing test tags.	8.206#implementation/unit_test/catch.hpp	Loc #9616 - 9624
CS-1.207#implementation/unit_test/catch.hpp	void TestRegistry::registerTest(TestCase const& testCase) {	Registers a test case; if unnamed, assigns a unique generated name before registering.	8.207#implementation/unit_test/catch.hpp	Loc #9842 - 9850
CS-1.207#implementation/unit_test/catch.hpp	std::vector<TestCase> const& TestRegistry::getAllTests() const {	Retrieves a list of all registered test cases for execution.	8.207#implementation/unit_test/catch.hpp	Loc #9852 - 9854
CS-1.207#implementation/unit_test/catch.hpp	std::vector<TestCase> const& TestRegistry::getAllTestsSorted(IConfig const& config) const {	Gets sorted tests, checking duplicates and resorting only when config changes or needed.	8.207#implementation/unit_test/catch.hpp	Loc #9855 - 9864
CS-1.208#implementation/unit_test/catch.hpp	TrackerContext& TrackerContext::instance() {	Provides a single global instance of the TrackerContext class using the Singleton pattern.	8.208#implementation/unit_test/catch.hpp	Loc #9911 - 9914
CS-1.208#implementation/unit_test/catch.hpp	ITracker& TrackerContext::startRun() {	Initializes a new tracking run, setting up the root tracker and state.	8.208#implementation/unit_test/catch.hpp	Loc #9916 - 9921

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.208#implementation/unit_test/catch.hpp	void TrackerContext::endRun() {	Stops the current tracking run by clearing internal tracker objects and resetting the state.	8.208#implementation/unit_test/catch.hpp	Loc #9923 - 9927
CS-1.208#implementation/unit_test/catch.hpp	void TrackerContext::startCycle() {	Prepares the tracker context to begin processing by setting the starting tracker and state.	8.208#implementation/unit_test/catch.hpp	Loc #9929 - 9932
CS-1.208#implementation/unit_test/catch.hpp	void TrackerContext::completeCycle() {	Updates the internal state to signify that a full cycle has been completed successfully.	8.208#implementation/unit_test/catch.hpp	Loc #9933 - 9935
CS-1.208#implementation/unit_test/catch.hpp	bool TrackerContext::completedCycle() const {	Checks if the tracker's current state is 'CompletedCycle' and returns the result.	8.208#implementation/unit_test/catch.hpp	Loc #9937 - 9939
CS-1.208#implementation/unit_test/catch.hpp	ITracker& TrackerContext::currentTracker() {	Returns a reference to the current active tracker object.	8.208#implementation/unit_test/catch.hpp	Loc #9940 - 9942
CS-1.208#implementation/unit_test/catch.hpp	void TrackerContext::setCurrentTracker(ITracker* tracker) {	Sets the active tracker object within the context.	8.208#implementation/unit_test/catch.hpp	Loc #9943 - 9945
CS-1.209#implementation/unit_test/catch.hpp	bool matchTest(TestCase const& testCase, TestSpec const& testSpec, IConfig const& config) {	Checks if a test case matches a specification and is allowed based on configuration.	8.209#implementation/unit_test/catch.hpp	Loc #9815 - 9817
CS-1.210#implementation/unit_test/catch.hpp	TestCaseInfo::SpecialProperties parseSpecialTag(std::string const& tag) {	Parses a string tag and returns a specific test case property flag based on predefined values.	8.210#implementation/unit_test/catch.hpp	Loc #9635 - 9651
CS-1.210#implementation/unit_test/catch.hpp	bool isReservedTag(std::string const& tag) {	Checks if a string tag is reserved based on special tag status, size, and first character.	8.210#implementation/unit_test/catch.hpp	Loc #9652 - 9654
CS-1.211#implementation/unit_test/catch.hpp	TestSpecParser::TestSpecParser(ITagAliasRegistry const& tagAliases) : m_tagAliases(&tagAliases) {	Parses an input string to create a test specification for filtering tests.	8.211#implementation/unit_test/catch.hpp	Loc #10245 - 10264
CS-1.212#implementation/unit_test/catch.hpp	std::ostream& cout() { return std::cout; }	Provides functions to access standard C++ output streams: std::cout, std::cerr, and std::clog.	8.212#implementation/unit_test/catch.hpp	Loc #9367 - 9371
CS-1.213#implementation/unit_test/catch.hpp	TrackerBase::TrackerHasName::TrackerHasName(NameAndLocation const& nameAndLocation) :	Defines a base class for tracking execution state hierarchically, managing children, and handling state transitions within a context.	8.213#implementation/unit_test/catch.hpp	Loc #9947 - 10055

ID	Feature Name	Description	Feature ID	Code Lines
	m_nameAndLocation(nameAndLocation) {}			
CS-1.214#implementation/unit_test/catch.hpp	bool replaceInPlace(std::string& str, std::string const& replaceThis, std::string const& withThis) {	Finds all occurrences of a substring within a string and replaces them with another substring directly in the original string, returning true if changes were made.	8.214#implementation/unit_test/catch.hpp	Loc #9421 - 9433
CS-1.215#implementation/unit_test/catch.hpp	TestCase::TestCase(ITestInvoker* testCase, TestCaseInfo const& info) : TestCaseInfo(info), test(testCase) {}	Defines methods for a TestCase, allowing creation, renaming, execution, comparison, and accessing info.	8.215#implementation/unit_test/catch.hpp	Loc #9762 - 9787
CS-1.216#implementation/unit_test/catch.hpp	NameAndLocation::NameAndLocation(std::string const& _name, SourceLineInfo const& _location)	Creates a NameAndLocation object, storing a name string and its source line info.	8.216#implementation/unit_test/catch.hpp	Loc #9904 - 9907
CS-1.217#implementation/unit_test/catch.hpp	StringRef::StringRef(char const* rawChars) noexcept	Creates a StringRef from a null-terminated C-style string by calculating its length and delegating construction.	8.217#implementation/unit_test/catch.hpp	Loc #9460 - 9462
CS-1.217#implementation/unit_test/catch.hpp	StringRef::operator std::string() const {	Creates a standard C++ string object containing the characters referenced by StringRef.	8.217#implementation/unit_test/catch.hpp	Loc #9464 - 9466
CS-1.217#implementation/unit_test/catch.hpp	void StringRef::swap(StringRef& other) noexcept {	Exchanges the internal state (start index, size, data reference) of this StringRef with another StringRef object.	8.217#implementation/unit_test/catch.hpp	Loc #9468 - 9472
CS-1.217#implementation/unit_test/catch.hpp	auto StringRef::c_str() const -> char const* {	Returns a null-terminated C-style string from the StringRef, handling ownership if it's a substring.	8.217#implementation/unit_test/catch.hpp	Loc #9474 - 9478
CS-1.217#implementation/unit_test/catch.hpp	auto StringRef::data() const noexcept -> char const* {	Returns a pointer to the beginning of the string data managed by this object.	8.217#implementation/unit_test/catch.hpp	Loc #9479 - 9481
CS-1.217#implementation/unit_test/catch.hpp	auto StringRef::isOwned() const noexcept -> bool {	Checks if the StringRef object holds a valid data pointer, indicating ownership.	8.217#implementation/unit_test/catch.hpp	Loc #9483 - 9485
CS-1.217#implementation/unit_test/catch.hpp	auto StringRef::isSubstring() const noexcept -> bool {	Checks if the string reference represents a substring not ending with a null terminator.	8.217#implementation/unit_test/catch.hpp	Loc #9486 - 9488
CS-1.217#implementation/unit_test/catch.hpp	void StringRef::takeOwnership() {	Copies string data to new memory if not owned,	8.217#implementation/unit_test/catch.hpp	Loc #9490 -

ID	Feature Name	Description	Feature ID	Code Lines
ion/unit_test/catch.hpp		becoming the owner.	ch.hpp	9497
CS-1.217#implementat ion/unit_test/catch.hpp	autoStringRef::substr(size_type start, size_type size) const noexcept -> StringRef {	Creates a new StringRef representing a portion of the original, starting at a given index and having a specified size.	8.217#implementation/unit_test/cat ch.hpp	Loc #9498 - 9503
CS-1.217#implementat ion/unit_test/catch.hpp	autoStringRef::operator == (StringRef const& other) const noexcept -> bool {	Compares if this string reference is equal to another by size and content.	8.217#implementation/unit_test/cat ch.hpp	Loc #9504 - 9508
CS-1.217#implementat ion/unit_test/catch.hpp	autoStringRef::operator != (StringRef const& other) const noexcept -> bool {	Checks if this StringRef is not equal to another StringRef by negating the equality result.	8.217#implementation/unit_test/cat ch.hpp	Loc #9509 - 9511
CS-1.217#implementat ion/unit_test/catch.hpp	autoStringRef::operator[](size_type index) const noexcept -> char {	Allows accessing a character at a specific position in the string reference.	8.217#implementation/unit_test/cat ch.hpp	Loc #9513 - 9515
CS-1.217#implementat ion/unit_test/catch.hpp	auto StringRef::numberOfCharacters() const noexcept -> size_type {	Counts characters in a string reference, adjusting for multi-byte UTF-8.	8.217#implementation/unit_test/cat ch.hpp	Loc #9517 - 9532
CS-1.217#implementat ion/unit_test/catch.hpp	auto operator + (StringRef const& lhs, StringRef const& rhs) -> std::string {	Concatenates two string references into a new string.	8.217#implementation/unit_test/cat ch.hpp	Loc #9534 - 9540
CS-1.217#implementat ion/unit_test/catch.hpp	auto operator + (StringRef const& lhs, const char* rhs) -> std::string {	Combines a string reference and a C-style string into a new standard string.	8.217#implementation/unit_test/cat ch.hpp	Loc #9541 - 9543
CS-1.217#implementat ion/unit_test/catch.hpp	auto operator + (char const* lhs, StringRef const& rhs) -> std::string {	Defines addition of C-style string and StringRef, returning a new concatenated string.	8.217#implementation/unit_test/cat ch.hpp	Loc #9544 - 9546
CS-1.217#implementat ion/unit_test/catch.hpp	auto operator << (std::ostream& os, StringRef const& str) -> std::ostream& {	Allows printing a StringRef object directly to standard output streams using the << operator.	8.217#implementation/unit_test/cat ch.hpp	Loc #9548 - 9550
CS-1.218#implementat ion/unit_test/catch.hpp	namespace Catch {	Initializes a TagAlias object with a tag string and source line information.	8.218#implementation/unit_test/cat ch.hpp	Loc #9560 - 9562
CS-1.219#implementat ion/unit_test/catch.hpp	TestCaseInfo::TestCaseInfo(std::string const& _name,	Initializes a test case information object with name, class, description, source line, and tags.	8.219#implementation/unit_test/cat ch.hpp	Loc #9718 - 9730

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.219#implementation/unit_test/catch.hpp	bool TestCaseInfo::isHidden() const {	Checks if the test case is marked as hidden based on its properties.	8.219#implementation/unit_test/catch.hpp	Loc #9732 - 9734
CS-1.219#implementation/unit_test/catch.hpp	bool TestCaseInfo::throws() const {	Checks if test case properties indicate it should throw.	8.219#implementation/unit_test/catch.hpp	Loc #9735 - 9737
CS-1.219#implementation/unit_test/catch.hpp	bool TestCaseInfo::okToFail() const {	Checks if test case properties allow failure based on 'ShouldFail' or 'MayFail' flags.	8.219#implementation/unit_test/catch.hpp	Loc #9738 - 9740
CS-1.219#implementation/unit_test/catch.hpp	bool TestCaseInfo::expectedToFail() const {	Checks if a test case is marked as expected to fail.	8.219#implementation/unit_test/catch.hpp	Loc #9741 - 9743
CS-1.219#implementation/unit_test/catch.hpp	std::string TestCaseInfo::tagsAsString() const {	Converts a list of tags into a single string, enclosing each tag within square brackets for easy display.	8.219#implementation/unit_test/catch.hpp	Loc #9745 - 9760
CS-1.220#implementation/unit_test/catch.hpp	std::vector<TestCase> sortTests(IConfig const& config, std::vector<TestCase> const& unsortedTestCases) {	Sorts a list of test cases based on a configuration specifying lexicographical, random, or declaration order.	8.220#implementation/unit_test/catch.hpp	Loc #9797 - 9814
CS-1.221#implementation/unit_test/catch.hpp	void setTags(TestCaseInfo& testCaseInfo, std::vector<std::string> tags) {	Sorts, removes duplicates, converts tags to lowercase, parses special properties, and updates test case info.	8.221#implementation/unit_test/catch.hpp	Loc #9705 - 9716
CS-1.222#implementation/unit_test/catch.hpp	IndexTracker::IndexTracker(NameAndLocation const& nameAndLocation, TrackerContext& ctx, ITracker* parent, int size)	Creates an IndexTracker object, initializing its base class with name, location, context, and parent, and storing the provided size value.	8.222#implementation/unit_test/catch.hpp	Loc #10106 - 10109
CS-1.222#implementation/unit_test/catch.hpp	bool IndexTracker::isIndexTracker() const { return true; }	Finds or creates an IndexTracker instance within a tracking hierarchy.	8.222#implementation/unit_test/catch.hpp	Loc #10106 - 10149
CS-1.223#implementation/unit_test/catch.hpp	SectionTracker::SectionTracker(NameAndLocation const& nameAndLocation, TrackerContext& ctx, ITracker* parent)	Creates a section tracker, inheriting filters from the nearest section tracker ancestor if a parent is provided.	8.223#implementation/unit_test/catch.hpp	Loc #10057 - 10067
CS-1.223#implementation/unit_test/catch.hpp	bool SectionTracker::isSectionTracker() const { return true; }	Manages tracking sections within a hierarchical structure, finding or creating them and applying filters.	8.223#implementation/unit_test/catch.hpp	Loc #10057 - 10104

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.224#implementation/unit_test/catch.hpp	std::string trim(std::string const& str) {	Removes leading and trailing whitespace characters (space, tab, newline) from a string.	8.224#implementation/unit_test/catch.hpp	N/A
CS-1.225#implementation/unit_test/catch.hpp	TestInvokerAsFunction::TestInvokerAsFunction(void(*testAsFunction)()) noexcept : m_testAsFunction(testAsFunction) {}	Invokes a test function passed during construction.	8.225#implementation/unit_test/catch.hpp	Loc #9866 - 9871
CS-1.226#implementation/unit_test/catch.hpp	void ReusableStringStream::cleanup() {	Calls a cleanup function from a related class to release resources.	8.226#implementation/unit_test/catch.hpp	Loc #9346 - 9348
CS-1.226#implementation/unit_test/catch.hpp	ReusableStringStream::ReusableStringStream()	Initializes object by getting a reusable string stream from a global pool.	8.226#implementation/unit_test/catch.hpp	Loc #9350 - 9353
CS-1.226#implementation/unit_test/catch.hpp	ReusableStringStream::~~ReusableStringStream() {	Resets a string stream and returns it to a pool for reuse.	8.226#implementation/unit_test/catch.hpp	Loc #9355 - 9359
CS-1.226#implementation/unit_test/catch.hpp	auto ReusableStringStream::str() const -> std::string {	Gets the accumulated string content from the internal output string stream.	8.226#implementation/unit_test/catch.hpp	Loc #9361 - 9363
CS-1.227#implementation/unit_test/catch.hpp	Totals Totals::operator - (Totals const& other) const {	Calculates the difference between two Totals objects by subtracting corresponding member values.	8.227#implementation/unit_test/catch.hpp	Loc #10634 - 10639
CS-1.227#implementation/unit_test/catch.hpp	Totals& Totals::operator += (Totals const& other) {	Adds counts from another Totals object to the current object.	8.227#implementation/unit_test/catch.hpp	Loc #10641 - 10645
CS-1.227#implementation/unit_test/catch.hpp	Totals Totals::delta(Totals const& prevTotals) const {	Calculates the difference between current and previous totals, determining the outcome of a single test case based on assertion changes.	8.227#implementation/unit_test/catch.hpp	Loc #10647 - 10656
CS-1.228#implementation/unit_test/catch.hpp	TestEventListenerBase::TestEventListenerBase(ReporterConfig const & _config)	Initializes the test event listener with a reporter configuration.	8.228#implementation/unit_test/catch.hpp	Loc #10955 - 10956
CS-1.228#implementation/unit_test/catch.hpp	void TestEventListenerBase::assertionStarting(AssertionInfo const &) {}	Default methods for a test event listener that perform no action on assertion events.	8.228#implementation/unit_test/catch.hpp	Loc #10955 - 10962
CS-1.229#implementation/unit_test/catch.hpp	std::string rawMemoryToString(const void *object, std::size_t size) {	Converts raw memory bytes to a hexadecimal string, handling byte order for different system architectures.	8.229#implementation/unit_test/catch.hpp	Loc #10418 - 10432

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<std::string>::convert(const std::string& str) {	Converts a string to a quoted string, potentially escaping invisible characters like newline or tab.	8.230#implementation/unit_test/catch.hpp	Loc #10461 - 10482
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<std::wstring>::convert(const std::wstring& wstr) {	Converts a wide string to a narrow string, replacing multi-byte characters with '?'.	8.230#implementation/unit_test/catch.hpp	Loc #10484 - 10491
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<char const*>::convert(char const* str) {	Converts a C-style string pointer to std::string, returning "{null string}" for null pointers.	8.230#implementation/unit_test/catch.hpp	Loc #10493 - 10499
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<char*>::convert(char* str) {	Converts a C-style 'char*' string to a C++ 'std::string', returning '{null string}' if the input is null.	8.230#implementation/unit_test/catch.hpp	Loc #10500 - 10506
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<wchar_t const*>::convert(wchar_t const* str) {	Converts a wide character string pointer to std::string, handling null.	8.230#implementation/unit_test/catch.hpp	Loc #10507 - 10513
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<wchar_t*>::convert(wchar_t* str) {	Converts a wide character pointer to a standard string representation, handling null pointers.	8.230#implementation/unit_test/catch.hpp	Loc #10514 - 10520
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<int>::convert(int value) {	Converts an integer value into its string representation.	8.230#implementation/unit_test/catch.hpp	Loc #10522 - 10524
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<long>::convert(long value) {	Converts a long integer value into its string representation for use in a testing framework.	8.230#implementation/unit_test/catch.hpp	Loc #10525 - 10527
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<long long>::convert(long long value) {	Converts a long long value to string, adding hex representation if large.	8.230#implementation/unit_test/catch.hpp	Loc #10528 - 10535
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<unsigned int>::convert(unsigned int value) {	Converts an unsigned integer into its string representation using a utility function.	8.230#implementation/unit_test/catch.hpp	Loc #10537 - 10539
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<unsigned long>::convert(unsigned long value) {	Converts an unsigned long number into its string representation.	8.230#implementation/unit_test/catch.hpp	Loc #10540 - 10542
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<unsigned long long>::convert(unsigned long long value) {	Convert unsigned long long to string, including hex representation if value exceeds a threshold.	8.230#implementation/unit_test/catch.hpp	Loc #10543 - 10550

ID	Feature Name	Description	Feature ID	Code Lines
	long value) {			
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<bool>::convert(bool b) {	Converts a boolean value into its string representation, returning 'true' for true and 'false' for false.	8.230#implementation/unit_test/cat ch.hpp	Loc #10552 - 10554
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<char>::convert(char value) {	Converts a character into a string representation, handling special characters and non-printable control characters.	8.230#implementation/unit_test/cat ch.hpp	Loc #10556 - 10572
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<signed char>::convert(signed char c) {	Converts a signed character value into its string representation using an internal helper.	8.230#implementation/unit_test/cat ch.hpp	Loc #10573 - 10575
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<unsigned char>::convert(unsigned char c) {	Converts an unsigned character to its string representation using a specific helper function.	8.230#implementation/unit_test/cat ch.hpp	Loc #10576 - 10578
CS-1.230#implementation/unit_test/catch.hpp	std::string StringMaker<std::nullptr_t>::convert(std::nullptr_t) {	Converts the null pointer type std::nullptr_t into the string "nullptr".	8.230#implementation/unit_test/cat ch.hpp	Loc #10580 - 10582
CS-1.231#implementation/unit_test/catch.hpp	std::string ratio_string<std::atto>::symbol() { return "a"; }	Provides standard string symbols for various C++ standard ratio types.	8.231#implementation/unit_test/cat ch.hpp	Loc #10591 - 10596
CS-1.232#implementation/unit_test/catch.hpp	XmlEncode::XmlEncode(std::string const& str, ForWhat forWhat)	Initializes an XML string encoder with the string content and its usage context.	8.232#implementation/unit_test/cat ch.hpp	Loc #10745 - 10748
CS-1.232#implementation/unit_test/catch.hpp	std::ostream& operator << (std::ostream& os, XmlEncode const& xmlEncode) {	Allows writing XmlEncode objects to output streams easily.	8.232#implementation/unit_test/cat ch.hpp	Loc #10790 - 10793
CS-1.233#implementation/unit_test/catch.hpp	auto estimateClockResolution() -> uint64_t {	Estimates the smallest time difference detectable by the system clock by averaging many measurements.	8.233#implementation/unit_test/cat ch.hpp	Loc #10334 - 10354
CS-1.233#implementation/unit_test/catch.hpp	auto getEstimatedClockResolution() -> uint64_t {	Retrieves a cached estimate of the system clock's resolution, calculating it only once.	8.233#implementation/unit_test/cat ch.hpp	Loc #10355 - 10358
CS-1.234#implementation/unit_test/catch.hpp	AssertionPrinter(std::ostream& _stream, AssertionStats const& _stats, bool _printInfoMessages)	Initializes AssertionPrinter with stream, stats, and message print flag.	8.234#implementation/unit_test/cat ch.hpp	Loc #11033 - 11038
CS-1.234#implementation/unit_test/catch.hpp	std::ostream& stream;	Stores data and messages for reporting a code	8.234#implementation/unit_test/cat	Loc #11187 -

ID	Feature Name	Description	Feature ID	Code Lines
ion/unit_test/catch.hpp		assertion result.	ch.hpp	11193
CS-1.235#implementat ion/unit_test/catch.hpp	std::string getFormattedDuration(double duration) {	Formats a double-precision number into a string with exactly three decimal places for display.	8.235#implementation/unit_test/cat ch.hpp	Loc #10937 - 10953
CS-1.236#implementat ion/unit_test/catch.hpp	std::string fpToString(T value, int precision) {	Converts a floating-point number to a string with specified precision, handles NaN, and removes trailing zeros.	8.236#implementation/unit_test/cat ch.hpp	Loc #10436 - 10453
CS-1.236#implementat ion/unit_test/catch.hpp	std::string StringMaker<float>::convert(float value) {	Converts a float value to a string with 5 decimal places and appends 'f'.	8.236#implementation/unit_test/cat ch.hpp	Loc #10584 - 10586
CS-1.236#implementat ion/unit_test/catch.hpp	std::string StringMaker<double>::co nvert(double value) {	Converts a double-precision floating-point number to its string representation.	8.236#implementation/unit_test/cat ch.hpp	Loc #10587 - 10589
CS-1.237#implementat ion/unit_test/catch.hpp	auto getCurrentNanosecondsSince Epoch() -> uint64_t {	Gets current time from high-resolution clock, calculates nanoseconds since epoch, returns count.	8.237#implementation/unit_test/cat ch.hpp	Loc #10330 - 10332
CS-1.238#implementat ion/unit_test/catch.hpp	Counts Counts::operator - (Counts const& other) const {	Subtracts counts from another Counts object, returning the difference.	8.238#implementation/unit_test/cat ch.hpp	Loc #10609 - 10615
CS-1.238#implementat ion/unit_test/catch.hpp	Counts& Counts::operator += (Counts const& other) {	Merges counts from another object into the current one by adding member values.	8.238#implementation/unit_test/cat ch.hpp	Loc #10617 - 10622
CS-1.238#implementat ion/unit_test/catch.hpp	std::size_t Counts::total() const {	Calculates the sum of passed, failed, and failed but ok counts.	8.238#implementation/unit_test/cat ch.hpp	Loc #10624 - 10626
CS-1.238#implementat ion/unit_test/catch.hpp	bool Counts::allPassed() const {	Checks if both 'failed' and 'failedButOk' counts are zero.	8.238#implementation/unit_test/cat ch.hpp	Loc #10627 - 10629
CS-1.238#implementat ion/unit_test/catch.hpp	bool Counts::allOk() const {	Checks if the internal failure counter is zero, indicating if any errors have occurred.	8.238#implementation/unit_test/cat ch.hpp	Loc #10630 - 10632
CS-1.239#implementat ion/unit_test/catch.hpp	WildcardPattern::WildcardPattern(std::string const& pattern,	Creates a pattern object for matching strings, handling case sensitivity and removing leading/trailing wildcards.	8.239#implementation/unit_test/cat ch.hpp	Loc #10704 - 10717
CS-1.239#implementat ion/unit_test/catch.hpp	bool WildcardPattern::matches(std::string const& str) const {	Checks if an input string matches a pattern based on wildcard rules.	8.239#implementation/unit_test/cat ch.hpp	Loc #10719 - 10732

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.239#implementation/unit_test/catch.hpp	std::string WildcardPattern::adjustCase(std::string const& str) const {	Returns the input string in lowercase if case-insensitivity is enabled, otherwise returns the original string.	8.239#implementation/unit_test/catch.hpp	Loc #10734 - 10736
CS-1.240#implementation/unit_test/catch.hpp	void Timer::start() {	Starts the timer by recording the current system time in nanoseconds for later calculation of elapsed time.	8.240#implementation/unit_test/catch.hpp	Loc #10360 - 10362
CS-1.240#implementation/unit_test/catch.hpp	auto Timer::getElapsedNanoseconds() const -> uint64_t {	Calculates and returns the time elapsed in nanoseconds since the timer was set.	8.240#implementation/unit_test/catch.hpp	Loc #10363 - 10365
CS-1.240#implementation/unit_test/catch.hpp	auto Timer::getElapsedMicroseconds() const -> uint64_t {	Calculates elapsed time since timer start in microseconds.	8.240#implementation/unit_test/catch.hpp	Loc #10366 - 10368
CS-1.240#implementation/unit_test/catch.hpp	auto Timer::getElapsedMilliseconds() const -> unsigned int {	Calculates and returns the elapsed time in milliseconds using the time measured in microseconds.	8.240#implementation/unit_test/catch.hpp	Loc #10369 - 10371
CS-1.240#implementation/unit_test/catch.hpp	auto Timer::getElapsedSeconds() const -> double {	Calculates and returns elapsed time in seconds from microseconds.	8.240#implementation/unit_test/catch.hpp	Loc #10372 - 10374
CS-1.241#implementation/unit_test/catch.hpp	void TestSpecParser::visitChar(char c) {	Parses input characters sequentially to build test specification patterns and filters based on current parsing mode.	8.241#implementation/unit_test/catch.hpp	Loc #10266 - 10298
CS-1.241#implementation/unit_test/catch.hpp	void TestSpecParser::startNewMode(Mode mode, std::size_t start) {	Sets the parser's current operational mode and starting position.	8.241#implementation/unit_test/catch.hpp	Loc #10299 - 10302
CS-1.241#implementation/unit_test/catch.hpp	void TestSpecParser::escape() {	Updates parser state to handle escaped characters, records the position, and marks start if in initial mode.	8.241#implementation/unit_test/catch.hpp	Loc #10303 - 10308
CS-1.241#implementation/unit_test/catch.hpp	std::string TestSpecParser::subString() const { return m_arg.substr(m_start, m_pos - m_start); }	Parses a string containing test specifications and filters into a TestSpec object using an internal parser.	8.241#implementation/unit_test/catch.hpp	Loc #10266 - 10320
CS-1.242#implementation/unit_test/catch.hpp	void prepareExpandedExpression(A	Calls the getExpandedExpression method on an	8.242#implementation/unit_test/catch.hpp	Loc #10931 -

ID	Feature Name	Description	Feature ID	Code Lines
ion/unit_test/catch.hpp	ssertionResult& result) {	AssertionResult object.	ch.hpp	10933
CS-1.243#implementat ion/unit_test/catch.hpp	void printTotals(std::ostream& out, const Totals& totals) {	Prints a summary of test case and assertion results to a given output stream.	8.243#implementation/unit_test/cat ch.hpp	Loc #10996 - 11026
CS-1.244#implementat ion/unit_test/catch.hpp	Version::Version	Creates a Version object initializing its components like major, minor, patch, branch, and build number.	8.244#implementation/unit_test/cat ch.hpp	Loc #10666 - 10677
CS-1.244#implementat ion/unit_test/catch.hpp	std::ostream& operator << (std::ostream& os, Version const& version) {	Formats and prints Version object data to an output stream.	8.244#implementation/unit_test/cat ch.hpp	Loc #10679 - 10689
CS-1.244#implementat ion/unit_test/catch.hpp	Version const& libraryVersion() {	Provides a constant reference to the library's compiled-in version number (2.1.0).	8.244#implementation/unit_test/cat ch.hpp	Loc #10691 - 10694
CS-1.245#implementat ion/unit_test/catch.hpp	XmlWriter::ScopedElement::Scoped Element(XmlWriter* writer)	Constructs a ScopedElement linking it to a specific XmlWriter object.	8.245#implementation/unit_test/cat ch.hpp	Loc #10795 - 10797
CS-1.245#implementat ion/unit_test/catch.hpp	XmlWriter::ScopedElement::Scoped Element(ScopedElement&& other) noexcept	Creates object by moving resource ownership from another ScopedElement.	8.245#implementation/unit_test/cat ch.hpp	Loc #10799 - 10802
CS-1.245#implementat ion/unit_test/catch.hpp	XmlWriter::ScopedElement& XmlWr iter::ScopedElement::operator=(ScopedElement&& other) noexcept {	Handles moving XML element management from one object to another, closing the previous element if needed.	8.245#implementation/unit_test/cat ch.hpp	Loc #10803 - 10810
CS-1.245#implementat ion/unit_test/catch.hpp	XmlWriter::ScopedElement::~~Scope dElement() {	Automatically writes the closing XML tag when the element's scope ends.	8.245#implementation/unit_test/cat ch.hpp	Loc #10812 - 10815
CS-1.245#implementat ion/unit_test/catch.hpp	XmlWriter::ScopedElement& XmlWr iter::ScopedElement::writeText(std::string const& text, bool indent) {	Writes text content within the current XML element using an internal writer.	8.245#implementation/unit_test/cat ch.hpp	Loc #10817 - 10820
CS-1.245#implementat ion/unit_test/catch.hpp	XmlWriter::XmlWriter(std::ostream& os) : m_os(os)	This constructor initializes an XML writer connected to a stream and writes the standard XML declaration.	8.245#implementation/unit_test/cat ch.hpp	Loc #10822 - 10825
CS-1.245#implementat	XmlWriter::~~XmlWriter() {	Destructor ensures all opened XML tags are	8.245#implementation/unit_test/cat	Loc #10827 -

ID	Feature Name	Description	Feature ID	Code Lines
ion/unit_test/catch.hpp		closed before the object is destroyed.	ch.hpp	10830
CS-1.245#implementat ion/unit_test/catch.hpp	XmlWriter& XmlWriter::startElement(std::string const& name) {	Starts writing an opening XML element with the provided name, managing nesting indentation and tracking open tags for proper structure.	8.245#implementation/unit_test/cat ch.hpp	Loc #10832 - 10840
CS-1.245#implementat ion/unit_test/catch.hpp	XmlWriter::ScopedElement XmlWriter::scopedElement(std::string const& name) {	Creates an object to manage XML element's start and end tags (RAII).	8.245#implementation/unit_test/cat ch.hpp	Loc #10842 - 10846
CS-1.245#implementat ion/unit_test/catch.hpp	XmlWriter& XmlWriter::endElement() {	Closes the current XML element, managing indentation and outputting either a self-closing tag or a separate closing tag.	8.245#implementation/unit_test/cat ch.hpp	Loc #10848 - 10861
CS-1.245#implementat ion/unit_test/catch.hpp	XmlWriter& XmlWriter::writeAttribute(std::string const& name, std::string const& attribute) {	Writes an XML attribute (name="value") to the stream if both are not empty.	8.245#implementation/unit_test/cat ch.hpp	Loc #10863 - 10867
CS-1.245#implementat ion/unit_test/catch.hpp	XmlWriter& XmlWriter::writeAttribute(std::string const& name, bool attribute) {	Writes a boolean attribute (name="value") to the current XML element, returning the writer object for chaining.	8.245#implementation/unit_test/cat ch.hpp	Loc #10869 - 10872
CS-1.245#implementat ion/unit_test/catch.hpp	XmlWriter& XmlWriter::writeText(std::string const& text, bool indent) {	Writes text content into an XML document, handling encoding and indentation.	8.245#implementation/unit_test/cat ch.hpp	Loc #10874 - 10884
CS-1.245#implementat ion/unit_test/catch.hpp	XmlWriter& XmlWriter::writeComment(std::string const& text) {	Writes an XML comment with given text to the output stream, ensuring correct format.	8.245#implementation/unit_test/cat ch.hpp	Loc #10886 - 10891
CS-1.245#implementat ion/unit_test/catch.hpp	void XmlWriter::writeStylesheetRef(std::string const& url) {	Writes an XML processing instruction to link the document to an XSLT stylesheet located at a specified URL.	8.245#implementation/unit_test/cat ch.hpp	Loc #10893 - 10895
CS-1.245#implementat ion/unit_test/catch.hpp	XmlWriter& XmlWriter::writeBlankLine() {	Adds a blank line (newline character) to the output stream, ensuring any open XML tag is closed before writing.	8.245#implementation/unit_test/cat ch.hpp	Loc #10897 - 10901
CS-1.245#implementat ion/unit_test/catch.hpp	void XmlWriter::ensureTagClosed() {	Ensures the current XML tag is closed by adding '>' to the output stream if it is currently open.	8.245#implementation/unit_test/cat ch.hpp	Loc #10903 - 10908

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.245#implementation/unit_test/catch.hpp	void XmlWriter::writeDeclaration() {	Writes the standard XML declaration line (version 1.0, UTF-8) to an internal output stream for XML file creation.	8.245#implementation/unit_test/catch.hpp	Loc #10910 - 10912
CS-1.245#implementation/unit_test/catch.hpp	void XmlWriter::newlineIfNecessary() {	Writes a newline character to the output stream only if a flag is set, then clears the flag.	8.245#implementation/unit_test/catch.hpp	Loc #10914 - 10919
CS-1.246#implementation/unit_test/catch.hpp	ReporterPreferences CompactReporter::getPreferences() const {	Returns default preferences for a compact reporter, disabling standard output redirection.	8.246#implementation/unit_test/catch.hpp	Loc #11201 - 11205
CS-1.247#implementation/unit_test/catch.hpp	std::string CompactReporter::getDescription() {	Returns a string describing this test reporter format.	8.247#implementation/unit_test/catch.hpp	Loc #11197 - 11199
CS-1.248#implementation/unit_test/catch.hpp	class ConsoleAssertionPrinter {	Formats and prints test assertion results, including success/failure status, source, expressions, and messages, to an output stream.	8.248#implementation/unit_test/catch.hpp	Loc #11267 - 11404
CS-1.249#implementation/unit_test/catch.hpp	class Duration {	Represents a time duration storing value in nanoseconds, capable of displaying in various time units.	8.249#implementation/unit_test/catch.hpp	Loc #11429 - 11499
CS-1.250#implementation/unit_test/catch.hpp	class TablePrinter {	Formats and prints data into a table structure to an output stream using provided column definitions and formatting operators.	8.250#implementation/unit_test/catch.hpp	Loc #11502 - 11572
CS-1.251#implementation/unit_test/catch.hpp	void CompactReporter::testRunEnded(TestRunStats const& _testRunStats) {	Reports test run results by printing totals and calling the base class end-of-run handler.	8.251#implementation/unit_test/catch.hpp	Loc #11238 - 11242
CS-1.252#implementation/unit_test/catch.hpp	bool CompactReporter::assertionEnded(AssertionStats const& _assertionStats) {	Handles assertion result, prints details unless successful results are skipped by config.	8.252#implementation/unit_test/catch.hpp	Loc #11213 - 11230
CS-1.253#implementation/unit_test/catch.hpp	CATCH_REGISTER_REPORTER("compact", CompactReporter)	Registers a custom test reporter named 'compact' with Catch2 using CompactReporter.	8.253#implementation/unit_test/catch.hpp	Loc #11246 - 11246

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.254#implementation/unit_test/catch.hpp	void CompactReporter::noMatchingTestCases(std::string const& spec) {	Reports to an output stream that no test cases matched a given specification pattern.	8.254#implementation/unit_test/catch.hpp	Loc #11207 - 11209
CS-1.255#implementation/unit_test/catch.hpp	void CompactReporter::sectionEnded(SectionStats const& _sectionStats) {	Reports a section's duration and name if the configuration is set to always show durations.	8.255#implementation/unit_test/catch.hpp	Loc #11232 - 11236
CS-1.256#implementation/unit_test/catch.hpp	void MultipleReporters::add(IStreamingReporterPtr&& reporter) {	Adds a streaming reporter object to an internal list using move semantics.	8.256#implementation/unit_test/catch.hpp	Loc #12107 - 12109
CS-1.257#implementation/unit_test/catch.hpp	ConsoleReporter::ConsoleReporter(ReporterConfig const& config)	Initializes a console reporter to display benchmark results in a structured table format using a configured stream.	8.257#implementation/unit_test/catch.hpp	Loc #11574 - 11582
CS-1.258#implementation/unit_test/catch.hpp	JUnitReporter::JUnitReporter(ReporterConfig const& _config)	Initializes the JUnit reporter, setting up XML output and enabling standard output redirection based on configuration.	8.258#implementation/unit_test/catch.hpp	Loc #11914 - 11919
CS-1.259#implementation/unit_test/catch.hpp	XmlReporter::XmlReporter(ReporterConfig const& _config)	Initializes an XML reporter with configuration, setting up base class and output stream.	8.259#implementation/unit_test/catch.hpp	Loc #12207 - 12212
CS-1.260#implementation/unit_test/catch.hpp	ConsoleReporter::~~ConsoleReporter() = default;	Defines the default destructor for the ConsoleReporter class, allowing the compiler to generate automatic cleanup logic.	8.260#implementation/unit_test/catch.hpp	Loc #11583 - 11583
CS-1.261#implementation/unit_test/catch.hpp	JUnitReporter::~~JUnitReporter() {};	This is the default destructor for the JUnitReporter class, performing no custom cleanup.	8.261#implementation/unit_test/catch.hpp	Loc #11921 - 11921
CS-1.262#implementation/unit_test/catch.hpp	XmlReporter::~~XmlReporter() = default;	Defines the default destructor for the XmlReporter class.	8.262#implementation/unit_test/catch.hpp	Loc #12214 - 12214
CS-1.263#implementation/unit_test/catch.hpp	std::string fileNameTag(const std::vector<std::string> &tags) {	Finds the first string starting with '#' in a list and returns content after '#', or empty.	8.263#implementation/unit_test/catch.hpp	Loc #11904 - 11911
CS-1.264#implementation/unit_test/catch.hpp	void ConsoleReporter::printHeaderString(std::string const& _string, std::size_t indent) {	Formats and prints a header string to a stream with specific indentation.	8.264#implementation/unit_test/catch.hpp	Loc #11754 - 11761

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.265#implementation/unit_test/catch.hpp	bool MultipleReporters::assertionEnded(AssertionStats const& assertionStats) {	Notifies multiple reporters that an assertion ended, combining their buffer clear requests.	8.265#implementation/unit_test/catch.hpp	Loc #12159 - 12164
CS-1.266#implementation/unit_test/catch.hpp	void MultipleReporters::assertionStarting(AssertionInfo const& assertionInfo) {	Notifies multiple registered reporters that a test assertion is about to begin, allowing each to react.	8.266#implementation/unit_test/catch.hpp	Loc #12153 - 12156
CS-1.267#implementation/unit_test/catch.hpp	void MultipleReporters::benchmarkEnded(BenchmarkStats const& benchmarkStats) {	Notifies all registered reporters that a benchmark has finished with provided statistics.	8.267#implementation/unit_test/catch.hpp	Loc #12128 - 12131
CS-1.268#implementation/unit_test/catch.hpp	void MultipleReporters::benchmarkStarting(BenchmarkInfo const& benchmarkInfo) {	Notifies all registered reporters that a benchmark is about to start.	8.268#implementation/unit_test/catch.hpp	Loc #12124 - 12127
CS-1.269#implementation/unit_test/catch.hpp	void MultipleReporters::noMatchingTestCases(std::string const& spec) {	Notify multiple reporters about no matching test cases using a given specification string.	8.269#implementation/unit_test/catch.hpp	Loc #12119 - 12122
CS-1.270#implementation/unit_test/catch.hpp	void MultipleReporters::sectionEnded(SectionStats const& sectionStats) {	Notifies all registered reporter objects that a section has finished processing, providing statistics.	8.270#implementation/unit_test/catch.hpp	Loc #12166 - 12169
CS-1.271#implementation/unit_test/catch.hpp	void MultipleReporters::sectionStarting(SectionInfo const& sectionInfo) {	Notifies multiple registered reporter objects that a new section is starting.	8.271#implementation/unit_test/catch.hpp	Loc #12148 - 12151
CS-1.272#implementation/unit_test/catch.hpp	void MultipleReporters::skipTest(TestCaseInfo const& testInfo) {	This function iterates through a collection of reporter objects and calls the skipTest method on each one, effectively notifying all reporters that a test was skipped.	8.272#implementation/unit_test/catch.hpp	Loc #12186 - 12189
CS-1.273#implementation/unit_test/catch.hpp	void MultipleReporters::testCaseEnded(TestCaseStats const& testCaseStats) {	Notifies all registered reporters that a test case has finished, passing case statistics.	8.273#implementation/unit_test/catch.hpp	Loc #12171 - 12174
CS-1.274#implementation/unit_test/catch.hpp	void MultipleReporters::testCaseStarting(TestCaseInfo const& testInfo)	Notifies all registered reporters that a test case is starting, passing relevant information to each.	8.274#implementation/unit_test/catch.hpp	Loc #12143 - 12146

ID	Feature Name	Description	Feature ID	Code Lines
	{			
CS-1.275#implementation/unit_test/catch.hpp	void MultipleReporters::testGroupEnded(TestGroupStats const& testGroupStats) {	Notifies multiple configured reporters that a test group has ended by calling a method on each.	8.275#implementation/unit_test/catch.hpp	Loc #12176 - 12179
CS-1.276#implementation/unit_test/catch.hpp	void MultipleReporters::testGroupStarting(GroupInfo const& groupInfo) {	Calls the 'testGroupStarting' method on all registered reporter objects.	8.276#implementation/unit_test/catch.hpp	Loc #12138 - 12141
CS-1.277#implementation/unit_test/catch.hpp	void MultipleReporters::testRunEnded(TestRunStats const& testRunStats) {	Notifies all registered test reporters that a test run has completed.	8.277#implementation/unit_test/catch.hpp	Loc #12181 - 12184
CS-1.278#implementation/unit_test/catch.hpp	void MultipleReporters::testRunStarting(TestRunInfo const& testRunInfo) {	Notifies multiple test reporters that a test run is starting.	8.278#implementation/unit_test/catch.hpp	Loc #12133 - 12136
CS-1.279#implementation/unit_test/catch.hpp	std::string getCurrentTimestamp() {	Get current UTC time, format as string YYYY-MM-DDTHH:MM:SSZ, return string.	8.279#implementation/unit_test/catch.hpp	Loc #11878 - 11902
CS-1.280#implementation/unit_test/catch.hpp	ReporterPreferences MultipleReporters::getPreferences() const {	Retrieves configuration preferences from the first reporter in a collection of reporters.	8.280#implementation/unit_test/catch.hpp	Loc #12111 - 12113
CS-1.281#implementation/unit_test/catch.hpp	std::string XmlReporter::getStylesheetRef() const {	Returns an empty string for the stylesheet reference of an XML report.	8.281#implementation/unit_test/catch.hpp	Loc #12220 - 12222
CS-1.282#implementation/unit_test/catch.hpp	std::set<Verbosity> MultipleReporters::getSupportedVerbosities() {	Provides the set of supported message verbosity levels.	8.282#implementation/unit_test/catch.hpp	Loc #12115 - 12117
CS-1.283#implementation/unit_test/catch.hpp	bool JunitReporter::assertionEnded(AssertionStats const& assertionStats) {	Checks assertion result for unexpected exceptions and updates count.	8.283#implementation/unit_test/catch.hpp	Loc #11946 - 11950
CS-1.284#implementation/unit_test/catch.hpp	void ConsoleReporter::assertionStarting(AssertionInfo const&) {}	Handles the start of an assertion evaluation, performing no action.	8.284#implementation/unit_test/catch.hpp	Loc #11593 - 11593

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.285#implementation/unit_test/catch.hpp	void XmlReporter::assertionStarting(AssertionInfo const&) { }	Defines an empty function called when an assertion starts.	8.285#implementation/unit_test/catch.hpp	Loc #12275 - 12275
CS-1.286#implementation/unit_test/catch.hpp	namespace Catch {	Initializes a static memory leak detector object within the Catch namespace for potential use in testing.	8.286#implementation/unit_test/catch.hpp	Loc #12410 - 12412
CS-1.287#implementation/unit_test/catch.hpp	void JunitReporter::noMatchingTestCases(std::string const& /*spec*/) {}	This function is called when no test cases match a specification but performs no action.	8.287#implementation/unit_test/catch.hpp	Loc #11927 - 11927
CS-1.288#implementation/unit_test/catch.hpp	void XmlReporter::noMatchingTestCases(std::string const& s) {	Notifies report listener when no matching test cases found.	8.288#implementation/unit_test/catch.hpp	Loc #12230 - 12232
CS-1.289#implementation/unit_test/catch.hpp	void ConsoleReporter::sectionStarting(SectionInfo const& _sectionInfo) {	Prepares the reporter for a new test section by resetting a header flag and calling the base class handler.	8.289#implementation/unit_test/catch.hpp	Loc #11612 - 11615
CS-1.290#implementation/unit_test/catch.hpp	void JunitReporter::testCaseEnded(TestCaseStats const& testCaseStats) {	Processes a finished test case by accumulating its output/error and passing stats to the base class.	8.290#implementation/unit_test/catch.hpp	Loc #11952 - 11956
CS-1.291#implementation/unit_test/catch.hpp	void JunitReporter::testCaseStarting(TestCaseInfo const& testCaseInfo) {	Records if the currently starting test case is marked as acceptable to fail during execution.	8.291#implementation/unit_test/catch.hpp	Loc #11942 - 11944
CS-1.292#implementation/unit_test/catch.hpp	bool MultipleReporters::isMulti() const {	This method checks if the object represents multiple reporters by always returning true.	8.292#implementation/unit_test/catch.hpp	Loc #12191 - 12193
CS-1.293#implementation/unit_test/catch.hpp	void ConsoleReporter::lazyPrint() {	Closes the internal table printer then performs a lazy print operation.	8.293#implementation/unit_test/catch.hpp	Loc #11680 - 11684
CS-1.294#implementation/unit_test/catch.hpp	void ConsoleReporter::lazyPrintWithOutClosingBenchmarkTable() {	Prints test run/group info and header lazily if not already used/printed.	8.294#implementation/unit_test/catch.hpp	Loc #11686 - 11697
CS-1.295#implementation/unit_test/catch.hpp	void ConsoleReporter::printClosedHeader(std::string const& _name) {	Prints a header with a given name, followed by a line of dots for closing.	8.295#implementation/unit_test/catch.hpp	Loc #11740 - 11743
CS-1.296#implementation/unit_test/catch.hpp	void ConsoleReporter::printOpenHeader(std::string const& _name) {	Prints a formatted header to the console stream	8.296#implementation/unit_test/catch.hpp	Loc #11744 - 11747

ID	Feature Name	Description	Feature ID	Code Lines
ion/unit_test/catch.hpp	ader(std::string const& _name) {	with a separator line and specific color.	ch.hpp	11750
CS-1.297#implementat ion/unit_test/catch.hpp	void ConsoleReporter::printSummaryDivider() {	Prints a line of hyphens followed by a newline character to an output stream for separation.	8.297#implementation/unit_test/cat ch.hpp	Loc #11856 - 11858
CS-1.298#implementat ion/unit_test/catch.hpp	void ConsoleReporter::printSummaryRow(std::string const& label, std::vector<SummaryColumn> const& cols, std::size_t row) {	Formats and prints a single row of summary data to a console stream, handling zero values and applying labels/colors.	8.298#implementation/unit_test/cat ch.hpp	Loc #11817 - 11833
CS-1.299#implementat ion/unit_test/catch.hpp	void ConsoleReporter::printTestCaseAndSectionHeader() {	Prints a formatted header to the console, showing the current test case name, any nested section names, and the source file location of the current section.	8.299#implementation/unit_test/cat ch.hpp	Loc #11716 - 11738
CS-1.300#implementat ion/unit_test/catch.hpp	void ConsoleReporter::lazyPrintGroupInfo() {	Prints group name header if group info is available and multiple groups exist.	8.300#implementation/unit_test/cat ch.hpp	Loc #11710 - 11715
CS-1.301#implementat ion/unit_test/catch.hpp	void ConsoleReporter::lazyPrintRunInfo() {	Prints introductory test run details like application name, library version, and random seed to an output stream.	8.301#implementation/unit_test/cat ch.hpp	Loc #11698 - 11709
CS-1.302#implementat ion/unit_test/catch.hpp	void ConsoleReporter::printTotals(Totals const& totals) {	Prints a summary of test results including counts for test cases and assertions by status.	8.302#implementation/unit_test/cat ch.hpp	N/A
CS-1.303#implementat ion/unit_test/catch.hpp	void ConsoleReporter::printTotalsDivider(Totals const& totals) {	Prints a colored dividing line based on test case results.	8.303#implementation/unit_test/cat ch.hpp	Loc #11835 - 11855
CS-1.304#implementat ion/unit_test/catch.hpp	std::string JunitReporter::getDescription() {	Reports test results in an XML format that mimics Ant's junitreport target for compatibility.	8.304#implementation/unit_test/cat ch.hpp	Loc #11923 - 11925
CS-1.305#implementat ion/unit_test/catch.hpp	int main (int argc, char * const argv[]) {	Sets up and runs automated tests using the Catch2 framework, handling Objective-C memory management if required.	8.305#implementation/unit_test/cat ch.hpp	Loc #12440 - 12453
CS-1.306#implementat ion/unit_test/catch.hpp	std::string ConsoleReporter::getDescription() {	Function returns a fixed string describing its reporting style.	8.306#implementation/unit_test/cat ch.hpp	Loc #11585 - 11587
CS-1.307#implementat ion/unit_test/catch.hpp	extern "C" int wmain (int argc, wchar_t * argv[], wchar_t * []) {	Initializes and runs a Catch2 test session using command-line arguments.	8.307#implementation/unit_test/cat ch.hpp	Loc #12428 - 12435
CS-1.308#implementat	std::string	Provides a string describing the reporter's	8.308#implementation/unit_test/cat	Loc #12216 -

ID	Feature Name	Description	Feature ID	Code Lines
ion/unit_test/catch.hpp	XmlReporter::getDescription() {	purpose.	ch.hpp	12218
CS-1.309#implementat ion/unit_test/catch.hpp	CATCH_REGISTER_REPORTER("console", ConsoleReporter)	Registers the built-in console test reporter with Catch2.	8.309#implementation/unit_test/cat ch.hpp	Loc #11860 - 11860
CS-1.310#implementat ion/unit_test/catch.hpp	CATCH_REGISTER_REPORTER("junit", JunitReporter)	Registers a test reporter class named 'junit' with the Catch2 framework, making it available for test reporting.	8.310#implementation/unit_test/cat ch.hpp	Loc #12099 - 12099
CS-1.311#implementat ion/unit_test/catch.hpp	CATCH_REGISTER_REPORTER("xml", XmlReporter)	Registers the XmlReporter class in the CATCH framework, allowing it to be selected by the name "xml" for test reporting.	8.311#implementation/unit_test/cat ch.hpp	Loc #12401 - 12401
CS-1.312#implementat ion/unit_test/catch.hpp	bool ConsoleReporter::assertionEnded(AssertionStats const& _assertionStats) {	Reports a test assertion result, printing it to the console stream based on success/failure and configuration.	8.312#implementation/unit_test/cat ch.hpp	Loc #11595 - 11610
CS-1.313#implementat ion/unit_test/catch.hpp	bool XmlReporter::assertionEnded(AssertionStats const& assertionStats) {	Reports test assertion results in XML format, including details based on success/failure and configuration.	8.313#implementation/unit_test/cat ch.hpp	Loc #12277 - 12349
CS-1.314#implementat ion/unit_test/catch.hpp	void ConsoleReporter::benchmarkEnded(BenchmarkStats const& stats) {	Calculates average time per benchmark iteration and prints results to a table.	8.314#implementation/unit_test/cat ch.hpp	Loc #11651 - 11657
CS-1.315#implementat ion/unit_test/catch.hpp	void ConsoleReporter::benchmarkStarting(BenchmarkInfo const& info) {	Formats and prints the benchmark name when a benchmark starts.	8.315#implementation/unit_test/cat ch.hpp	Loc #11636 - 11650
CS-1.316#implementat ion/unit_test/catch.hpp	void ConsoleReporter::noMatchingTestCases(std::string const& spec) {	Reports a message to the console stream indicating that no test cases matched the specified pattern string.	8.316#implementation/unit_test/cat ch.hpp	Loc #11589 - 11591
CS-1.317#implementat ion/unit_test/catch.hpp	void ConsoleReporter::sectionEnded(SectionStats const& _sectionStats) {	Reports statistics when a test section ends, including warnings for missing assertions and printing duration based on configuration.	8.317#implementation/unit_test/cat ch.hpp	Loc #11616 - 11634
CS-1.318#implementat ion/unit_test/catch.hpp	void XmlReporter::sectionEnded(SectionStats const& sectionStats) {	Handles the end of a test section, writing summary statistics to the XML report if it's a nested section.	8.318#implementation/unit_test/cat ch.hpp	Loc #12351 - 12364
CS-1.319#implementat	void XmlReporter::sectionStarting(	Reports the start of a nested section in XML	8.319#implementation/unit_test/cat	Loc #12264 -

ID	Feature Name	Description	Feature ID	Code Lines
ion/unit_test/catch.hpp	SectionInfo const& sectionInfo) {	format during reporting.	ch.hpp	12273
CS-1.320#implementat ion/unit_test/catch.hpp	void ConsoleReporter::testCaseEnd ed(TestCaseStats const& _testCaseStats) {	Completes processing for a test case, closing resources and resetting state.	8.320#implementation/unit_test/cat ch.hpp	Loc #11659 - 11663
CS-1.321#implementat ion/unit_test/catch.hpp	void XmlReporter::testCaseEnded(TestCaseStats const& testCaseStats) {	Writes test case results including success, duration, stdout, and stderr into an XML report structure.	8.321#implementation/unit_test/cat ch.hpp	Loc #12366 - 12380
CS-1.322#implementat ion/unit_test/catch.hpp	void XmlReporter::testCaseStarting(TestCaseInfo const& testInfo) {	Records information about a test case starting for XML reporting and starts a timer if needed.	8.322#implementation/unit_test/cat ch.hpp	Loc #12250 - 12262
CS-1.323#implementat ion/unit_test/catch.hpp	void XmlReporter::testGroupEnded(TestGroupStats const& testGroupStats) {	Processes test group statistics and writes XML element with counts of passed, failed, and expected failed assertions.	8.323#implementation/unit_test/cat ch.hpp	Loc #12382 - 12390
CS-1.324#implementat ion/unit_test/catch.hpp	void ConsoleReporter::testGroupEn ded(TestGroupStats const& _testGroupStats) {	Reports test group summary statistics to console stream.	8.324#implementation/unit_test/cat ch.hpp	Loc #11664 - 11672
CS-1.325#implementat ion/unit_test/catch.hpp	void JUnitReporter::testGroupEnded(TestGroupStats const& testGroupStats) {	Signals a test group finished, calculates suite time, and writes group results.	8.325#implementation/unit_test/cat ch.hpp	Loc #11958 - 11962
CS-1.326#implementat ion/unit_test/catch.hpp	void JUnitReporter::testGroupStarting(GroupInfo const& groupInfo) {	Initializes state for a new test group, starting a timer and clearing buffers for capturing output.	8.326#implementation/unit_test/cat ch.hpp	Loc #11934 - 11940
CS-1.327#implementat ion/unit_test/catch.hpp	void XmlReporter::testGroupStarting(GroupInfo const& groupInfo) {	Handles test group start event by calling base method and creating XML element.	8.327#implementation/unit_test/cat ch.hpp	Loc #12244 - 12248
CS-1.328#implementat ion/unit_test/catch.hpp	void JUnitReporter::testRunEndedC umulative() {	Closes the XML element representing the cumulative test run in the JUnit report.	8.328#implementation/unit_test/cat ch.hpp	Loc #11964 - 11966
CS-1.329#implementat ion/unit_test/catch.hpp	void XmlReporter::testRunEnded(TestRunStats const& testRunStats) {	Reports the final test run statistics in XML format.	8.329#implementation/unit_test/cat ch.hpp	Loc #12392 - 12399

ID	Feature Name	Description	Feature ID	Code Lines
CS-1.330#implementation/unit_test/catch.hpp	void ConsoleReporter::testRunEnded(TestRunStats const& _testRunStats) {	Reports the end of a test run by printing summary statistics and totals to a stream.	8.330#implementation/unit_test/catch.hpp	Loc #11673 - 11678
CS-1.331#implementation/unit_test/catch.hpp	void JunitReporter::testRunStarting(TestRunInfo const& runInfo) {	Handles the start of a test run by calling base class method and writing the start tag for XML test suites.	8.331#implementation/unit_test/catch.hpp	Loc #11929 - 11932
CS-1.332#implementation/unit_test/catch.hpp	void XmlReporter::testRunStarting(TestRunInfo const& testInfo) {	Initializes XML reporting for a test run, including base setup and root element.	8.332#implementation/unit_test/catch.hpp	Loc #12234 - 12242
CS-1.333#implementation/unit_test/catch.hpp	void JunitReporter::writeAssertions(SectionNode const& sectionNode) {	Iterates through assertions in a section node and writes each assertion using a helper method.	8.333#implementation/unit_test/catch.hpp	Loc #12047 - 12050
CS-1.334#implementation/unit_test/catch.hpp	void JunitReporter::writeAssertion(AssertionStats const& stats) {	Writes details of a failed test assertion to an XML report, categorizing it as error or failure.	8.334#implementation/unit_test/catch.hpp	Loc #12052 - 12097
CS-1.335#implementation/unit_test/catch.hpp	void JunitReporter::writeGroup(TestGroupNode const& groupNode, double suiteTime) {	Formats and writes test group statistics and cases into a JUnit XML testsuite element.	8.335#implementation/unit_test/catch.hpp	Loc #11968 - 11988
CS-1.336#implementation/unit_test/catch.hpp	void JunitReporter::writeSection(std::string const& className,	Writes a test section and its contents (assertions, output, timing) as a JUnit XML testcase, handling nested sections recursively.	8.336#implementation/unit_test/catch.hpp	Loc #12012 - 12045
CS-1.337#implementation/unit_test/catch.hpp	void XmlReporter::writeSourceInfo(SourceLineInfo const& sourceInfo) {	Writes source file name and line number as XML attributes using an internal XML writer.	8.337#implementation/unit_test/catch.hpp	Loc #12224 - 12228
CS-1.338#implementation/unit_test/catch.hpp	void JunitReporter::writeTestCase(TestCaseNode const& testCaseNode) {	Processes a test case node, determines its class name, and prepares it for reporting by calling writeSection.	8.338#implementation/unit_test/catch.hpp	Loc #11990 - 12010
CS-2.1#implementation/x86/rs232.h	int RS232_IsCTSEnabled(int);	Checks if the Clear To Send (CTS) signal is enabled for a specified RS232 serial port.	7.1#implementation/x86/rs232.h	Loc #70 - 70
CS-2.2#implementation/x86/rs232.h	int RS232_IsDCDEnabled(int);	Checks if the Data Carrier Detect (DCD) signal is currently enabled for a specific RS232 port.	7.2#implementation/x86/rs232.h	Loc #69 - 69
CS-2.3#implementation	int RS232_IsDSREnabled(int);	Checks if the DSR line is active for a specific	7.3#implementation/x86/rs232.h	Loc #71 - 71

ID	Feature Name	Description	Feature ID	Code Lines
n/x86/rs232.h		RS232 communication port.		
CS-2.4#implementation/x86/rs232.h	void RS232_clearBreak(int);	Clears the break condition on an RS232 communication channel.	7.4#implementation/x86/rs232.h	Loc #81 - 81
CS-2.5#implementation/x86/rs232.h	void RS232_CloseComport(int);	Declares a function to close a previously opened communication port.	7.5#implementation/x86/rs232.h	N/A
CS-2.6#implementation/x86/rs232.h	#ifndef rs232_INCLUDED	Prevents multiple inclusions of this header file during compilation.	7.6#implementation/x86/rs232.h	Loc #33 - 34
CS-2.7#implementation/x86/rs232.h	void RS232_disableDTR(int);	Disables the Data Terminal Ready (DTR) signal on an RS232 serial port.	7.7#implementation/x86/rs232.h	N/A
CS-2.8#implementation/x86/rs232.h	void RS232_disableRTS(int);	Disables the Request To Send (RTS) signal for an RS232 communication port.	7.8#implementation/x86/rs232.h	N/A
CS-2.9#implementation/x86/rs232.h	void RS232_enabledDTR(int);	Function declaration to enable Data Terminal Ready (DTR) on an RS232 port.	7.9#implementation/x86/rs232.h	Loc #72 - 72
CS-2.10#implementation/x86/rs232.h	void RS232_enableRTS(int);	Declares a function to enable the RS232 RTS line for a specified port.	7.10#implementation/x86/rs232.h	Loc #74 - 74
CS-2.11#implementation/x86/rs232.h	#endif	Marks the end of a block of code that is conditionally compiled based on preprocessor directives.	7.11#implementation/x86/rs232.h	Loc #38 - 38
CS-2.12#implementation/x86/rs232.h	#ifdef __cplusplus	Ends a C-style linkage block in header files, active only when compiled by a C++ compiler for C compatibility.	7.12#implementation/x86/rs232.h	Loc #86 - 88
CS-2.13#implementation/x86/rs232.h	void RS232_flushRXTX(int);	Clears both receive and transmit buffers for a specified RS232 port.	7.13#implementation/x86/rs232.h	N/A
CS-2.14#implementation/x86/rs232.h	void RS232_flushRX(int);	Declares a function to clear the receive buffer of a specific RS232 port.	7.14#implementation/x86/rs232.h	Loc #76 - 76
CS-2.15#implementation/x86/rs232.h	void RS232_flushTX(int);	Declares function to clear/flush RS232 transmit buffer.	7.15#implementation/x86/rs232.h	Loc #77 - 77
CS-2.16#implementation/x86/rs232.h	int RS232_GetPortnr(const char *);	Gets the integer number for a given RS232 port name.	7.16#implementation/x86/rs232.h	Loc #79 - 79

ID	Feature Name	Description	Feature ID	Code Lines
CS-2.17#implementation/x86/rs232.h	#if defined(__linux__) defined(__FreeBSD__)	Includes platform-specific system header files based on the operating system (Linux/FreeBSD or Windows) to ensure compatibility for system-level operations.	7.17#implementation/x86/rs232.h	Loc #45 - 61
CS-2.18#implementation/x86/rs232.h	#include <stdio.h>	Includes standard input/output and string manipulation libraries.	7.18#implementation/x86/rs232.h	Loc #40 - 41
CS-2.19#implementation/x86/rs232.h	int RS232_OpenComport(int, int, const char *);	Opens a specified serial communication port for RS232 communication.	7.19#implementation/x86/rs232.h	Loc #63 - 63
CS-2.20#implementation/x86/rs232.h	int RS232_PollComport(int, unsigned char *, int);	Reads available data from a specified RS232 serial port into a given buffer.	7.20#implementation/x86/rs232.h	Loc #64 - 64
CS-2.21#implementation/x86/rs232.h	int RS232_PollComportEx(int, unsigned char *, int, int *);	Reads data from a serial port into a buffer up to a maximum size.	7.21#implementation/x86/rs232.h	Loc #80 - 80
CS-2.22#implementation/x86/rs232.h	void RS232_cputs(int, const char *);	Declares a function to send a null-terminated string through a specified RS232 communication channel.	7.22#implementation/x86/rs232.h	Loc #68 - 68
CS-2.23#implementation/x86/rs232.h	int RS232_SendBuf(int, unsigned char *, int);	Sends a block of data from a buffer over an RS232 serial port.	7.23#implementation/x86/rs232.h	N/A
CS-2.24#implementation/x86/rs232.h	int RS232_SendByte(int, unsigned char);	Sends a single byte over a specified RS232 serial port connection.	7.24#implementation/x86/rs232.h	Loc #65 - 65
CS-2.25#implementation/x86/rs232.h	void RS232_setBreak(int);	Declares a function to set the break condition state for an RS232 serial interface using an integer value.	7.25#implementation/x86/rs232.h	Loc #82 - 82
CS-2.26#implementation/x86/rs232.h	extern "C" {	Incomplete code snippet, cannot determine purpose.	7.26#implementation/x86/rs232.h	Loc #36 - 38
CS-2.27#implementation/x86/rs232.c	int RS232_IsCTSEnabled(int comport_number)	Checks if the Clear To Send (CTS) signal is enabled for a serial port.	7.27#implementation/x86/rs232.c	Loc #371 - 379
CS-2.28#implementation/x86/rs232.c	int RS232_IsDCDEnabled(int comport_number)	Checks if the Data Carrier Detect (DCD) signal is active for a specified serial port.	7.28#implementation/x86/rs232.c	Loc #360 - 368
CS-2.29#implementation/x86/rs232.c	int RS232_IsDSREnabled(int comport_number)	Checks if the Data Set Ready (DSR) line is active for a specified serial port.	7.29#implementation/x86/rs232.c	Loc #382 - 390

ID	Feature Name	Description	Feature ID	Code Lines
on/x86/rs232.c	comport_number)	enabled for a specific serial port using ioctl and returns 1 if enabled, 0 otherwise.		
CS-2.30#implementati on/x86/rs232.c	void RS232_clearBreak(int comport_number)	Clears the break signal on a specific RS232 serial port.	7.30#implementation/x86/rs232.c	Loc #825 - 829
CS-2.31#implementati on/x86/rs232.c	void RS232_CloseComport(int comport_number)	Closes a serial port by controlling handshake lines, restoring settings, releasing lock, and closing.	7.31#implementation/x86/rs232.c	Loc #320 - 341
CS-2.32#implementati on/x86/rs232.c	void RS232_disableDTR(int comport_number)	This function disables the Data Terminal Ready (DTR) signal line for a specified serial communication port.	7.32#implementation/x86/rs232.c	Loc #411 - 426
CS-2.33#implementati on/x86/rs232.c	void RS232_disableRTS(int comport_number)	Disables the Request To Send (RTS) signal for a specified serial communication port.	7.33#implementation/x86/rs232.c	Loc #447 - 462
CS-2.34#implementati on/x86/rs232.c	void RS232_enabledDTR(int comport_number)	Enables the Data Terminal Ready (DTR) signal on a specified serial port by reading and setting its status.	7.34#implementation/x86/rs232.c	Loc #393 - 408
CS-2.35#implementati on/x86/rs232.c	void RS232_enableRTS(int comport_number)	Enables the Ready To Send (RTS) control line for a specified serial communication port.	7.35#implementation/x86/rs232.c	Loc #429 - 444
CS-2.36#implementati on/x86/rs232.c	void RS232_flushRXTX(int comport_number)	Flushes (discards) data in both receive and transmit buffers for a specified serial port.	7.36#implementation/x86/rs232.c	Loc #477 - 480
CS-2.37#implementati on/x86/rs232.c	void RS232_flushRX(int comport_number)	Clears the incoming data buffer for a specified RS232 serial port.	7.37#implementation/x86/rs232.c	Loc #465 - 468
CS-2.38#implementati on/x86/rs232.c	void RS232_flushTX(int comport_number)	Clears the outgoing data buffer for a specific serial port.	7.38#implementation/x86/rs232.c	Loc #471 - 474
CS-2.39#implementati on/x86/rs232.c	int i;	Declares an integer variable and a character array for storing text.	7.39#implementation/x86/rs232.c	Loc #890 - 892
CS-2.39#implementati on/x86/rs232.c	#if defined(__linux__) defined(__FreeBSD__) /* Linux & FreeBSD */	Initializes a string prefix based on the detected operating system (Linux/FreeBSD vs. others).	7.39#implementation/x86/rs232.c	Loc #894 - 896
CS-2.39#implementati on/x86/rs232.c	strncat(str, devname, 16);	Searches a list of comport names for a match after appending a device name to a base string.	7.39#implementation/x86/rs232.c	Loc #899 - 908

ID	Feature Name	Description	Feature ID	Code Lines
CS-2.40#implementation/x86/rs232.c	int RS232_OpenComport(int comport_number, int baudrate, const char *mode)	Opens a serial port with specified communication speed, data bits, parity, and stop bits for RS232 communication.	7.40#implementation/x86/rs232.c	Loc #57 - 264
CS-2.41#implementation/x86/rs232.c	int RS232_PollComport(int comport_number, unsigned char *buf, int size)	Reads data from a serial port non-blockingly into a buffer.	7.41#implementation/x86/rs232.c	Loc #267 - 279
CS-2.42#implementation/x86/rs232.c	int RS232_PollComportEx(int comport_number, unsigned char *buf, int size, int *out_break){	Polls a serial port for data or break signals using event-driven I/O with a timeout.	7.42#implementation/x86/rs232.c	Loc #754 - 823
CS-2.43#implementation/x86/rs232.c	int RS232_SendBuf(int comport_number, unsigned char *buf, int size)	This function sends a specified number of bytes from a data buffer to a particular RS232 serial port connection using the system's write call.	7.43#implementation/x86/rs232.c	Loc #301 - 317
CS-2.44#implementation/x86/rs232.c	void RS232_cputs(int comport_number, const char *text) /* sends a string to serial port */	Send a string character by character over a specified serial port.	7.44#implementation/x86/rs232.c	Loc #881 - 884
CS-2.45#implementation/x86/rs232.c	int RS232_SendByte(int comport_number, unsigned char byte)	Sends a single byte of data over a specified RS232 serial port connection.	7.45#implementation/x86/rs232.c	Loc #282 - 298
CS-2.46#implementation/x86/rs232.c	void RS232_setBreak(int comport_number)	Sets the break state on a specific RS232 serial port, suspending transmission.	7.46#implementation/x86/rs232.c	Loc #747 - 752
CS-3.1#open_lin_data_layer.h	extern open_lin_checksum_t open_lin_data_layer_checksum(open_lin_pid_t pid, l_u8 lenght, const l_u8* data_ptr);	Calculates the checksum for a LIN data layer message.	6.1#open_lin_data_layer.h	Loc #25 - 25
CS-3.2#open_lin_data_layer.h	extern open_lin_pid_t open_lin_data_layer_parity(open_lin_pid_t pid);	Calculates or verifies parity bits for a LIN Protocol Identifier (PID).	6.2#open_lin_data_layer.h	Loc #24 - 24
CS-3.3#open_lin_data_layer.h	/**	This structure defines a LIN communication frame, holding its identifier, data size, data pointer, and checksum.	6.3#open_lin_data_layer.h	Loc #14 - 22
CS-3.4#open_lin_data_layer.c	open_lin_checksum_t open_lin_data_layer_checksum(open_lin_pid_t	Calculates the LIN checksum for a message using PID, length, and data bytes, following LIN	6.4#open_lin_data_layer.c	Loc #22 - 47

ID	Feature Name	Description	Feature ID	Code Lines
	pid, l_u8 lenght, const l_u8* data_ptr)	specification rules.		
CS-3.5#open_lin_data_layer.c	open_lin_pid_t open_lin_data_layer_parity(open_lin_pid_t pid) {	Calculates and adds LIN data layer parity bits (P0, P1) to an input Packet Identifier (PID).	6.5#open_lin_data_layer.c	Loc #10 - 20
CS-3.6#open_lin_hw.h	/**	Checks the UART hardware to determine if a communication break signal condition has been detected.	6.6#open_lin_hw.h	Loc #13 - 18
CS-3.7#open_lin_hw.h	void open_lin_hw_init(void);	Initializes the necessary hardware components for an open LIN communication system.	6.7#open_lin_hw.h	Loc #29 - 29
CS-3.8#open_lin_hw.h	/**	Resets hardware to its default state for Open LIN communication.	6.8#open_lin_hw.h	Loc #20 - 25
CS-3.9#open_lin_hw.h	void open_lin_set_rx_enabled(l_bool status);	Controls the status of LIN reception, enabling or disabling it based on the input status.	6.9#open_lin_hw.h	Loc #27 - 27
CS-3.10#open_lin_hw.h	#ifdef OPEN_LIN_AUTO_BAUND	Starts the automatic baud rate detection procedure for LIN hardware.	6.10#open_lin_hw.h	Loc #37 - 44
CS-3.11#open_lin_hw.h	l_bool open_lin_hw_tx_break(void);	Initiates or enables the hardware transmission of a LIN break signal for communication.	6.11#open_lin_hw.h	Loc #33 - 33
CS-3.12#open_lin_hw.h	l_bool open_lin_hw_tx_byte(l_u8 data);	Sends a single byte using the LIN hardware interface and indicates if the transmission was successful.	6.12#open_lin_hw.h	Loc #35 - 35
CS-3.13#open_lin_hw.h	l_bool open_lin_hw_tx_data(l_u8* data, l_u8 len);	Initiates transmission of a specified length of data bytes over the LIN hardware.	6.13#open_lin_hw.h	Loc #31 - 31
CS-3.14#open_lin_network_layer.c	};	No code provided for analysis.	6.14#open_lin_network_layer.c	Loc #66 - 69
CS-3.15#open_lin_network_layer.c	void open_lin_net_init(open_lin_frame_slot_t *a_slot_array, l_u8 a_slot_array_len)	Sets up LIN network communication by storing frame slot information and enabling reception.	6.15#open_lin_network_layer.c	Loc #15 - 20
CS-3.16#open_lin_network_layer.c	open_lin_frame_slot_t* open_lin_net_get_slot(open_lin_pid_t pid)	Searches for a frame slot entry matching the given identifier, optionally translating the ID.	6.16#open_lin_network_layer.c	Loc #26 - 64

ID	Feature Name	Description	Feature ID	Code Lines
CS-3.17#open_lin_master_data_layer.h	typedef struct {	Defines configuration for one LIN frame slot in a master schedule table.	6.17#open_lin_master_data_layer.h	Loc #13 - 17
CS-3.18#open_lin_master_data_layer.h	l_bool open_lin_master_dl_rx(l_u8 rx_byte);	Processes a received byte within the LIN master's data link layer communication.	6.18#open_lin_master_data_layer.h	Loc #20 - 20
CS-3.19#open_lin_master_data_layer.h	void open_lin_master_dl_init(t_master_frame_table_item *p_master_frame_table, l_u8 p_master_frame_table_size);	Initializes the LIN master data link layer using a provided frame table and its size.	6.19#open_lin_master_data_layer.h	Loc #19 - 19
CS-3.20#open_lin_master_data_layer.h	void open_lin_master_dl_rx_callback(open_lin_frame_slot_t *slot);	Processes a received LIN frame slot in the LIN master's data link layer.	6.20#open_lin_master_data_layer.h	Loc #22 - 22
CS-3.21#open_lin_master_data_layer.h	void open_lin_master_dl_handler(l_u8 ms_passed);	Handles timing and events for the LIN master data link layer.	6.21#open_lin_master_data_layer.h	Loc #21 - 21
CS-3.22#open_lin_master_data_layer.h	l_bool open_lin_master_data_tx_data(open_lin_frame_slot_t *slot);	Prepares data for transmission for a specific LIN frame slot.	6.22#open_lin_master_data_layer.h	Loc #26 - 26
CS-3.23#open_lin_master_data_layer.h	/* this functions by default are handled by open_lin_mastr_dl_handler */	Handles sending the LIN frame header as a master.	6.23#open_lin_master_data_layer.h	Loc #24 - 25
CS-3.24#open_lin_transport_layer.h	void open_lin_transport_layer_handle(open_lin_frame_slot_t *slot);	Handle LIN transport layer tasks for a frame slot, managing data transmission and reception according to the slot's identifier.	6.24#open_lin_transport_layer.h	Loc #56 - 56
CS-3.25#open_lin_transport_layer.h	typedef l_u8 open_lin_NAD_t;	Define new type names for unsigned 8-bit numbers used in LIN communication.	6.25#open_lin_transport_layer.h	Loc #13 - 14
CS-3.26#open_lin_transport_layer.h	typedef enum {	Defines integer constants representing different types of LIN Protocol Control Information.	6.26#open_lin_transport_layer.h	Loc #30 - 34
CS-3.27#open_lin_transport_layer.h	typedef struct {	Defines a data structure containing two 4-bit fields: 'additional' and 'PCI_type', likely used for Protocol Control Information in the Open LIN protocol.	6.27#open_lin_transport_layer.h	Loc #25 - 28
CS-3.28#open_lin_transport_layer.h	typedef struct {	Defines a structure to represent a single-frame	6.28#open_lin_transport_layer.h	Loc #42 - 47

ID	Feature Name	Description	Feature ID	Code Lines
sport_layer.h		LIN message request containing address, control info, service ID, and data.		
CS-3.29#open_lin_transport_layer.h	typedef struct {	Defines a structure to hold data for an Open LIN single frame response.	6.29#open_lin_transport_layer.h	Loc #49 - 54
CS-3.30#open_lin_transport_layer.h	typedef enum {	Defines types of LIN node addresses using clear names for specific values (Sleep, Functional, Broadcast).	6.30#open_lin_transport_layer.h	Loc #36 - 40
CS-3.31#open_lin_transport_layer.h	typedef enum {	Defines standard LIN (Local Interconnect Network) service identifiers (SIDs) for various diagnostic operations.	6.31#open_lin_transport_layer.h	Loc #16 - 23
CS-3.32#open_lin_transport_layer.h	#ifdef OPEN_LIN_TRANSPORT_LAYER	Checks if a specific macro is defined to conditionally compile code.	6.32#open_lin_transport_layer.h	Loc #11 - 11
CS-3.33#open_lin_network_layer.h	void open_lin_on_rx_frame(open_lin_frame_slot_t *slot);	Processes a received LIN communication frame using the provided slot information.	6.33#open_lin_network_layer.h	Loc #32 - 32
CS-3.34#open_lin_network_layer.h	open_lin_frame_slot_t* open_lin_network_layer_get_slot(open_lin_pid_t pid);	Retrieves a LIN frame slot structure pointer based on its Protocol ID (PID).	6.34#open_lin_network_layer.h	Loc #29 - 29
CS-3.35#open_lin_network_layer.h	void open_lin_net_rx_handler(open_lin_pid_t pid);	Handles reception of a LIN message identified by its Protocol ID.	6.35#open_lin_network_layer.h	Loc #30 - 30
CS-3.36#open_lin_network_layer.h	void open_lin_net_init(open_lin_frame_slot_t *a_slot_array, l_u8 a_slot_array_len);	Initializes the LIN network module with frame slot timing configurations.	6.36#open_lin_network_layer.h	Loc #31 - 31
CS-3.37#open_lin_network_layer.h	#ifdef OPEN_LIN_DYNAMIC_IDS	Defines a structure to store pairs of LIN message IDs, typically for translation or mapping purposes, enabled conditionally.	6.37#open_lin_network_layer.h	Loc #22 - 27
CS-3.38#open_lin_network_layer.h	typedef struct {	Defines a structure to hold details for a single Open LIN frame slot.	6.38#open_lin_network_layer.h	Loc #15 - 20
CS-3.39#open_lin_types.h	/**	Handles errors within the open_lin system by processing a specific error code.	6.39#open_lin_types.h	N/A
CS-3.40#open_lin_types.h	typedef bool l_bool;	Defines custom type names (aliases) for boolean, 8-bit, and 16-bit unsigned integer types.	6.40#open_lin_types.h	Loc #16 - 18

ID	Feature Name	Description	Feature ID	Code Lines
CS-3.41#open_lin_types.h	#define I_true true	Defines custom aliases 'I_true' and 'I_false' for standard boolean literals.	6.41#open_lin_types.h	Loc #20 - 21
CS-3.42#open_lin_types.h	#define OPEN_LIN_DIAG_REQUEST 0x3C	Defines constant numerical values for LIN diagnostic request and response message identifiers.	6.42#open_lin_types.h	Loc #30 - 31
CS-3.43#open_lin_types.h	typedef enum {	Defines a list of possible error codes for an Open LIN communication system, covering master and slave issues.	6.43#open_lin_types.h	Loc #43 - 57
CS-3.44#open_lin_types.h	#define OPEN_LIN_MAX_FRAME_LEN 8u	Defines constants for maximum LIN frame length and checksum size.	6.44#open_lin_types.h	Loc #13 - 14
CS-3.44#open_lin_types.h	#define OPEN_LIN_MAX_FRAME_LENGTH 0x8u	Defines constants and a macro for LIN protocol frame processing and identification.	6.44#open_lin_types.h	Loc #23 - 28
CS-3.45#open_lin_types.h	typedef enum {	Defines types for LIN communication frames: transmitting or receiving.	6.45#open_lin_types.h	Loc #38 - 41
CS-3.46#open_lin_types.h	typedef I_u8 open_lin_checksum_t;	Creates a custom type name for an 8-bit unsigned integer used for checksums.	6.46#open_lin_types.h	Loc #36 - 37
CS-3.47#open_lin_types.h	#define open_lin_memcpy memcpy	Creates a macro alias 'open_lin_memcpy' for the standard C 'memcpy' function.	6.47#open_lin_types.h	Loc #34 - 34
CS-3.48#open_lin_slave_data_layer.c	if (open_lin_slave_state != OPEN_LIN_SLAVE_IDLE)	Checks LIN slave state on break signal reception, reports error if not IDLE.	6.48#open_lin_slave_data_layer.c	Loc #59 - 64
CS-3.48#open_lin_slave_data_layer.c	#ifdef OPEN_LIN_HW_BREAK_IS_SYNCH_BYTE	Sets initial LIN slave state based on config and enables auto-baud if defined.	6.48#open_lin_slave_data_layer.c	Loc #57 - 73
CS-3.49#open_lin_slave_data_layer.c	static I_bool open_lin_slave_set_lin_frame(t_open_lin_data_layer_frame *frame){	Finds configuration for a LIN frame by ID, sets frame properties (length, data pointer), and updates the slave's operational state (transmit/receive).	6.49#open_lin_slave_data_layer.c	Loc #20 - 40
CS-3.50#open_lin_slave_data_layer.c	open_lin_slave_reset();	Resets the LIN slave peripheral and reports an error for receiving invalid data.	6.50#open_lin_slave_data_layer.c	Loc #148 - 151
CS-3.51#open_lin_slave_data_layer.c	void open_lin_slave_init(void)	Initializes the LIN slave module by performing a reset operation.	6.51#open_lin_slave_data_layer.c	Loc #42 - 45

ID	Feature Name	Description	Feature ID	Code Lines
CS-3.52#open_lin_slave_data_layer.c	void open_lin_slave_rx_header(l_u8 rx_byte)	Processes incoming LIN bytes (break, sync, PID) for slave state management and potential data transmission.	6.52#open_lin_slave_data_layer.c	Loc #52 - 118
CS-3.52#open_lin_slave_data_layer.c	break;	An empty case within a switch statement intended for handling received LIN slave data.	6.52#open_lin_slave_data_layer.c	Loc #123 - 126
CS-3.52#open_lin_slave_data_layer.c	if (open_lin_slave_state_data_count < open_lin_data_layer_frame.lenght)	Checks if the number of data bytes processed is less than the total frame length.	6.52#open_lin_slave_data_layer.c	Loc #127 - 128
CS-3.52#open_lin_slave_data_layer.c	open_lin_slave_state_data_count ++;	Increments a counter variable as part of a conditional block of code.	6.52#open_lin_slave_data_layer.c	Loc #130 - 132
CS-3.52#open_lin_slave_data_layer.c	/* checksum calculation */	Compares a received byte with a checksum calculated from a data frame's identifier to validate data integrity.	6.52#open_lin_slave_data_layer.c	Loc #133 - 134
CS-3.52#open_lin_slave_data_layer.c	}	Processes LIN slave data reception substates and byte counts, resetting the slave or signaling errors.	6.52#open_lin_slave_data_layer.c	Loc #118 - 154
CS-3.53#open_lin_slave_data_layer.c	{	Processes a received byte, validates LIN ID parity, sets up the frame, and transmits data if the slave is in the data transmission state.	6.53#open_lin_slave_data_layer.c	Loc #91 - 118
CS-3.54#open_lin_slave_data_layer.c	/* synch byte reception do nothing */	Checks if the received byte is the LIN synchronization byte to validate communication start.	6.54#open_lin_slave_data_layer.c	Loc #78 - 83
CS-3.54#open_lin_slave_data_layer.c	break;	Exits the nearest enclosing loop or switch statement immediately.	6.54#open_lin_slave_data_layer.c	Loc #86 - 88
CS-3.55#open_lin_slave_data_layer.c	if (open_lin_slave_state_data_count < open_lin_data_layer_frame.lenght)	Checks if the current count of processed slave data is less than the total expected length of the data layer frame.	6.55#open_lin_slave_data_layer.c	Loc #127 - 128
CS-3.55#open_lin_slave_data_layer.c	open_lin_slave_state_data_count ++;	Increments a counter variable tracking state data under a specific condition.	6.55#open_lin_slave_data_layer.c	Loc #130 - 132
CS-3.55#open_lin_slave_data_layer.c	/* checksum calculation */	Compares a received byte with the calculated	6.55#open_lin_slave_data_layer.c	Loc #133 - 134

ID	Feature Name	Description	Feature ID	Code Lines
e_data_layer.c		checksum of a LIN frame for data integrity verification.		
CS-3.55#open_lin_slave_data_layer.c	}	Resets the LIN slave using a dedicated function call.	6.55#open_lin_slave_data_layer.c	Loc #125 - 147
CS-3.56#open_lin_slave_data_layer.c	void open_lin_slave_reset(void){	Resets the state and data counter of a LIN slave component.	6.56#open_lin_slave_data_layer.c	Loc #47 - 50
CS-3.57#open_lin_slave_data_layer.c	break;	Exits the current loop or switch statement immediately.	6.57#open_lin_slave_data_layer.c	Loc #103 - 117
CS-3.58#open_lin_slave_data_layer.h	typedef enum {	Defines the possible operational states for a LIN slave device.	6.58#open_lin_slave_data_layer.h	Loc #10 - 16
CS-3.59#open_lin_slave_data_layer.h	/**	Declare function to handle single received byte for a LIN slave.	6.59#open_lin_slave_data_layer.h	Loc #28 - 32
CS-3.60#open_lin_slave_data_layer.h	/**	Initializes the LIN slave component, preparing it for communication on the LIN bus.	6.60#open_lin_slave_data_layer.h	Loc #18 - 21
CS-3.61#open_lin_slave_data_layer.h	/**	Resets the LIN slave component to its initial state.	6.61#open_lin_slave_data_layer.h	Loc #23 - 26
CS-3.62#open_lin_master_data_layer.c	t_master_frame_table_item* master_table_item = get_current_item();	Gets current item pointer and checks if the master table size is greater than zero.	6.62#open_lin_master_data_layer.c	Loc #123 - 126
CS-3.62#open_lin_master_data_layer.c	time_passed_since_last_frame_ms += ms_passed;	Updates elapsed time and checks if the LIN master communication state is idle.	6.62#open_lin_master_data_layer.c	Loc #127 - 128
CS-3.62#open_lin_master_data_layer.c	if ((master_table_item->offset_ms) < time_passed_since_last_frame_ms)	Resets elapsed time if a frame offset is smaller than the time passed since the last frame.	6.62#open_lin_master_data_layer.c	Loc #130 - 132
CS-3.62#open_lin_master_data_layer.c	if (open_lin_master_data_tx_header (&master_table_item->slot) == !_true)	Calls a function to open a LIN data transmission header using slot info and checks if the operation was successful.	6.62#open_lin_master_data_layer.c	Loc #133 - 134
CS-3.62#open_lin_master_data_layer.c	}	Handles a LIN master transmission header error by calling an error handler and setting the master state to idle.	6.62#open_lin_master_data_layer.c	Loc #121 - 150

ID	Feature Name	Description	Feature ID	Code Lines
CS-3.62#open_lin_master_data_layer.c	/* do nothing */	Manage LIN master states for data reception and transmission, handling timeouts and errors.	6.62#open_lin_master_data_layer.c	Loc #154 - 194
CS-3.63#open_lin_master_data_layer.c	open_lin_error_handler(OPEN LIN MASTER_ERROR_CHECKSUM);	Calls an error handling function four times, each time specifying a distinct LIN master error code.	6.63#open_lin_master_data_layer.c	N/A
CS-3.64#open_lin_master_data_layer.c	void open_lin_master_dl_init(t_master_frame_table_item *p_master_frame_table, l_u8 p_master_frame_table_size)	Initializes LIN master data link layer with frame table setup and entering idle state.	6.64#open_lin_master_data_layer.c	Loc #45 - 50
CS-3.65#open_lin_master_data_layer.c	static l_u8 master_table_index = 0;	Declare static variables to manage a master frame table, tracking current index and table size.	6.65#open_lin_master_data_layer.c	Loc #22 - 24
CS-3.65#open_lin_master_data_layer.c	static void data_layer_next_item(void);	Manages moving through and getting the current item in a data sequence.	6.65#open_lin_master_data_layer.c	Loc #28 - 29
CS-3.65#open_lin_master_data_layer.c	static t_master_frame_table_item* get_current_item(void)	Provides access to the current frame table item and advances the index cyclically.	6.65#open_lin_master_data_layer.c	Loc #52 - 66
CS-3.66#open_lin_master_data_layer.c	typedef enum {	Defines states and variables for managing a LIN master communication module.	6.66#open_lin_master_data_layer.c	Loc #14 - 21
CS-3.66#open_lin_master_data_layer.c	static l_u16 time_passed_since_last_frame_ms = 0;	Declares a static variable to track time since the last frame and a static function prototype for transitioning the LIN master to an idle state.	6.66#open_lin_master_data_layer.c	Loc #25 - 27
CS-3.66#open_lin_master_data_layer.c	static void open_lin_master_goto_idle(l_bool next_item)	Transitions LIN master to idle state, resets counters, and optionally advances data layer item.	6.66#open_lin_master_data_layer.c	Loc #31 - 42
CS-3.67#open_lin_master_data_layer.c	l_bool open_lin_master_dl_rx(l_u8 rx_byte)	Processes an incoming byte during LIN master data reception, buffering data and handling completion or errors.	6.67#open_lin_master_data_layer.c	Loc #91 - 118
CS-3.68#open_lin_master_data_layer.c	l_bool open_lin_master_data_tx_data(open_lin_frame_slot_t *slot)	Prepares data for transmission in a LIN master frame slot by iterating its data bytes.	6.68#open_lin_master_data_layer.c	Loc #78 - 83
CS-3.68#open_lin_master_data_layer.c		Calculates checksum, transmits it, and updates status based on transmission.	6.68#open_lin_master_data_layer.c	Loc #86 - 88

ID	Feature Name	Description	Feature ID	Code Lines
CS-3.69#open_lin_master_data_layer.c	l_bool open_lin_master_data_tx_header(open_lin_frame_slot_t *slot)	Transmits the break, sync byte, and protected ID for a LIN master frame header.	6.69#open_lin_master_data_layer.c	Loc #69 - 76
CS-3.70#open_lin_transport_layer.c	typedef struct {	Defines a structure to associate a LIN Service ID with a specific function for processing messages.	6.70#open_lin_transport_layer.c	Loc #16 - 19
CS-3.70#open_lin_transport_layer.c	/* static service handlers */	Defines a table mapping LIN service IDs to specific handler functions. Used for routing LIN requests to the correct processing logic.	6.70#open_lin_transport_layer.c	Loc #25 - 28
CS-3.71#open_lin_transport_layer.c	void open_lin_transport_layer_handler(open_lin_frame_slot_t *slot)	Handles different transport layer frames based on the frame's protocol identifier (PID) using a switch statement for specific message processing.	6.71#open_lin_transport_layer.c	Loc #89 - 92
CS-3.71#open_lin_transport_layer.c	default:	Handles cases not explicitly matched in a switch statement by performing no operation.	6.71#open_lin_transport_layer.c	Loc #89 - 105
CS-3.72#open_lin_transport_layer.c	/* assign NAD service implementation */	Processes a LIN frame to check if supplier and function IDs match the device's IDs and updates its Node Address (NAD) if they do.	6.72#open_lin_transport_layer.c	Loc #30 - 50
CS-3.73#open_lin_transport_layer.c	static void open_lin_transport_layer_request_handling(open_lin_frame_slot_t *slot)	Processes received LIN frames as transport layer requests, specifically Single Frame type, matching NAD and SID to call registered handlers.	6.73#open_lin_transport_layer.c	Loc #53 - 87
CS-3.74#open_lin_cfg.h	#define OPEN_LIN_HW_BREAK_IS_SYNCH_BYTE	Defines a preprocessor macro flag indicating LIN hardware break acts as a synchronization byte, enabling a specific configuration.	6.74#open_lin_cfg.h	Loc #17 - 17
CS-3.75#open_lin_cfg.h	#define OPEN_LIN_DYNAMIC_IDS	Defines a preprocessor symbol 'OPEN_LIN_DYNAMIC_IDS' used as a flag to enable or disable features related to dynamic IDs in the Open LIN protocol.	6.75#open_lin_cfg.h	Loc #21 - 21
CS-3.76#open_lin_cfg.h	#define PLATFORM_IS_BIG_ENDIAN	Defines a preprocessor macro indicating that the platform uses big-endian byte order.	6.76#open_lin_cfg.h	Loc #15 - 15
CS-3.77#open_lin_cfg.h	#define OPEN_LIN_TRANSPORT_LAYER	Defines a symbol to enable specific code blocks related to an Open LIN Transport Layer.	6.77#open_lin_cfg.h	Loc #24 - 24

ID	Feature Name	Description	Feature ID	Code Lines
CS-3.78#open_lin_cfg.h	#define OPEN_LIN_OPEN_LIN_CFG_H_	Defines a preprocessor macro, typically used for header guards in header files.	6.78#open_lin_cfg.h	Loc #8 - 9
CS-3.78#open_lin_cfg.h	#endif /* OPEN_LIN_OPEN_LIN_CFG_H_ */	This directive marks the end of a conditional block, usually preventing a header file from being included multiple times during compilation.	6.78#open_lin_cfg.h	Loc #26 - 26
CS-3.87#implementation/x86/main_slave_example.c	open_lin_frame_slot_t slot_array[] =	Initializes configurations for LIN frames (ID, type, size, buffer) and counts total configurations.	6.87#implementation/x86/main_slave_example.c	Loc #14 - 20
CS-3.88#implementation/x86/main_slave_example.c	l_u8 dataBuffer1[] = {0,0};	Declares and initializes two arrays of unsigned 8-bit integers (bytes) with specific values.	6.88#implementation/x86/main_slave_example.c	Loc #11 - 12
CS-3.89#implementation/x86/main_slave_example.c	void open_lin_on_rx_frame(open_lin_frame_slot_t *slot){	Prints details of a received LIN frame including its identifier and data bytes.	6.89#implementation/x86/main_slave_example.c	Loc #26 - 33
CS-3.90#implementation/x86/main_slave_example.c	int main()	Initializes hardware and network layers for an OPEN_LIN slave device and then enters an infinite loop.	6.90#implementation/x86/main_slave_example.c	Loc #41 - 54
CS-3.91#implementation/x86/main_slave_example.c	void rx_byte_handle(uint8_t byte)	Intended to process a received byte, but currently does nothing.	6.91#implementation/x86/main_slave_example.c	Loc #35 - 36
CS-3.91#implementation/x86/main_slave_example.c	open_lin_slave_rx_header(byte);	Processes a single byte received as part of a LIN slave header.	6.91#implementation/x86/main_slave_example.c	Loc #38 - 39
CS-3.92#implementation/x86/main_slave_example.c	void open_lin_master_dl_rx_callback(open_lin_frame_slot_t* slot){	Placeholder callback function for LIN master data reception, indicating it's incorrectly used in a slave setup.	6.92#implementation/x86/main_slave_example.c	Loc #22 - 24
CS-3.99#implementation/ST32/open_lin_hw_st32.c	l_bool open_lin_hw_check_for_break(void)	Checks if the UART1 LIN Break Detection Flag is set, clears it if set, and returns the status.	6.99#implementation/ST32/open_lin_hw_st32.c	Loc #23 - 32
CS-3.100#implementation/ST32/open_lin_hw	void open_lin_frame_set_auto_baud(void)	Configures system or hardware for LIN communication, potentially enabling auto baud	6.100#implementation/ST32/open_lin_hw_st32.c	Loc #34 - 35

ID	Feature Name	Description	Feature ID	Code Lines
_st32.c		rate detection.		
CS-3.100#implementation/ST32/open_lin_hw_st32.c	}		6.100#implementation/ST32/open_lin_hw_st32.c	Loc #21 - 21
CS-3.101#implementation/ST32/open_lin_hw_st32.c	void open_lin_set_rx_enabled(l_bool v)	Configures the LIN receiver based on input, either resetting hardware or disabling UART interrupts.	6.101#implementation/ST32/open_lin_hw_st32.c	Loc #48 - 57
CS-3.102#implementation/ST32/open_lin_hw_st32.c	void open_lin_error_handler(t_open_lin_error error_code)	This function is intended to handle LIN errors but currently performs no actions. It's a placeholder error handler.	6.102#implementation/ST32/open_lin_hw_st32.c	Loc #18 - 21
CS-3.103#implementation/ST32/open_lin_hw_st32.c	}	No code provided for analysis.	6.103#implementation/ST32/open_lin_hw_st32.c	Loc #21 - 21
CS-3.104#implementation/ST32/open_lin_hw_st32.c	void open_lin_hw_reset(void) {	Initializes hardware for LIN communication, setting up break detection and enabling interrupt-driven reception.	6.104#implementation/ST32/open_lin_hw_st32.c	Loc #39 - 45
CS-3.105#implementation/ST32/open_lin_hw_st32.c	if (HAL_UART_Transmit(&huart1, data, len, 1000) == HAL_OK)	Sends data using UART and returns true if transmission succeeds.	6.105#implementation/ST32/open_lin_hw_st32.c	Loc #75 - 77
CS-3.105#implementation/ST32/open_lin_hw_st32.c	}	No code provided for analysis.	6.105#implementation/ST32/open_lin_hw_st32.c	Loc #21 - 21
CS-3.106#implementation/ST32/open_lin_hw_st32.c	l_bool open_lin_hw_tx_break(void){	Attempts to send a LIN break signal via hardware and reports if successful.	6.106#implementation/ST32/open_lin_hw_st32.c	Loc #66 - 71
CS-3.107#implementation/ST32/open_lin_hw_st32.c	l_bool open_lin_hw_tx_byte(l_u8 byte){	Sends a single byte of data using the UART hardware and checks if transmission succeeded.	6.107#implementation/ST32/open_lin_hw_st32.c	Loc #59 - 64
CS-3.108#implementation/unit_test/test_run.cpp	const l_u8 lenght_of_slot_array = sizeof(slot_array) / sizeof(open_lin_frame_slot_t);	Calculates the number of elements in the 'slot_array' by dividing its total size by the size of one element.	6.108#implementation/unit_test/test_run.cpp	Loc #28 - 28

ID	Feature Name	Description	Feature ID	Code Lines
CS-3.109#implementation/unit_test/test_run.cpp	open_lin_frame_slot_t slot_array[] =	Initializes an array defining configurations for LIN communication frame slots.	6.109#implementation/unit_test/test_run.cpp	Loc #20 - 25
CS-3.110#implementation/unit_test/test_run.cpp	extern l_u8 open_lin_frame_parity(l_u8 pid);	Calculates parity byte for a LIN frame PID.	6.110#implementation/unit_test/test_run.cpp	Loc #13 - 14
CS-3.111#implementation/unit_test/test_run.cpp	extern l_bool sim_break;	Declares external simulation control and error handling elements, including a break flag and error status function.	6.111#implementation/unit_test/test_run.cpp	Loc #60 - 63
CS-3.112#implementation/unit_test/test_run.cpp	l_bool tx_invalid = false;	Declares and initializes boolean variables representing transaction state status flags.	6.112#implementation/unit_test/test_run.cpp	Loc #101 - 102
CS-3.113#implementation/unit_test/test_run.cpp	l_u8 dataBuffer1[] = {0,0,0,0};	This code initializes three arrays of unsigned 8-bit integers with specific sizes and predefined values.	6.113#implementation/unit_test/test_run.cpp	Loc #16 - 18
CS-3.114#implementation/unit_test/test_run.cpp		Includes necessary header files for the Open LIN library components.	6.114#implementation/unit_test/test_run.cpp	Loc #5 - 11
CS-3.115#implementation/unit_test/test_run.cpp	#define CATCH_CONFIG_MAIN // This tells Catch to provide a main() - only do this in one cpp file	Sets up the Catch2 testing framework to generate a main executable for tests.	6.115#implementation/unit_test/test_run.cpp	Loc #1 - 3
CS-3.116#implementation/unit_test/test_run.cpp	l_bool open_lin_hw_tx_data(l_u8* data, l_u8 len){	Validates incoming LIN data against an expected buffer or checksum byte, setting transmission status flags (`tx_invalid`, `tx_done`) accordingly.	6.116#implementation/unit_test/test_run.cpp	Loc #106 - 132
CS-3.117#implementation/unit_test/test_run.cpp	open_lin_id_translation_item_t open_lin_id_translation_tab[] = {	Tests LIN slave dynamic ID translation and reception of data based on translated ID.	6.117#implementation/unit_test/test_run.cpp	Loc #208 - 242
CS-3.118#implementation/unit_test/test_run.cpp	TEST_CASE("calc_checksum", "[open_lin_data_layer]") {	Tests the checksum calculation for LIN data frames with different data sets.	6.118#implementation/unit_test/test_run.cpp	Loc #42 - 56
CS-3.119#implementation/unit_test/test_run.cpp	open_lin_slave_rx_header(0x92); //	Tests LIN slave error handling by processing	6.119#implementation/unit_test/test_run.cpp	Loc #154 - 168

ID	Feature Name	Description	Feature ID	Code Lines
ion/unit_test/test_run.cpp	0x01	various headers and checking for an invalid checksum error.	t_run.cpp	
CS-3.120#implementation/unit_test/test_run.cpp	open_lin_slave_rx_header(0x93); // 0x01	Tests LIN slave header reception expecting a specific parity error after receiving PID 0x93.	6.120#implementation/unit_test/test_run.cpp	Loc #144 - 152
CS-3.121#implementation/unit_test/test_run.cpp	TEST_CASE("calc_parity", "[open_lin_data_layer]") {	Test case verifying open_lin_data_layer_parity function calculates parity correctly for different inputs.	6.121#implementation/unit_test/test_run.cpp	Loc #33 - 40
CS-3.122#implementation/unit_test/test_run.cpp	TEST_CASE("slave break detection", "[open_lin_slave]") {	This code block sets up an Open LIN network using provided slot configuration data and then resets the slave device's state, preparing it for testing break detection scenarios.	6.122#implementation/unit_test/test_run.cpp	Loc #65 - 68
CS-3.122#implementation/unit_test/test_run.cpp	{	Checks for and clears a simulation error, asserting that no error was reported by the simulator.	6.122#implementation/unit_test/test_run.cpp	Loc #70 - 74
CS-3.122#implementation/unit_test/test_run.cpp	#define CATCH_CONFIG_MAIN // This tells Catch to provide a main() - only do this in one cpp file	Configures the Catch2 test framework to automatically provide the main function for executing unit tests.	6.122#implementation/unit_test/test_run.cpp	Loc #1 - 1
CS-3.122#implementation/unit_test/test_run.cpp	sim_break = true;	Simulates a LIN break condition and processes a LIN header, testing error handling.	6.122#implementation/unit_test/test_run.cpp	Loc #78 - 83
CS-3.122#implementation/unit_test/test_run.cpp	l_u8 dataBuffer1[] = {0,0,0,0};	Defines an array named dataBuffer1 with four unsigned 8-bit integer elements initialized to zero.	6.122#implementation/unit_test/test_run.cpp	Loc #16 - 16
CS-3.123#implementation/unit_test/test_run.cpp	TEST_CASE("reset test", "[open_lin_slave]") {	Tests the reset functionality of the open LIN slave component.	6.123#implementation/unit_test/test_run.cpp	Loc #94 - 99
CS-3.124#implementation/unit_test/test_run.cpp	TEST_CASE("frame reception, tx header", "[open_lin_slave]") {	Tests LIN slave receiving a valid header (break, sync, PID) and verifying state.	6.124#implementation/unit_test/test_run.cpp	Loc #134 - 142
CS-3.125#implementation/unit_test/test_run.cpp	TEST_CASE("diagnostic frame	Test verifying LIN slave correctly processes a	6.125#implementation/unit_test/test_run.cpp	Loc #170 - 183

ID	Feature Name	Description	Feature ID	Code Lines
ion/unit_test/test_run.cpp	reception, rx header valid checksum", "[open_lin_slave]") {	diagnostic frame header with valid checksum without reporting errors.	t_run.cpp	
CS-3.125#implementation/unit_test/test_run.cpp	l_u8 dataBuffer1[] = {0,0,0,0};	Declares a data buffer array of 4 unsigned bytes and initializes all its elements to zero, preparing it for use.	6.125#implementation/unit_test/test_run.cpp	Loc #16 - 16
CS-3.126#implementation/unit_test/test_run.cpp	open_lin_slave_reset();	Simulates a LIN slave receiving a data frame and verifies the received data and error state.	6.126#implementation/unit_test/test_run.cpp	Loc #187 - 206
CS-3.132#implementation/unit_test/open_lin_frame_winOS.c	t_open_lin_error sim_handler_called = OPEN_LIN_NO_ERROR;	Gets the current simulation error status and then clears it.	6.132#implementation/unit_test/open_lin_frame_winOS.c	Loc #12 - 19
CS-3.132#implementation/unit_test/open_lin_frame_winOS.c	void open_lin_error_handler(t_open_lin_error error_code)	Records an error code received from an Open LIN system for simulation/debugging purposes, performing no complex error handling.	6.132#implementation/unit_test/open_lin_frame_winOS.c	Loc #26 - 30
CS-3.133#implementation/unit_test/open_lin_frame_winOS.c	void open_lin_hw_reset(void) {	Resets a transmit index variable to zero, likely preparing for new data.	6.133#implementation/unit_test/open_lin_frame_winOS.c	Loc #75 - 77
CS-3.134#implementation/unit_test/open_lin_frame_winOS.c	l_bool sim_break = false;	Checks a simulated hardware break flag, resets it, and returns the flag's state before the reset.	6.134#implementation/unit_test/open_lin_frame_winOS.c	Loc #32 - 38
CS-3.135#implementation/unit_test/open_lin_frame_winOS.c	open_lin_frame_slot_t* last_rx_slot;	Stores pointer to last received LIN frame slot data.	6.135#implementation/unit_test/open_lin_frame_winOS.c	Loc #79 - 83
CS-3.136#implementation/unit_test/open_lin_frame_winOS.c	uint8_t lasttx[255];	Manage a buffer and index for tracking transmitted data.	6.136#implementation/unit_test/open_lin_frame_winOS.c	Loc #40 - 53
CS-3.137#implementation/unit_test/open_lin_frame_winOS.c	l_bool open_lin_hw_tx_break(void){	Records a zero value in an array, increments index if within bounds, and always returns success.	6.137#implementation/unit_test/open_lin_frame_winOS.c	Loc #55 - 60
CS-3.138#implementation/unit_test/open_lin_frame_winOS.c	l_bool open_lin_hw_tx_byte(l_u8 byte)	Stores a byte in a transmit buffer array and advances the buffer index, indicating success.	6.138#implementation/unit_test/open_lin_frame_winOS.c	Loc #67 - 73

ID	Feature Name	Description	Feature ID	Code Lines
rame_winOS.c				
CS-3.139#implementation/unit_test/open_lin_frame_winOS.c	void open_lin_set_rx_enabled(l_bool v)	Sets the LIN receiver enabled state based on input value, but currently does nothing.	6.139#implementation/unit_test/open_lin_frame_winOS.c	Loc #21 - 24
CS-3.139#implementation/unit_test/open_lin_frame_winOS.c	void open_lin_frame_set_auto_baud(void)	Intended to open a LIN frame and set baud automatically, but currently empty.	6.139#implementation/unit_test/open_lin_frame_winOS.c	Loc #62 - 65
CS-3.140#implementation/pic16/open_lin_hw_pic16.c	l_bool open_lin_hw_check_for_break(void)	Checks LIN hardware for a framing error (potential break condition), clears the error, and reports detection.	6.140#implementation/pic16/open_lin_hw_pic16.c	Loc #43 - 53
CS-3.141#implementation/pic16/open_lin_hw_pic16.c	#define LIN_EUSART_DataReady EUSART1_DataReady	Defines macros to alias hardware peripherals (USART, Timer) for use in LIN communication.	6.141#implementation/pic16/open_lin_hw_pic16.c	Loc #17 - 32
CS-3.142#implementation/pic16/open_lin_hw_pic16.c	void open_lin_set_rx_enabled(l_bool v)	This function enables or disables the LIN receiver. Enabling it performs a hardware reset.	6.142#implementation/pic16/open_lin_hw_pic16.c	Loc #66 - 71
CS-3.142#implementation/pic16/open_lin_hw_pic16.c	{	Configures a LIN module to enable receiving and receive interrupts.	6.142#implementation/pic16/open_lin_hw_pic16.c	Loc #66 - 76
CS-3.143#implementation/pic16/open_lin_hw_pic16.c	void open_lin_error_handler(t_open_lin_error error_code)	Handles LIN errors based on input code, but currently performs no action or logging.	6.143#implementation/pic16/open_lin_hw_pic16.c	Loc #38 - 41
CS-3.144#implementation/pic16/open_lin_hw_pic16.c	void open_lin_hw_reset(void) {	Resets LIN hardware by initiating a check for a break signal.	6.144#implementation/pic16/open_lin_hw_pic16.c	Loc #60 - 63
CS-3.145#implementation/pic16/open_lin_hw_pic16.c	void open_lin_frame_set_auto_baud(void)	Initializes a LIN communication frame and prepares for automatic baud rate detection.	6.145#implementation/pic16/open_lin_hw_pic16.c	Loc #55 - 56
CS-3.146#implementation/pic16/open_lin_hw_pic16.c	open_lin_NAD_t open_lin_NAD = 0;	Initializes variables related to LIN communication node addressing and identification numbers to zero.	6.146#implementation/pic16/open_lin_hw_pic16.c	Loc #34 - 36

ID	Feature Name	Description	Feature ID	Code Lines
CS-3.147#implementation/pic16/open_lin_hw_pic16.c	I_bool open_lin_hw_tx_data(I_u8* data, I_u8 len)	Transmits an array of data bytes over LIN hardware by sending each byte one by one using a helper function.	6.147#implementation/pic16/open_lin_hw_pic16.c	Loc #85 - 93
CS-3.148#implementation/pic16/open_lin_hw_pic16.c	I_bool open_lin_hw_tx_byte(I_u8 byte){	Transmits a single byte of data using the LIN EUSART hardware peripheral for communication.	6.148#implementation/pic16/open_lin_hw_pic16.c	Loc #78 - 83